



2024-2025

# Library Management System Project



Oluwatobiloba Ayodele

## Table of Contents

|                                                   |    |
|---------------------------------------------------|----|
| Analysis.....                                     | 3  |
| Suitability for Computational Method.....         | 3  |
| Stakeholder Description and Requirements.....     | 4  |
| Stakeholder 1.....                                | 4  |
| Stakeholder 2.....                                | 5  |
| Stakeholder 3.....                                | 6  |
| Stakeholder Requirements.....                     | 7  |
| Research.....                                     | 8  |
| Example 1 Key Features of Interest.....           | 8  |
| Example 2 Review: Southend Libraries system ..... | 9  |
| Example 2 Key Features of Interest.....           | 9  |
| Example 3 Review: Essex libraries Mobile App..... | 11 |
| Example 3 Key Features of Interest.....           | 11 |
| Features for Computational Solution.....          | 13 |
| Limitations.....                                  | 15 |
| Software and Hardware Requirements .....          | 15 |
| Success Criteria.....                             | 16 |
| Design.....                                       | 17 |
| Decomposition Diagram and Explanation.....        | 17 |
| Structure Diagram and Explanation.....            | 20 |
| Key Algorithms.....                               | 22 |
| Usability and User Interface .....                | 28 |
| Key Variables, Classes, and Data Structures.....  | 30 |
| Validation.....                                   | 35 |
| Testing Plan for Development Cycle.....           | 35 |
| Post Development Testing with Required Data.....  | 40 |
| Development .....                                 | 42 |
| Development Cycle Part 1.....                     | 42 |
| Code Development.....                             | 42 |
| Testing Table and Evidence.....                   | 45 |
| Key Stage Review .....                            | 46 |
| Development Cycle Part 2.....                     | 48 |
| Code Development.....                             | 48 |
| Validation .....                                  | 53 |
| Testing Table and Evidence.....                   | 53 |
| Key Stage Review .....                            | 57 |
| Development Cycle Part 3.....                     | 59 |

|                                                                                      |     |
|--------------------------------------------------------------------------------------|-----|
| Code Development.....                                                                | 59  |
| Testing Table and Evidence.....                                                      | 67  |
| Key Stage Review .....                                                               | 69  |
| Development Cycle Part 4.....                                                        | 71  |
| Code Development.....                                                                | 71  |
| Testing Table and Evidence.....                                                      | 76  |
| Key Stage Review .....                                                               | 78  |
| Development Cycle Part 5.....                                                        | 79  |
| Code Development.....                                                                | 79  |
| Testing Table and Evidence.....                                                      | 89  |
| Key Stage Review .....                                                               | 97  |
| Testing of Final Solution for Evaluation .....                                       | 98  |
| Testing of final solution for function, robustness, and usability with evidence..... | 98  |
| Stakeholder feedback.....                                                            | 107 |
| Evaluation.....                                                                      | 110 |
| Success Criteria Evaluation and Test Evidence.....                                   | 110 |
| Partial/Unmet Criteria and Future Work .....                                         | 111 |
| Evidence of Usability Features .....                                                 | 111 |
| Partially or unmet Usability Features and further development.....                   | 113 |
| Maintenance and Limitations .....                                                    | 114 |
| Algorithm Critique.....                                                              | 114 |
| Future Development.....                                                              | 114 |
| Full Code .....                                                                      | 116 |

## Analysis

### Suitability for Computational Method

A new library has opened at a local sixth form college and as such they require an efficient and reliable library management system in response to daily activities. As the library grows and more students and staff interact with the system, the volume of activities will significantly grow such as the logging of loans and returns and tracking of inventory. This means that as a result, manually managing these tasks using paper records would be too time-consuming and would likely lead to errors and data inconsistencies. As well as this, manual systems also lack the ability to search or filter books which may hinder users from finding the books they wish to read.

To address this issue, my project presents a more scalable solution using a computerised library management system. It would be able to automate several of the repetitive tasks that librarians would have to do, such as managing stock levels and loaning and returning books. It will also be able to provide functionality to searching and filtering of books so users can find new recommendations for books which they like. This is increasingly valuable for supporting the educational environment of the sixth form college as it can help to ensure that students have consistent access to learning resources and can get access to recommendations to widen their independent learning.

My project will involve the processing of data of several hundred books and several hundred library members meaning SQL is an efficient tool to use as it can handle a large and versatile amount of data as well as being able to process complex queries quickly. Python is as an efficient use as it can also process a large amount of data quickly and it is easy to implement efficient sorting algorithms as well as being a programming language with which I am familiar. I am also able to include a graphical interface with the help of the Tkinter Python module making it easier for the end user to use. This would be significantly more effective than a physical database, as it would be significantly harder and time-consuming to order all books and members by hand. They are also more portable as SQL databases can be easily exported and imported allowing the user to monitor books and members from anywhere in the library. It would also be more secure and allow for multiple backups.

## Stakeholder Description and Requirements

| Stakeholder Number | Stakeholder Name          | Stakeholder Occupation | Stakeholder Gender |
|--------------------|---------------------------|------------------------|--------------------|
| 1                  | Mrs J Oats                | Librarian              | Female             |
| 2                  | Theodore Warwick Ebenezer | Student                | Male               |
| 3                  | Jeremiah Wilkinson        | Student                | Male               |

As well as creating a profile of each of my stakeholders, I conducted a stakeholder survey with my three main stakeholders to help develop requirements for the system. I asked five questions to have a better idea of their requirements:

1. How often would you use the Library System?
2. How would you use the Library System?
3. What problems have you faced with previous library systems?
4. What requirements would you need for a new system?
5. Are there any accessibility needs to consider for a new system?

### Stakeholder 1

Mrs J Oats is a Librarian at a newly established library where the project will be tested. Throughout her experience as a librarian, she has an extensive amount of experience using several library organisation systems, however, is looking for a new system to be developed at her new library to help to streamline the management process of the library. She is also experienced in the DDC library organisational system and aims to use the program to help manage book loans as well as the availability of certain checked in and out books.

Her main responsibilities include checking in and out library books, managing the members of the system as well as loans. She also uses the system to chase up overdue library books and organise the library.

Mrs Ots has limited availability, especially out of library hours. However, meetings can be arranged to gain feedback throughout the development process, with the opportunity for development feedback and evaluation through interviews to ensure she is satisfied with the final product.

### Stakeholder 1 Survey

|                                                                           |                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><u>How often would you use the library system?</u></b>                 | I would use the system daily within school hours. It is essential for my job and would prevent me from doing my work effectively if it does not work or if there are issues.                                                                                |
| <b><u>How would you use the library system?</u></b>                       | I would use the system daily to catalogue books, to issue books and mark them as returned. I would also use it to add new books to the library database and remove outdated or damaged ones. I would also use the system to add and remove library members. |
| <b><u>What problems have you faced with previous library systems?</u></b> | Some systems I have used in the pass have been too complex or not user friendly. I have also encountered limitations in scalability of the program in terms of many books or users.                                                                         |

|                                                                               |                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><u>What requirements would you need for a new system?</u></b>              | Core features of a library management system are necessary, being cataloguing, loaning and returning books however a new system would also benefit from a modern user interface that is not too complex. |
| <b><u>Are there any accessibility needs to consider for a new system?</u></b> | The system needs to use a readable font, and at a large enough font size for me to easily read it and not strain my eyes for long periods of time.                                                       |

## Stakeholder 2

Theodore Warwick Ebenezer (Theo) is an A-Level Student who studies Advanced Level Economics, Computer Science and History. He typically uses libraries to read up on his studies particularly History and Economics and is excited for the new library which is opening near him. He mainly uses a library to enhance his subject knowledge during his independent learning. His main interactions with a library system are going to the librarian to check books in and out as well as managing his overdue loans. It is also important to note that he is a student so can be spoken to in formal and informal settings but has important exams which may negatively affect his availability.

## Stakeholder 2 Survey

|                                                                               |                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><u>How often would you use the library system?</u></b>                     | Fairly frequently, at least once a week if my schedule allows and depending on whether books or articles, I would like to read are available.                                                       |
| <b><u>How would you use the library system?</u></b>                           | I would use the library system to reserve books, and check which books are in stock or are already reserved, particularly for books in my area of interest (Economics and History).                 |
| <b><u>What problems have you faced with previous library systems?</u></b>     | With some systems that I have used before, it can be difficult to see what books are available and what I would want (like what genre/topic to look for).                                           |
| <b><u>What requirements would you need for a new system?</u></b>              | It must be easy and fast to use, able to be sorted to see what books I can get related to what I would be interested in, easy to monitor what books I have taken out and when they are due back in. |
| <b><u>Are there any accessibility needs to consider for a new system?</u></b> | I have some colour blindness (protanopia) so there should be accessibility features, so I am still able to use the system without difficulty.                                                       |

### Stakeholder 3

Jeremiah Wilkinson is also an A-Level student studying Maths, Further Maths and Computer Science. However, Jeremiah is an avid reader of non-fiction and fiction books and so this would be his main reason for interacting with the system. He indicates that his main interaction with the system would be to check out and return books and check his loans. Like Stakeholder 2, it is important to consider that while he is a student and so is available for informal interviews he must also prepare for his A-Level Examinations.

### Stakeholder 3 Survey

|                                                                               |                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><u>How often would you use the library system?</u></b>                     | Once a month or so to return a new book and check out a new one.                                                                                                                                                           |
| <b><u>How would you use the library system?</u></b>                           | I would use the library system to check in and out new books as well as look for new books from my favourite author or books in my favourite genre.                                                                        |
| <b><u>What problems have you faced with previous library systems?</u></b>     | Systems can be very slow meaning I must wait in front of the librarian for long periods of time while they scan the books both in and out. Also, some systems have inaccurate information about the availability of books. |
| <b><u>What requirements would you need for a new system?</u></b>              | It must be easy and fast to use, able to be sorted to see what books I can get related to what I would be interested in, easy to monitor what books I have taken out and when they are due back in.                        |
| <b><u>Are there any accessibility needs to consider for a new system?</u></b> | I have a visual impairment meaning I need text to be high contrast so I can see it more easily.                                                                                                                            |

## Stakeholder Requirements

| Requirement                                                                                                                                    | Success Criteria                                                                                                                                                                                                          | Importance<br>(Rating between 1 and 5) | Completed (to be completed in evaluation) |
|------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|-------------------------------------------|
| A clean, simple to use GUI                                                                                                                     | The GUI should include Intuitive navigation and should be consistent, making it easy for users to understand how to use it. It should also avoid clutter. This should be evidenced by positive user feedback              | 4                                      |                                           |
| The user should be able to monitor what books they have taken out and how long they have until they must return it.                            | This would include having a user system, which would link the books they have checked out to the user as well as logging their check out date.                                                                            | 5                                      |                                           |
| New books and old books and media should be able to be catalogued and removed and media types should have their associated information logged. | A database should be created and easily editable containing all books and information: author, release date ISBN etc.                                                                                                     | 5                                      |                                           |
| Should be able to search for different books                                                                                                   | A working search function for books                                                                                                                                                                                       | 4                                      |                                           |
| Should be able to sort the books into different categories.                                                                                    | Books should be able to be sorted into different categories, such as genre, author, release date etc.                                                                                                                     | 4                                      |                                           |
| Books should be able to be checked in and out                                                                                                  | Users must be able to check in and out books, with a loan date and return date being generated. The same book should not be able to be checked out multiple times, unless there are multiple quantities of the same book. | 5                                      |                                           |

## Research

### Example I Review

#### Example I: Example School Library Overdue Books Database

| 1  | Title                                                   | Author                                      | Name                 | Due        | Days Over Charge | Barcode                |
|----|---------------------------------------------------------|---------------------------------------------|----------------------|------------|------------------|------------------------|
| 2  | Harry Potter and the Order of the Phoenix               | Rowling, J.K                                | Lella, Karthikeya    | 28/05/2021 | 1113             | £0.00 51435            |
| 3  | Echo Mountain                                           | Wolk, Lauren                                | Lella, Karthikeya    | 28/05/2021 | 1113             | £0.00 51493            |
| 4  | On midnight beach                                       | Fitzpatrick, Marie-Louise                   | McGowan, Stacie      | 04/06/2021 | 1106             | £0.00 51464            |
| 5  | The fountains of silence                                | Sepetys, Ruta                               | McGowan, Stacie      | 21/07/2021 | 1059             | £0.00 51490            |
| 6  | Run, rebel                                              | Mann, Manjeet                               | McGowan, Stacie      | 21/07/2021 | 1059             | £0.00 51472            |
| 7  | Stalin: Man and Ruler                                   | McNeal, Robert H                            | Vlas, Eugene         | 24/09/2021 | 994              | £0.00 7720             |
| 8  | Russia Under the Bolshevik Regime 1919-1924             | Pipes, Richard                              | Vlas, Eugene         | 24/09/2021 | 994              | £0.00 14016            |
| 9  | Germany and Austria, 1814-2000                          | Allinson, Mark                              | Vlas, Eugene         | 24/09/2021 | 994              | £0.00 18594            |
| 10 | Noughts & crosses                                       | Blackman, Malorie                           | Marshall, Stanley    | 27/09/2021 | 991              | £0.00 50850            |
| 11 | DESIGN AND TECHNOLOGY CLASSBOOK KEY STAGE 3             | Bull, Peter                                 | Chekuri, Suhas Varma | 28/09/2021 | 990              | £0.00 16077            |
| 12 | Real-life mysteries : Can you explain the unexplained?  | Martineau, Susan; Barker, Vicky             | Lynch-Wyatt, Conor   | 28/09/2021 | 990              | £0.00 50373            |
| 13 | Assassination of John F Kennedy, The                    | Hoare, Stephen                              | Lynch-Wyatt, Conor   | 28/09/2021 | 990              | £0.00 2161             |
| 14 | World Leaders Era of the Second World War               | Ross, Stewart                               | Lynch-Wyatt, Conor   | 28/09/2021 | 990              | £0.00 10932            |
| 15 | Joy of Nature                                           |                                             | Parsons, Lennon      | 01/10/2021 | 987              | £0.00 17039            |
| 16 | The hobbit                                              | Tolkien, J.R.R                              | Walton, Tobias       | 05/10/2021 | 983              | £0.00 50027            |
| 17 | Greek Legends                                           | Deary, Terry                                | Achogbuo, Samuel     | 06/10/2021 | 982              | £0.00 17729            |
| 18 | Gods & Men Myths & Legends from the World's Religions   | Bailey, John; McLeish, Kenneth; Spearman, D | Achogbuo, Samuel     | 06/10/2021 | 982              | £0.00 12802            |
| 19 | Greek myths /                                           | Graves, Robert, 1895-1985                   | Achogbuo, Samuel     | 06/10/2021 | 982              | £0.00 A00000000021747A |
| 20 | L'homme au bras coupe Lire le Français                  | Laporte, Michel                             | Ahmad, Hassan        | 07/10/2021 | 981              | £0.00 11273            |
| 21 | How to Score - Science and the Beautiful Game           | Bray, Ken                                   | Poynter, Daniel      | 07/10/2021 | 981              | £0.00 A00000000021067A |
| 22 | The knife of never letting go                           | Ness, Patrick                               | Walton, Isaac        | 07/10/2021 | 981              | £0.00 51286            |
| 23 | Mandela: A Biography                                    | Meredith, Martin                            | Chekuri, Suhas Varma | 12/10/2021 | 976              | £0.00 51437            |
| 24 | Dreams from my father : a story of race and inheritance | Obama, Barack                               | Lynch-Wyatt, Conor   | 12/10/2021 | 976              | £0.00 51260            |
| 25 | City of Glass                                           |                                             | Pocock, Dylan        | 12/10/2021 | 976              | £0.00 21716            |
| 26 | To kill a mockingbird                                   | Lee, Harper                                 | Pliszka, Patrick     | 18/10/2021 | 970              | £0.00 50990            |
| 27 | Clap When you Land                                      | Acevedo, Elizabeth                          | Fakunle, Michael     | 02/11/2021 | 955              | £0.00 51440            |
| 28 | Barney British Empire                                   | Deary, Terry; Brown, Martin                 | Gupta, Alesh         | 02/11/2021 | 955              | £0.00 51977            |
| 29 | Check mates                                             | Foster, Stewart                             | Walton, Isaac        | 04/11/2021 | 953              | £0.00 50022            |
| 30 | Careers in dentistry : your questions and answers /     | Trotman; Higgins, Nick                      | Khan, Musa           | 16/11/2021 | 941              | £0.00 A00000000021004A |
| 31 | Second Foundation                                       | Asimov, Isaac                               | Fatai, Temitayo      | 23/11/2021 | 934              | £0.00 51752            |
| 32 | The maze runner                                         | Dashner, James                              | Chowdhury, Ibrahim   | 23/11/2021 | 934              | £0.00 52345            |
| 33 | The fault in our stars                                  | Green, John                                 | Francis, Tobey       | 06/12/2021 | 921              | £0.00 51398            |
| 34 | The hobbit, or, There and back again                    | Tolkien, J.R.R; Wyatt, David                | Herman, George       | 06/12/2021 | 921              | £0.00 51112            |
| 35 | A promised land                                         | Obama, Barack                               | Mosabbir, Wadi       | 08/12/2021 | 919              | £0.00 52539            |
| 36 | The Psychopath Test                                     | Ronson, Jon                                 | Mawela, Tino         | 08/12/2021 | 919              | £0.00 52310            |
| 37 | The selfish gene /                                      | Dawkins, Richard, 1941-                     | Mawela, Tino         | 08/12/2021 | 919              | £0.00 A00000000021902A |
| 38 | After the war                                           | Palmer, Tom                                 | Khan, Zidan          | 10/12/2021 | 917              | £0.00 51813            |
| 39 | Malamander                                              | Taylor, Thomas                              | Chekuri, Suhas Varma | 05/01/2022 | 891              | £0.00 50980            |

This is an example spreadsheet that one of my stakeholders ‘Mrs Oats’ has used in the past uses to chase up overdue library books as well as to train up junior Librarians. It is a singular Excel spreadsheet with columns dictating: the title of the book, its author, the loanee, the date it was loaned out, how many days since it was loaned out, their overdue fine amount as well as the internal barcode for the associated book. This is relevant to my project as it links directly to one of my requirements, as it allows the librarians to manage overdue books.

#### Example I Key Features of Interest

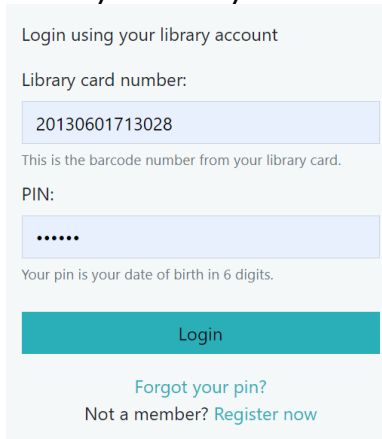
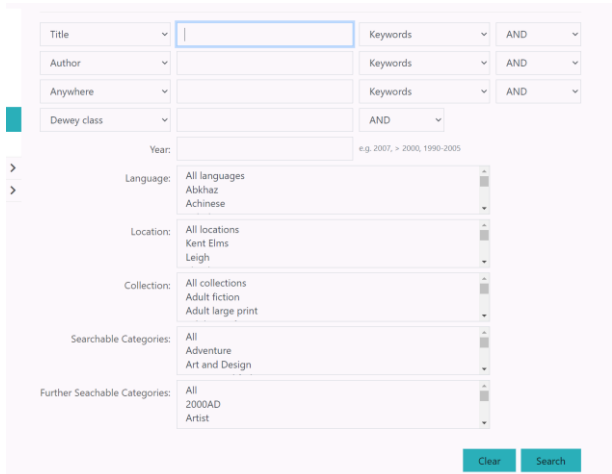
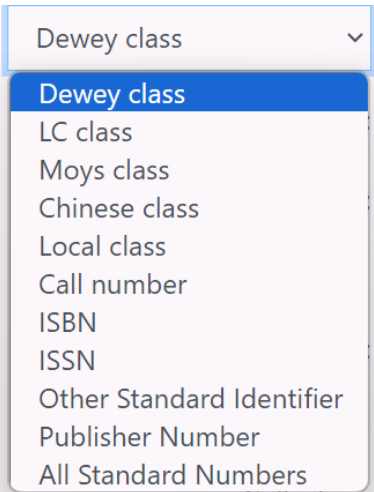
| Feature                                                                                                                                            | Why is it of interest                                                                                                                                                                                                                                                                                                  | Relevant to my work? |
|----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| Each book is given a 5-digit identifying barcode, however not all of them have been standardised (beginning with A followed by a string of zero's) | This makes it easier to identify books, rather than using its ISBN which has a longer number of digits. This system also means different types of media such as movies as well as allowing for multiple copies of the same book -which would have the same ISBN number but different barcode numbers- to be catalogued | Yes                  |
| Organised as one large spreadsheet table with 300 records.                                                                                         | This makes it harder to search for specific books which are missing, and one is unable to order by genre, loanee, or author example.                                                                                                                                                                                   | Yes                  |
| There is no way for the librarians to check out books.                                                                                             | This means that the librarians have to manually enter books which are overdue and delete records when they returned meaning they have to have an up-to-date copy of the spreadsheet. at all times                                                                                                                      | Yes                  |

## Example 2 Review: Southend Libraries system

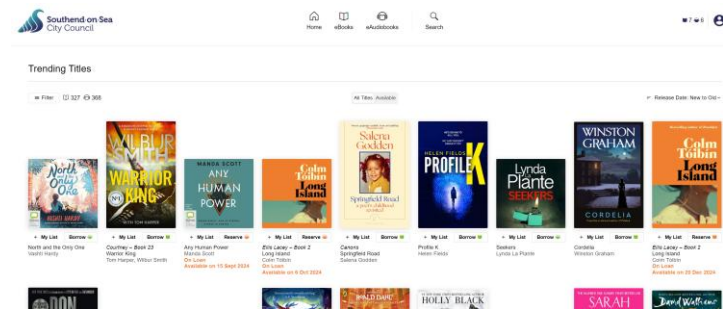
This is the online system that Southend Libraries users use to check books in and out and reserve them at their local library as well as checking their current loans from their home. It can be found at: <https://southend.spydus.co.uk/>.

This is very relevant to my project as it fulfils many of requirements such as allowing users to check books in and out as well as manage their overdue loans.

## Example 2 Key Features of Interest

| Feature                                                                                                                                                                                                                                  | Why is it of interest                                                                                                                                                                                                                                   | Relevant to my work? |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| <b>A library account system</b>                                                                                                                         | <p>This allows the user to log into their account so they can check in and out books as well as reserve them. Users are given a 16-digit pin as their library card number and a 6 digit number which is their date of birth to allow them to login.</p> | Yes                  |
| <b>Advanced search allowing the user to specify keywords surround the book as well as the title author, and the libraries that have it in stock.</b>  | <p>It allows the user to specify keywords surround the book as well as the title author, and the libraries that have it in stock.</p>                                                                                                                   | Yes                  |
| <b>Classification methods</b>                                                                                                                         | <p>Allows the user to organise the outputted books using different classification methods such as the Dewey Decimal System or the Library of Congress system.</p>                                                                                       | No                   |

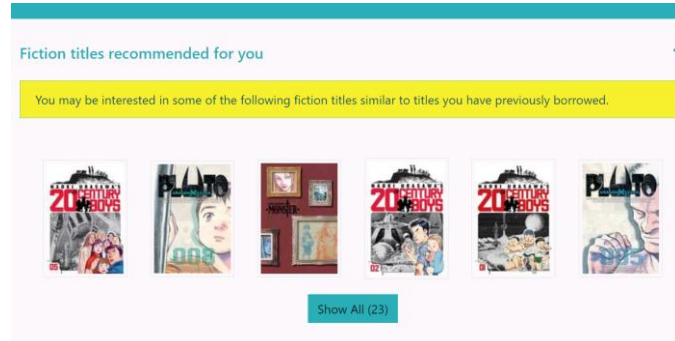
Online system allowing the user to borrow eBooks and audiobooks.



System that allows the user to borrow such media from their home allowing them to download them onto their device and listen or read it and then sends the user to delete the file after their 21-day loan time.

No

## Personalised Recommendations



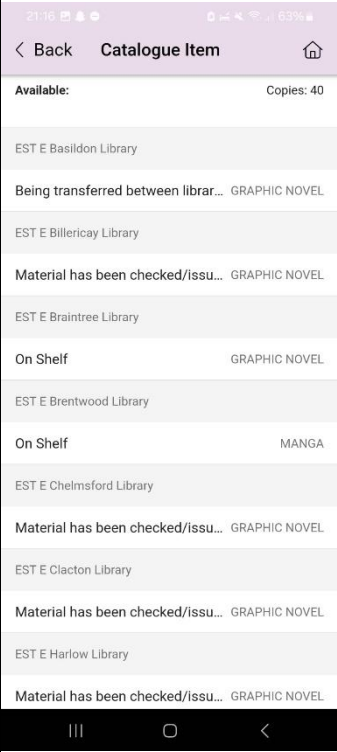
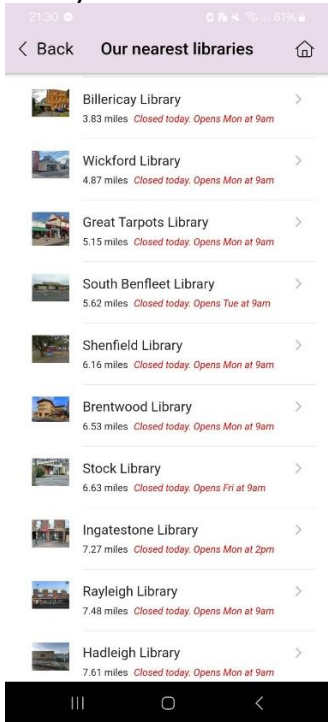
Titles similar to ones you have previously taken out, such as titles from the same author or genre or newer volumes from the same series are recommended to each user

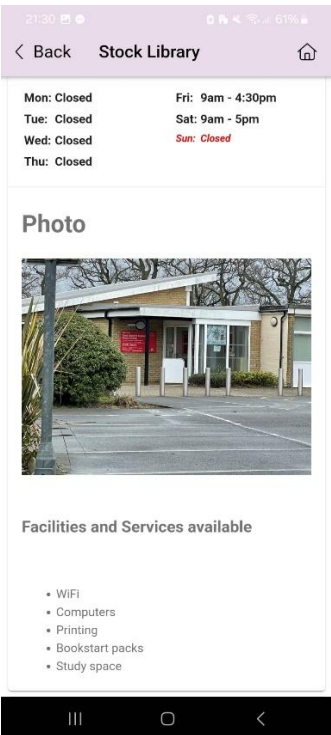
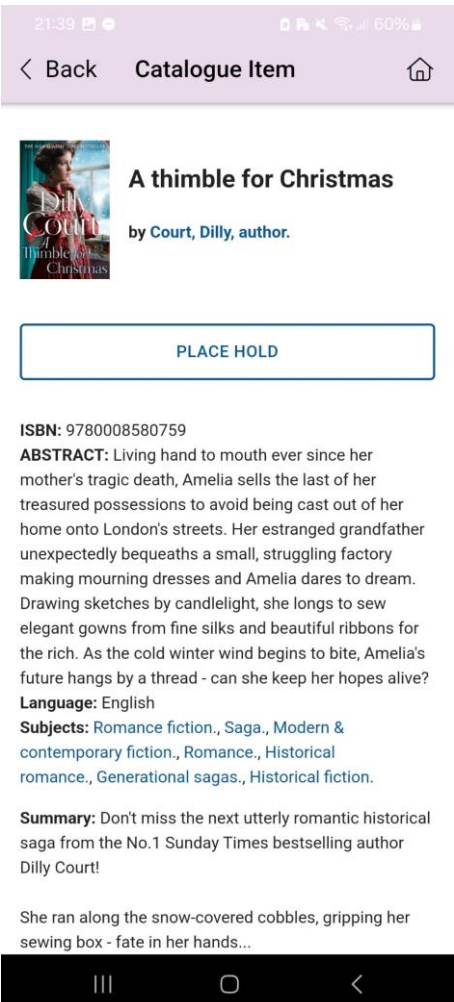
No=

### Example 3 Review: Essex libraries Mobile App

This is an online library system for a local library group, which is used to give users information on their libraries such as their opening and closing hours as well as their available services and facilities. This takes the form of a mobile app which multiple mobile app stores can download. The user can also check the stock of books in their library and make holds of books.

### Example 3 Key Features of Interest

| Feature                                                                                                       | Why is it of interest                                                                                                                                                                                                       | Relevant to my work? |
|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| <b>ISBN Scanner</b><br>     | <p>This allows the user to scan the ISBN of a book using their camera to see available copies and at which libraries and the state of each copy – such as checked out, on shelf or being transferred between libraries.</p> | No                   |
| <b>Library Locator</b><br> | <p>It uses the user's device location services to determine the closest library and its approximate distance allowing them to get more information about each library.</p>                                                  | No                   |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                          |            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <p><b>Classification methods</b></p>  <p>The screenshot shows a mobile app interface for a library. At the top, there's a status bar with the time 21:30 and battery level 61%. Below it, a navigation bar has a back arrow, the text 'Stock Library', and a home icon. The main content area is divided into two sections. The first section, titled 'Photo', shows a photograph of a library building. The second section, titled 'Facilities and Services available', lists several services: WiFi, Computers, Printing, Bookstart packs, and Study space. At the bottom, there's a black navigation bar with three icons: a list, a square, and a back arrow.</p>                                                                                                                                                                                                       | <p>Allows the user to find out information about the libraries in the system, such as its address, opening and closing times as well as the specific libraries facilities, such as the availability of WIFI, computers and printers.</p> | <p>No</p>  |
| <p><b>Place Hold on books feature</b></p>  <p>The screenshot shows a mobile app interface for a library. At the top, there's a status bar with the time 21:39 and battery level 60%. Below it, a navigation bar has a back arrow, the text 'Catalogue Item', and a home icon. The main content area is divided into two sections. The first section, titled 'A thimble for Christmas', shows a book cover and the author 'Court, Dilly, author.'. Below this is a blue button labeled 'PLACE HOLD'. The second section contains the book's ISBN (9780008580759), an abstract, language (English), subjects (Romance fiction., Saga., Modern &amp; contemporary fiction., Romance., Historical romance., Generational sagas., Historical fiction.), and a summary. At the bottom, there's a black navigation bar with three icons: a list, a square, and a back arrow.</p> | <p>Users can place a hold on a book, allowing them to then come into to borrow the book and join a list to borrow the book if it is currently out on loan.</p>                                                                           | <p>Yes</p> |

## Features for Computational Solution

| Feature ID | Feature                                                                                 | Usability | Requirement                                                                                                             | Justification                                                                                                                                                              |
|------------|-----------------------------------------------------------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| F1         | User interaction using graphical user interface                                         | Yes       | A graphical user interface will be made to navigate the program making it easier to understand                          | It makes it easier for the user to understand and reduces the learning curve of the program allowing a wider range of people to be able to use the program                 |
| F2         | There should a database of books that is edited by the program.                         | No        | A database of all books in the library, and associated information such as the author and genre.                        | This makes it easier for users to use the program as it would be easier and quicker to access. This also enables data to be easily edited and saved locally.               |
| F3         | There should be and algorithm used to sort and output books- a search and filter system | Yes       | The user should be able to sort and output relevant books after searching for a specific genre, author or date released | This links to one of the main problems my stakeholders, presented, that it is hard to find books or media relevant to their topic. It can also help to improve performance |
| F4         | There should be user authentication allowing users to sign into the program             | Yes       | The user should be able to use a username and password before being able to access sensitive data                       | This aims to create security, so unauthorised users cannot tamper with the system                                                                                          |
| F5         | There should be Role Based Access Control (RBAC)                                        | No        | The RBAC should allow different user roles so only librarians are allowed to manage loans and normal users borrow books | This aims to enforce security so unauthorised users cannot tamper with the system or other users' accounts                                                                 |
| F6         | Cataloguing books                                                                       | Yes       | Librarians should be able to update add or delete books from the system. It                                             | According to my stakeholder, this is one of their                                                                                                                          |

|    |                                               |     |                                                                                           |                                                                                                                                                                                                                |
|----|-----------------------------------------------|-----|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    |                                               |     | should include relevant details such as the title, author, genre, release date and ISBN.  | main jobs as an assistant librarian and so their job would not be able to be completed if they cannot                                                                                                          |
| F7 | Borrowing and returning process               | Yes | Users should be able to borrow and return their books.                                    | This makes it easier for librarians as they no longer need to do this themselves, also allowing more books to be loaned out at a time making the library more efficient.                                       |
| F8 | Stock Monitoring                              | No  | The number of copies of a book should automatically update when checked out or returned.  | This prevents more books being checked out than available and makes keeping track of stock easier.                                                                                                             |
| F9 | Adding and removing members from the database | Yes | Users should be able to be added or removed from the system, by a librarian user account. | This allows the different students of the sixth form to be added at the start of the year and for inactive accounts to be removed, to prevent an insecure system if their inactive account details are leaked. |

## Limitations

A key limitation of my project is that it will not run on a server and so cannot be used in real time at a large scale. This would mean that it would be a system that all library users and librarians can access on any device however it would require additional hardware which I do not have access to, to be functional.

Furthermore, there will not be mobile app support as this would drastically increase the scale of the project as well as having to write in a completely different programming language which thusly increases the hardware requirements,

Another limitation is that the role-based access control (RBAC). It will be limited to two main roles, a librarian and a normal member. While additional roles or sub-roles such as a child member or adult member would be useful in restricting which books users can and cannot check out, this would make the software more cumbersome to manage and require additional information such as a user's date of birth to implement which can cause security issues. By setting clear and functional roles, I aim to make the program more intuitive to use.

Another limitation is support for other types of media such as e-books and DVDs in terms of storing their metadata. This feature is likely to not be added as the user does not require the feature as the library, they are in charge of does not contain any digital media or physical media except from books.

The program will also not have support for barcode scanners as I do not have a barcode scanner available to me. Additionally, the library mainly uses book IDs to sign books in and out. While this functionality could have been added using a camera and Optical Character Recognition, this is likely to be inaccurate which would be unacceptable as this is very important to the operation of the program.

The books in the database will also not have their book cover art displayed on the system. While this would be useful in encouraging users to check out books and help them find the book within the library, this feature would be difficult to add without online functionality and may impose on an author or artist's copyrights.

## Software and Hardware Requirements

### Hardware Requirements

- Keyboard and Mouse – A keyboard and mouse will be required for all interactions with the program. Ideally a keyboard with a numeric keypad to make data entry easier.
- A Monitor – Greater than 21 inches and 1080p resolution. A clear and readable display will be essential as large amounts of data will have to be managed.
- A computer or laptop with multiple cores for smooth multitasking and enough RAM. While the program itself is quite lightweight, a large hard drive (greater than 500GB) is recommended as it allows librarians like my stakeholder Mrs Oats to store thousands of books and backups without the risk of any data loss and enabling further scalability in the future

### Software Requirements

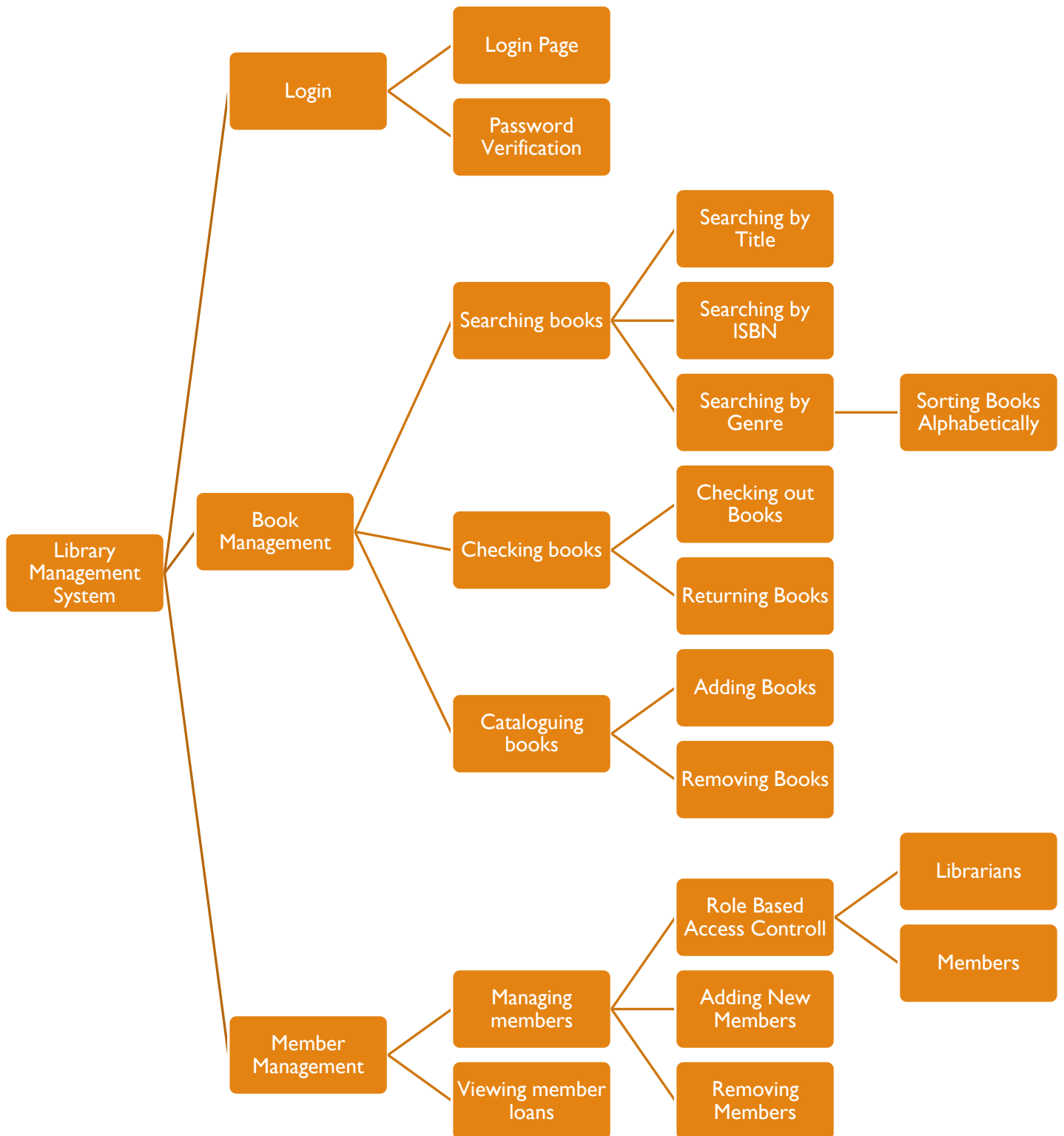
- A Windows operating system as these all have support for Python and the required libraries that will be used.
- A Python Interpreter – The program was coded in Python, so an appropriate Python Interpreter is required for it to work.
- Tkinter Python module – This is the module which produces and displays the Graphical User Interface
- Sqlite3 Python module - This is the module which is used to create and edit the databases used in the program.
- Bcrypt Python module – This is the module used to hash user passwords for safety.
- Regular expression (re) module – This is the module used for email validation,

## Success Criteria

| Success Criteria ID | Success Criteria                                                                                                                                                                                                                                              | Relevant Feature ID | Further Justification                                                                                                                                                                                                                                                                                             |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S001                | There will be a graphical user interface where every button works.                                                                                                                                                                                            | F1                  | This is fundamental to making the program easy and intuitive to use. Using a different interface such as a command line interface may be hard to understand and grasp for members and librarians and a natural user interface is likely to be inconvenient to use and so slow down the productivity of librarians |
| S002                | There should be a database containing tables for books, users and loans                                                                                                                                                                                       | F2                  | Having a database with these features allows the data to be easily portable, in one or two files. This is also the easiest to edit and so it is easy to keep a well-maintained store of all required information                                                                                                  |
| S003                | There should be a function to search for books by name and by their ISBN and then the book and relevant information should be outputted to the user                                                                                                           | F3                  | This enables members and librarians to check what books are in stock as well as information such as the books name or genre, essential to the operation of a library management system.                                                                                                                           |
| S004                | There should be a function to sort books by genre and output them alphabetically                                                                                                                                                                              | F3                  | This allows members to search for books with their desired genre, but also to help the organisation of the library as the books are organised by genre then in alphabetical order                                                                                                                                 |
| S005                | There should be a log in page                                                                                                                                                                                                                                 | F4                  | This aims to protect sensitive information or functions from unauthorised users and allows to dictate between member and librarian users – note Feature F5.                                                                                                                                                       |
| S006                | There should be Role Based Access control (between normal members and privileged librarian accounts) which have access to different features                                                                                                                  | F5                  | Primarily successful through success criteria S005. This is essential as there are features that librarians should be able to access to enable them to do their work that would be useless or even cause security risks for normal member users to have access to such as adding or deleting books or members.    |
| S007                | There should be a feature to add or remove books                                                                                                                                                                                                              | F6                  | Fundamental to the job of a librarian, adding and removing books as well as its metadata such as the name of the author and its genre. This is necessary to improve the database and enable users to see which books are available.                                                                               |
| S008                | There should be a feature to borrow and return books. It should check whether the book is in the database and prevent already checked out books from being checked out again. It should also log which member logged out the book according to its member ID. | F7                  | Also fundamental to the operation of a library. Enables users to check books in and out themselves, relieving this duty from librarians. Also, it will add to a loans table, allowing librarians to check loans.<br>Note- also F8                                                                                 |
| S009                | There should be a feature to add and remove new members                                                                                                                                                                                                       | F9                  | This allows for new members to be added to the system, particularly at the start of the school year such as new sixth form students                                                                                                                                                                               |

## Design

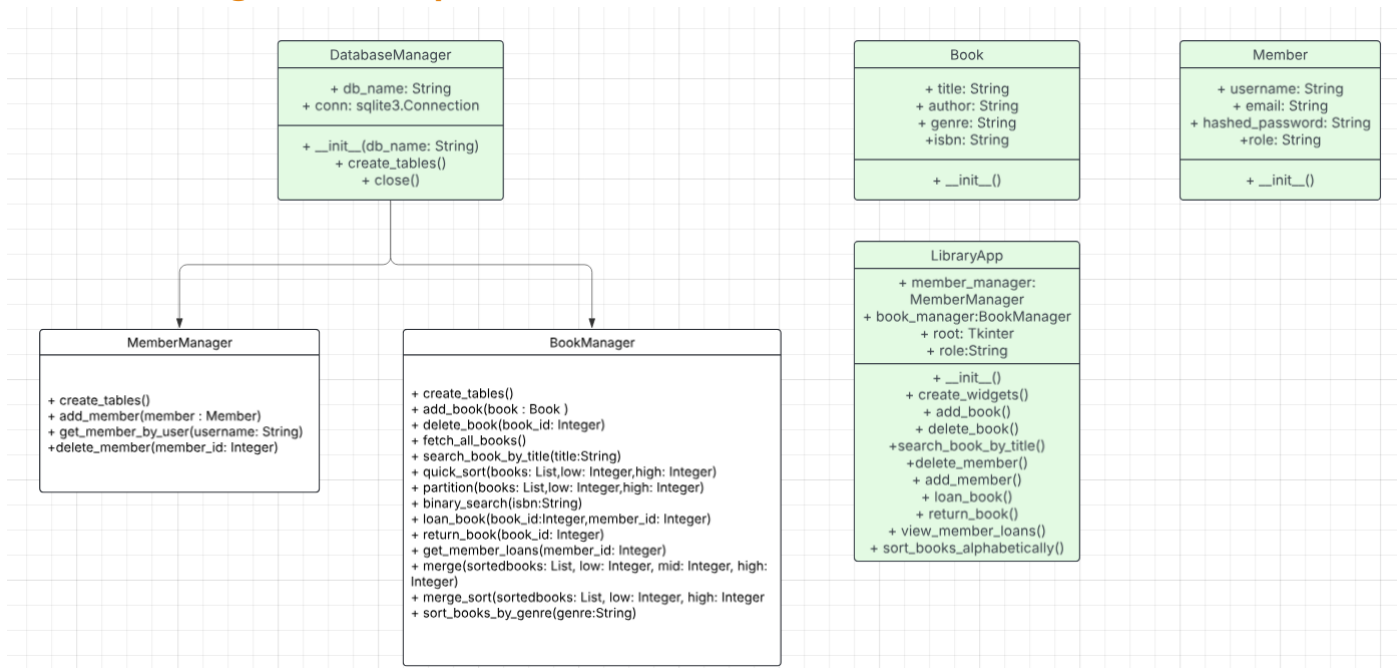
### Decomposition Diagram and Explanation



| Decomposition            |                                  | Justification          |                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------|----------------------------------|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Login</b>             | <i>Login Page</i>                |                        | There should a page where the user must enter credentials to enter the main system to protect the security of the system and preventing unauthorised users tampering with the system                                                                                                                                                                                                |
|                          | <i>Password Verification</i>     |                        | The credentials should include a public username, but a private password which is necessary to prove that it is an authorised user accessing the system and not someone pretending to be them. This password should be verified to see whether it matches the password so a user cannot enter any string of characters and get access to the Library Management System              |
| <b>Book Management</b>   | <i>Searching Books</i>           | Searching by Title     | As users may not directly know the name of the book, or are looking for books as a part of a series this is likely to use a LIKE SQL query rather than a strict search to allow for wildcards and books which match a certain pattern to be returned                                                                                                                                |
|                          |                                  | Searching by ISBN      | This feature is essential for librarians' logs and cataloguing, helping to avoid errors. As the ISBN is unique to the book a strict search is likely to be used.                                                                                                                                                                                                                    |
|                          |                                  | Searching by Genre     | The library sorts the books by genre and then by alphabetic order so a sort must be implemented for this feature.                                                                                                                                                                                                                                                                   |
|                          | <i>Checking Books</i>            | Checking Out Books     | Tracks the books being borrowed and by whom allowing for easier implementation of borrowing limits.                                                                                                                                                                                                                                                                                 |
|                          |                                  | Returning Books        | Updates the availability of the book back to available to the member does not get the fine. Allows for the tracking for overdue books.                                                                                                                                                                                                                                              |
|                          | <i>Cataloguing Books</i>         | Adding Books           | Allows an easy implementation for new books to expand the library collection quickly, allowing for easier expansion such as the addition of other information such as the publication year                                                                                                                                                                                          |
|                          |                                  | Deleting Books         | Allows for outdated or damaged books to be easily removed from the system without affecting the database, such as changing all the IDs of all other books to fill the gap.                                                                                                                                                                                                          |
| <b>Member Management</b> | <i>Role-Based Access Control</i> | Librarians and Members | Allows for differentiation between Librarians and Members to enforce their different permissions, such as Librarians being able to add or remove books whereas students can only borrow or return books. This prevents unauthorised access to important features which could be exploited and enables new roles or sub roles, such as a 'Child' or 'Adult' member to be added later |
|                          | <i>Adding Members</i>            |                        | Both the adding and removing members are separated from book transactions to allow for future expansion, such as membership expiry dates and new types of members, such as temporary accounts to be made. It also allows for integration with school databases for automated account creation or altering when new users join the school or move up the school.                     |

|  |                             |  |                                                                                                                                                                                                                                                                                |
|--|-----------------------------|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <i>Removing Members</i>     |  | Allows for unauthorised or inactive users to be removed from the system. It can also be scaled with integration to school systems so old accounts can automatically be deleted, so it is easier to manage members.                                                             |
|  | <i>Viewing Member Loans</i> |  | Allows librarians to monitor who has borrowed books as well as when they are due. This helps to enable accountability so members can return books on time and can lead to further enhancement such as automatic email overdue reminders or borrowing restrictions for members. |

## Structure Diagram and Explanation



There is a base class for the classes related to databases called “DatabaseManager.” Both the “BookManager” and “MemberManager” classes take this class as their parent, inheriting the database name (`db_name`) for initialisation of the databases and an attribute to form the connection to the database. They also inherit the method to close the connection to the database and an unspecialised method to initially create the books, loans, and members tables and databases. Polymorphism is used with the MemberManager and BookManager to specialise the `create_tables()` method for their specialised tables.

Within the MemberManager class, it has methods to add a new member to the database, taking in an object of the Member class as a parameter. There is also a method to search the database for a user by their username and returning its information, used within the login process and takes in a username as a parameter. There is also a `delete_member()` method which takes in a user’s `member_id` as a parameter to delete them from the database.

Within the BookManager class, a child class of the DatabaseManager, there are many methods to facilitate the function of the books and loans tables. Polymorphism is used to adjust the inherited `create_tables()` method, so it creates the books and loans tables. It contains an `add_book()` method to add an object of the Book class, taken in as a parameter to the database. There is also a `delete_book()` method, which takes in the `book_id` of the book to be deleted as a parameter to remove it from the database. There is a `fetch_all_books()` method to fetch all books from the database and create objects of the Book class and then append them to a list of all books, so it can be used for sorted later. There is also a `search_book_by_title()` method which takes in the user’s entered title as a parameter and uses an SQL LIKE query to return the applicable books in the database. The `quick_sort()` and `partition()` methods take in the list of all books as well as low and high pointers to sort the books in order of the ISBNs. This is then used by the `binary_search()` method, which takes in a target ISBN as a parameter to search for the book with a matching ISBN. The `loan_book()` method takes in a `book_id` and `member_id` as a parameter to check out the book and log it to the loans table. The `return_book()` table takes in the target `book_id` as a parameter to check the loan status of the book to be returned. The `get_member_loans()` method takes in a member’s ID as a parameter and returns all of their loans. The `merge()` and `merge sort()` functions have parameters of a low, mid, and high integer as a pointer as well as a sorted books list, sorted by the `sort_books_by_genre()` method, which takes in a target genre and returns a list of all books with that genre.

The Book class determines all attributes each book in the database has, with title, author, genre and ISBN attributes. In addition to this, the Member class has attributes of their username, email address, hashed password and role- either Librarian or unprivileged Member.

The `LibraryApp` class determines the user interface of the program, with attributes containing objects of the `MemberManager` and `BookManager` classes, as well as a Tkinter root, and it takes in the role of logged in user. The `create_widgets()` method displays the main page as well as all of its buttons depending on the role of the logged in user. The other methods include the display and interactions for the other methods, and then links to the respective, `BookManager` or `MemberManager` methods.

## Key Algorithms

### Class Definitions

This defines the Book and Member classes, which are used and changed frequently throughout the program and thus have public attributes. The Book class has attributes for the Title of the book, its author, the genre of the book and its ISBN and the Member class has attributes for the username of the member, their email, the hashed password, and the fetched role of the member, either librarian or a member.

```
3  ~ CLASS Book
4      PUBLIC Title: String
5      PUBLIC Author: String
6      PUBLIC Genre: String
7      PUBLIC ISBN: String
8  END CLASS
9
10 ~ CLASS Member
11     PUBLIC Username: String
12     PUBLIC Email: String
13     PUBLIC Password: String
14     PUBLIC Role: String
15 END CLASS
```

### Login Function

This defines the login function of the program. It takes in a username and password from user inputs and then does a database query to check whether the username entered is found in the database. If it is, then it checks whether the entered password is the same as the hashed password stored in the database and runs the mainapp function with the user's role as a parameter if they match. If the username is not found within the database or the passwords do not match, it prints Login failed for security. These are essential as they provide the role-based access control (RBAC) for the system and aim to prevent unauthorized users from getting into the system. This was decomposed from the main program to allow for separate development between the two popups.

```
17 ~ FUNCTION Login()
18     Username = INPUT("Please enter your username")
19     Password = INPUT("Please enter your password")
20     QUERY Database FOR Username
21     IF FOUND == True THEN
22         IF Password = UsernameHashedPassword THEN
23             Mainapp(userRole)
24         ELSE
25             PRINT("Login Failed")
26         END IF
27     ELSE
28         PRINT("Login Failed" )
29     END IF
30 END FUNCTION
```

### Add Book Function

This is the procedure to add a book to the database to achieve one half of Success Criteria S007. The procedure takes in user inputs for title, author, genre, and ISBN and creates a new Book class object named book and attempts to insert that into the database. If it is entered, it prints that the user is added, but if there is an error, it prints that there was an error in adding the new book.

```
32 PROCEDURE AddBook()
33
34     Title = INPUT("Please enter the title of the book")
35     Author = INPUT("Please enter the author of the book")
36     Genre = INPUT("Please enter the genre of the book")
37     ISBN = INPUT("Please enter the isbn of the book")
38
39     Book = NEW Book(Title, Author, Genre, ISBN)
40     IF INSERT Book INTO Database THEN
41         PRINT("Book Added")
42     ELSE
43         PRINT("Error")
44     END IF
45
46 END PROCEDURE
```

### Delete Book Function

This is the procedure to delete a book to the database to achieve Success Criteria S007. The procedure takes in user inputs for the library Book ID and does a delete SQL query for the relevant record for the Book ID. If it is successful and the book is deleted, it will print that the book was deleted successfully, but if it does not, then it prints that the book was not found.

```
PROCEDURE DeleteBook()

    BookID = INPUT("Please enter the ID of the book to delete")

    IF DELETE FROM Database WHERE ID = BookID THEN
        PRINT("Book deleted successfully")
    ELSE
        PRINT("Book not found")
    END IF

END PROCEDURE
```

### Fetch all Books Procedure

This is a procedure to fetch all the books from the database for sorting. It initially creates an empty list of Books and then does an SQL query to select all information from the database. Then, for each row, it will create a new Book object containing the details from fields in that record and then add that book object to the books list. It then returns that list of books.

```
PROCEDURE FetchAllBooks()  
  
    Books = []  
  
    Rows = SELECT ALL FROM Database  
  
    FOR EACH Row IN Rows  
        Book = NEW Book(Row[1], Row[2], Row[3], Row[4])  
        ADD Book TO Books  
    NEXT Row  
  
    RETURN Books  
  
END PROCEDURE
```

### Search Book by Title

This procedure takes in a user input and then does an SQL query for all the books in the database which matches the title entered and then returns the books found. This is effectively a linear search and as such has a time complexity of  $O(n)$  but is simpler to implement than a binary search which would require the books to be extracted from the database and be sorted before it can be searched.

```
135 PROCEDURE SearchBookByTitle()  
136  
137     Title = INPUT("Enter title to search for")  
138  
139     Books = SELECT * FROM Database WHERE Title CONTAINS Title  
140  
141     RETURN Books  
142  
143 END PROCEDURE
```

### Quicksort

This is the appropriate quicksort procedure. The quicksort procedure takes in the list of books, which is output from the search all books procedure, a low index and high index as parameters. As a recursive algorithm, the base case of recursion is when low is smaller than or equal to the high pointer meaning the subarray has one or no elements in it, thusly meaning it is already sorted. It then calls the partition function to reorder the list so smaller books, by ISBN, are to the left and larger books are to the right of the pivot and then returns the index of the pivot after it has been placed, it then recursively sorts the right side of the list and then recursively sorts the left side of the list. The partition procedure takes the same parameters and sets the pivot to be the book found in the books list at index low. It then sets its left wall to low. It will then loop through each element after the pivot, making sure that if it is smaller than the pivot, it goes to the left side of the pivot by swapping it and adjusting the Left wall. It then will return the Left wall variable. This is important as it helps to complete success criteria S003 so relevant ISBN values can be returned. The quick sort algorithm was selected to sort books by ISBN as it has an average case time complexity of  $O(n \log n)$  making it efficient for moderately large datasets, improving its scalability, I chose it over a merge sort as it has a lower memory usage.

```
PROCEDURE QuickSort(Books, Low, High)  
  
    IF Low < High THEN  
        PivotIndex = Partition(Books, Low, High)  
        QuickSort(Books, Low, PivotIndex - 1)  
        QuickSort(Books, PivotIndex + 1, High)  
    END IF  
  
    RETURN Books  
  
END PROCEDURE
```

```
PROCEDURE Partition(Books, Low, High)  
  
    Pivot = Books[Low]  
    LeftWall = Low  
  
    FOR I = (Low + 1) TO High  
        IF Books[I].ISBN < Pivot.ISBN THEN  
            LeftWall = LeftWall + 1  
  
            Temp = Books[I]  
            Books[I] = Books[LeftWall]  
            Books[LeftWall] = Temp  
        END IF  
    NEXT I  
  
    Temp = Books[Low]  
    Books[Low] = Books[LeftWall]  
    Books[LeftWall] = Temp  
  
    RETURN LeftWall  
  
END PROCEDURE
```

## Binary Search

This is a function for a Binary Search, used to search for a specific book by its ISBN to achieve Success Criteria S003. It will initially take in an ISBN value by user input and then create an array for all books by calling the `FetchAllBooks()` function. It will then set a low and high pointer to the first and last indexes in the array and sort the Books using the Quicksort function. It will then complete a binary sort by repeated calculating the middle element in the sublist, and then checking whether the value of the ISBN of the book is equal, lower or higher than the ISBN to search for and will then adjust the sublist according to such, by setting low to mid+1 if the book in the middle has an ISBN lower than target or setting high to mid-1 if the book at the middle is greater than the target ISBN. It will then return the book if it is found or return not found if it is not found (when the sublist is empty). The Binary Search has a time complexity of  $O(\log n)$  which is a significant improvement over a linear search which has a time complexity of  $O(n)$  which would significantly impact performance as the number of books increases. However, it requires the input list to be sorted hence a Quick sort is called before the sort.

```
PROCEDURE BinarySearch()  
    ISBN = INPUT("Enter the ISBN to search for")  
  
    Books = FetchAllBooks()  
    Low = 0  
    High = LENGTH(Books) - 1  
  
    Books = QuickSort(Books, Low, High)  
  
    WHILE Low <= High DO  
        Mid = (Low + High) DIV 2  
  
        IF Books[Mid].ISBN = ISBN THEN  
            RETURN Books[Mid]  
        ELSE IF Books[Mid].ISBN < ISBN THEN  
            Low = Mid + 1  
        ELSE  
            High = Mid - 1  
        END IF  
    END WHILE  
  
    RETURN "Book not found"  
END PROCEDURE
```

## Loan Books

This is the function of loaning books out of the database to achieve part of Success Criteria S008, which is to loan and return books from the database. It takes in a Book ID and Member ID as user inputs and then checks the database to see whether the relevant book from the entered Book ID is checked out already. If the book ID is not found in the database, it returns that the book is not found, and if the field relating to whether the book is checked in is equal to 0, it will return that the book is already checked out. If not, it will calculate the loan date and the due date for the book, 14 days after the loan date and with the insert that relevant information into the loans table of the database and will update the book to "checked out" in the Books table, returning to the user that the book was loaned out successfully. This system prevents multiple loans of the same book from being taken out at the same time and ensures data integrity.

```
PROCEDURE LoanBook()  
  
    BookID = INPUT("Enter the book ID")  
    MemberID = INPUT("Enter the member ID")  
  
    Book = SELECT CheckedIn FROM Books WHERE ID = BookID  
  
    IF Book DOES NOT EXIST THEN  
        RETURN "Book not found"  
    END IF  
  
    IF Book.CheckedIn = 0 THEN  
        RETURN "Book is already checked out"  
    END IF  
  
    LoanDate = TODAY'S DATE  
    DueDate = TODAY'S DATE + 14 DAYS  
  
    INSERT (BookID, MemberID, LoanDate, DueDate) INTO Loans  
  
    UPDATE Books SET CheckedIn = 0 WHERE ID = BookID  
  
    RETURN "Book loaned out successfully"  
END PROCEDURE
```

## Return Books

This is the function to return books into the database to achieve part of Success Criteria S008, which is to loan and return books from the database. It takes in a Book ID as a user input and then updates the loans table to set the returned field to one, indicating it has been returned, at the relevant loan. If there were no rows that were updated, that means either the book was never loaned out before or has already been returned and so does not have an active loan. In this scenario, it returns that there was not an active loan found for the

```
PROCEDURE ReturnBook()  
  
    BookID = INPUT("Enter the book ID")  
  
    UPDATE Loans SET Returned = 1 WHERE BookID = BookID AND Returned = 0  
  
    IF NO ROWS WERE UPDATED THEN  
        RETURN "No active loan found for the book"  
    ELSE  
        UPDATE Books SET CheckedIn = 1 WHERE ID = BookID  
        COMMIT CHANGES  
        RETURN "Book returned successfully"  
    END IF
```

book, otherwise it confirms to the user that the book was returned successfully. Along with the Loan Books procedure, this prevents loans of the same book at the same time and ensures data integrity is maintained.

### Get all Member Loans

This aims to allow users to check the loans which they currently have taken out as well as its related information. It takes the member's ID as an input and then creates a list of loans, selecting the book's ID, title and author, the due date of the loan and the status of whether it has been returned or not. It uses a JOIN to join the loans and books table together and joins them where the loan's book id matches the book's ID, returning rows where the loan matches the specific member ID, and then returns the full list of results.

```
PROCEDURE GetMemberLoans()  
  
    MemberID = INPUT("Enter the member ID")  
  
    Loans = SELECT Books.ID, Books.Title, Books.Author, Loans.DueDate, Loans.Returned  
            FROM Loans JOIN Books ON Loans.BookID = Books.ID  
            WHERE Loans.MemberID = MemberID  
  
    RETURN Loans
```

### Merge Sort

This function aims to sort the books alphabetically so they can be used by my librarian shareholder to organise newly returned books back into the library. The merge sort function takes in all the books in a list, and a low and a high pointer as parameters. As a recursive algorithm, the base case is whether the high pointer is greater than the low pointer, it then calculates the mid pointer and then sorts the left half of the list recursively by making mid the high pointer, and then the right side by making mid + 1 the low pointer, then uses the merge function to merge the two sublists together. The merge function works by taking in the books list, and the low, mid and high pointers. It creates a new list called left which contains the left half of the list, and a right list, which contains the right half. It then initiates three pointers, loc1, loc2 and locM which indicate the position in the left list, the right list and the position in the original books list respectively. It then loops while the left and right lists still have items which need to be compared. It then compares the title of two books, if the left book comes before the right book, the left book is copied into the original list otherwise the right book is copied into the original list, and then the locM is incremented. Otherwise, if the left side or right side had any leftovers the must be already sorted and so they are copied into the original list. The Merge sort was chosen over the Quick sort it ensures stability, especially with books with identical titles which remain in the same relative order allowing the librarian to sort secondary attributes such as the author or genre. It is also important to note Merge sort has a worse and average time complexity of  $O(n \log n)$  but a space complexity of  $O(n)$ .

```
PROCEDURE MergeSort(SortedBooks, Low, High)  
  
    IF High > Low THEN  
        Mid = (Low + High) DIV 2  
        MergeSort(SortedBooks, Low, Mid)  
        MergeSort(SortedBooks, Mid + 1, High)  
        Merge(SortedBooks, Low, Mid, High)  
    END IF  
  
    RETURN SortedBooks  
  
END PROCEDURE
```

```
PROCEDURE Merge(SortedBooks, Low, Mid, High)  
  
    Left = SortedBooks[Low TO Mid]  
    Right = SortedBooks[Mid + 1 TO High]  
  
    Loc1 = 0  
    Loc2 = 0  
    LocM = Low  
  
    WHILE Loc1 < LENGTH(Left) AND Loc2 < LENGTH(Right)  
        IF Left[Loc1].Title <= Right[Loc2].Title THEN  
            SortedBooks[LocM] = Left[Loc1]  
            Loc1 = Loc1 + 1  
        ELSE  
            SortedBooks[LocM] = Right[Loc2]  
            Loc2 = Loc2 + 1  
        END IF  
        LocM = LocM + 1  
    END WHILE  
  
    WHILE Loc1 < LENGTH(Left)  
        SortedBooks[LocM] = Left[Loc1]  
        LocM = LocM + 1  
        Loc1 = Loc1 + 1  
    END WHILE  
  
    WHILE Loc2 < LENGTH(Right)  
        SortedBooks[LocM] = Right[Loc2]  
        Loc2 = Loc2 + 1  
        LocM = LocM + 1  
    END WHILE  
  
END PROCEDURE
```

## Sort Books by Genre

This aims to satisfy Success Criteria S004 by allowing the user to find related books by genre. The target genre is taken as a user input and then calls the fetch all books list to get a list of book objects of the Book class. It then does a linear search to see whether the genre of the book matches the target genre and then adds this book to the list of filtered (SortedBooks) and then returns the list of sorted books. The linear search has a time complexity of  $O(n)$  and would be easier to implement than a binary search which would require sorting.

```
PROCEDURE SortBooksByGenre()  
  
    Genre = INPUT("Enter the genre to sort by")  
  
    Books = FetchAllBooks()  
    SortedBooks = []  
  
    FOR EACH Book IN Books  
        IF Book.Genre = Genre THEN  
            ADD Book TO SortedBooks  
        END IF  
    END FOR  
  
    RETURN SortedBooks  
  
END PROCEDURE
```

## Add Members

This is a function to help the librarians with their duty of adding new members to the system. It takes a username, email, password, and role as a user input and creates a new member object with that information. It then tries to insert this into the members database. If it is inserted correctly then it returns that the member was added successfully, and if it does not this means that the email used is already found in the database and so is not unique and so returns that the email is already found in the database.

```
PROCEDURE AddMember()  
    Username= INPUT("Enter a username")  
    Email= INPUT("Enter your Email")  
    Password= INPUT("Enter a password")  
    Role= INPUT("Enter your role")  
  
    newMember=member(Username,Email>Password,Role)  
  
    IF INSERT INTO members (newMember.username, newMember.email, newMember.password, newMember.role) THEN  
        RETURN "Member added successfully."  
    ELSE  
        RETURN "Error: Email already exists in the database."  
    END IF  
  
END PROCEDURE
```

## Remove Members

This function is also essential in the job of my librarian shareholder, enabling them to delete members where necessary. It takes in a member ID as a user input and then does an SQL query to delete the record of a member with the member ID which is matched. However, if no records are affected, meaning there is nobody with that member ID, it will return that the member is not found, but if at least one record is affected, then it returns that the member was deleted successfully.

```
PROCEDURE DeleteMember()  
    MemberID = INPUT("Enter the ID of the Member to be Deleted")  
    DELETE FROM members WHERE id = MemberID  
  
    IF RowsAffected = 0 THEN  
        RETURN "Member not found."  
    ELSE  
        RETURN "Member deleted successfully."  
    END IF  
  
END PROCEDURE
```

### Get Member by username.

This function allows librarians to get the information of users in the database using their username. It takes in a username as a user input. It then does an SQL query to return everything in the relating record where the username matches that of the target username. If a record is found, then the returned data is applied to a new member object otherwise "User not found is returned."

```
PROCEDURE GetMemberByUsername()  
    Username = INPUT("Enter Username")  
    User =SELECT * FROM members WHERE username = Username  
    IF User IS NOT NULL THEN  
        RETURN NEW Member(Row[1], Row[2], Row[3], Row[4])  
    ELSE  
        RETURN "User not Found"  
    END IF  
  
END PROCEDURE
```

## Usability and User Interface

Because the library system relies a lot on filling in several forms or entries which may be prone to user misinput, if there is an error, the form does not erase all the entered data. This allows the error to be highlighted and fixed without the user having to re-enter everything or the form crashing, preventing frustration.

To ensure a high amount of readability, my program's user interface will only consist of black text on a white background for high contrasts which aims to improve the visibility for visually impaired users or colourblind users. This also ensures compatibility with certain screen readers. Furthermore, the font used within the user interface will be the font used on their operating system, such as Segue UI for Microsoft Windows devices and Helvetica for Apple Mac devices by default. This is important in keeping a cohesive look across the whole program but ensuring that the text is easy to read on their device as well as giving the user the ability to customise it based on their user font.

For visually impaired users, there should also be a toggleable high visibility version, consisting of yellow text on a black background, which increases the contrast further, for any users with a significant visual impairment to ensure readability for those users.

The use of images or icons should be used sparingly for emphasis, particularly to highlight users to any errors or to highlight changes in the database. This rule also helps to improve useability by explaining how to use the system using words rather than images which may not be as intuitive for the user.

The program should be split into separate screens, such as the log in page and main page which should close when necessary, and there should be buttons with correct names which prompt the user into another section of the program, so they can do another function. This helps to not keep the program overcrowded with different features and functions and highlights the relevant function when required to make sure the user does not get confused.

From my research I learnt that my program should not have a lot of windy sections, meaning pages which lead onto other pages which lead on to other pages which goes on further. This is likely to confuse the user and frustrate them as they end up going in circles and do not find the feature they are looking for. Therefore, pages should be kept brief to ensure that they are easy to navigate. Error Messages should also be clear and avoid being out of context, such as error codes which a user will not understand, particularly if it is a user issue to improve their chance of fixing the issue, however they are should not be completely transparent for security reasons.

### GUI Mock Ups

This screenshot (Figure 1) shows possible mock up design schemes for the login page of the library management system. The login page itself is very simple, with a basic username and password entry fields along with a large, Bold "Library Management System" title and a login button to submit the user's entries and log into the system. All colour schemes also use the standard Microsoft Segue UI font family to ensure high levels of readability.

In the centre there is the normal colour scheme of the system. This consists of black text on a white background, ensuring a high level of contrasts which allows for readability. User interaction is also highlighted using a grey box with a black stroke outline.

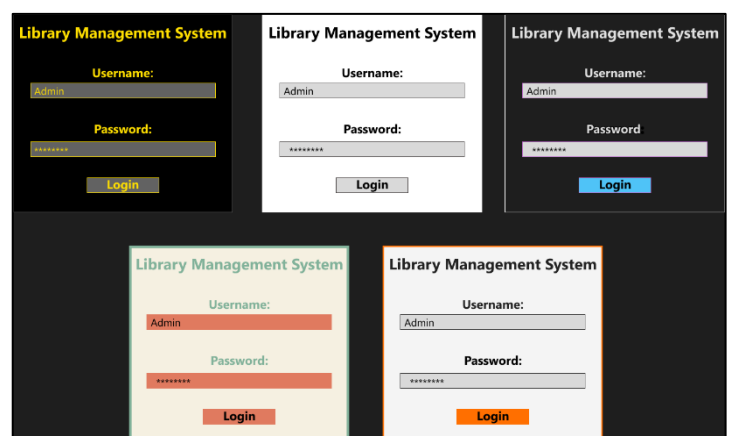


Figure 1: Mock-up of login page. From top left to bottom right, "High Visibility Theme", "Normal Theme", "Dark Theme", "Earth Theme", "Coloured Orange Accent Theme".

To the right is a dark theme alternative which helps users who are using the system. This consists of a dark grey background, which is less bright than a white background. Text is of a light grey colour and this same colour is used to represent, user entry fields which have a purple stroke. A light blue colour is used as a call to action, imploring users to click the log in button, making its design more intuitive. However, it should be noted that on lower quality screens, which are typically less bright, this design theme may be hard to use.

To the left is a high visibility theme which consist of canary yellow text on a black background. This is typically preferred by people with low vision like “Stakeholder 3” and does not impact the readability of people with colour-blindness like Stakeholder 2, as simulated with figure 2. This mock up also has the added benefit from the dark mode theme of being better for the eyes at night however its negative of not being easy to use on worse quality screens is not as significant due to the high contrast between the canary yellow text and the black background.

In the bottom left is an Earth theme consistent of beige background, like the pages of a book, with green text and orange user prompts, the login button and user inputs for username and password. This aims to imitate colours you typically see in nature such as trees and auburn autumn leaves. This has heavy connotations to books and thusly is an appropriate theme for a library. However, this is not an appropriate theme when considering accessibility as shown by figure 2 which simulates how someone with protanopia colour-blindness like “Stakeholder 2” will see the program. Several of the colours looks very similar and as such means there could be significant difficulties in reading the text or interacting with the system.

On the bottom right is a monochrome accent theme with similar black text on a white background like the normal theme. However, it has an orange-coloured accent to draw the user’s attention, such as in this scenario to the log in button, to make the program more intuitive to use. While it is important to consider the use of a vibrant colour like orange, which is more likely to catch the user’s attention, Figure 2 also shows how the use of such colours can be unappealing and make it harder to read the text on the screen.

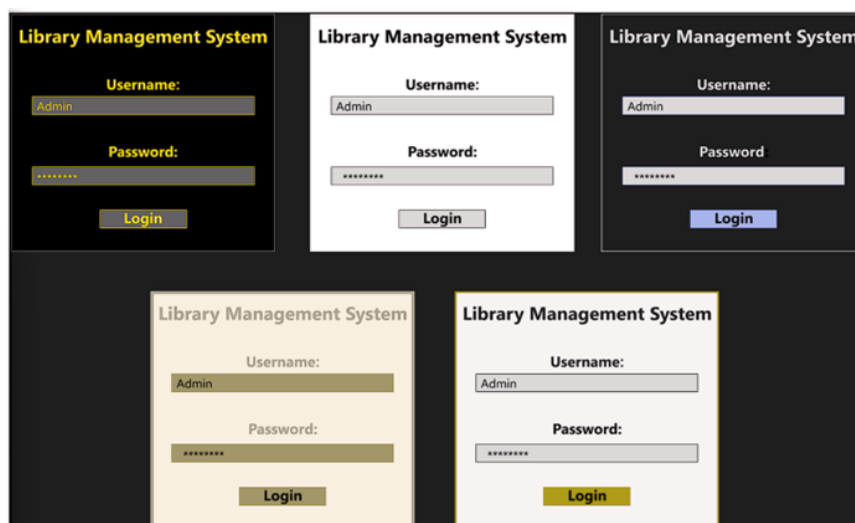
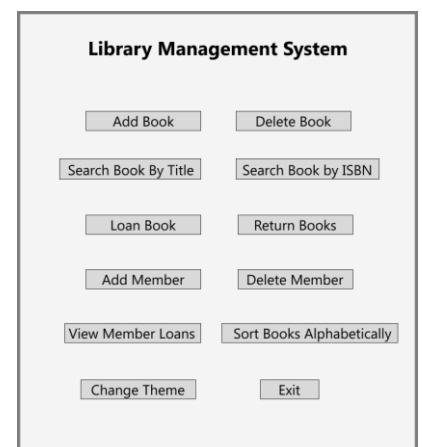


Figure 2: Mock-ups of Login page with Protanopia filter.

## Main Page

This is a mock up for the main page of the library system for librarians, using the Normal theme. On the page contains buttons for every function of the system which helps for clarity and avoids confusing nested navigation or “Maze like UI.” This is furthered as liked functions like add or remove book and add or delete members are separated to avoid unnecessary menus. There are no icons or images which rely on implied knowledge, which may be confusing and so are replaced by understandable key names on buttons, which are clear and link directly to an action, which are highlighted by easy to notice buttons. This would be different than students who would have access to less functions of the system.



## Key Variables, Classes, and Data Structures

The database consists of three tables, being Books, Loans and Members. The Books table should include all information about the books stored in the database, including an automatically incrementing ID which is unique for the book. It should also include a title, author, genre field which are all non-unique strings, an ISBN value which should be unique and a Boolean Yes/No field for whether the book is checked-in/ available or not.

| id | title | author | genre | isbn | checked_in |
|----|-------|--------|-------|------|------------|
|----|-------|--------|-------|------|------------|

Example table design for the Books table

The Member table should contain all information about the members of the program, including an automatically incrementing unique ID. The username and email fields should also be unique string fields, and the password field should contain the hashed passwords of users for security purposes. There should also be a role field to dictate whether the user is a librarian or student; kept as a string rather than a variable to allow for new roles in the future.

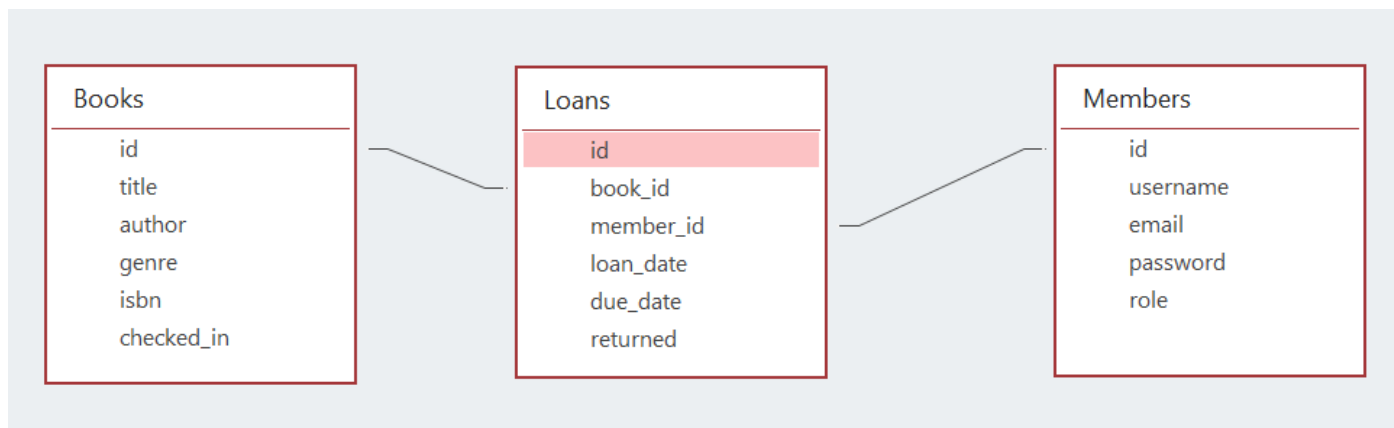
| id | username | email | password | role |
|----|----------|-------|----------|------|
|----|----------|-------|----------|------|

Example table design for the Member

The Loans table should include a unique primary key dictating the individual loan. There should be foreign keys of the member and book id linking to the id of both the book and member tables. There should be one to many relationships between the id in the books and members table to the loans table as on book or member can be attributed to multiple loans in the loans table. The loans table should also have a returned Boolean Value dictating whether the loan was complete or not.

| id | book_id | member_id | loan_date | due_date | returned |
|----|---------|-----------|-----------|----------|----------|
|----|---------|-----------|-----------|----------|----------|

Example table design for the Loans table



Entity Relationship Diagram showing relationship between the id of the books and members table (the primary key) and the book id and members id of the loans table

| File    | function                 | Local/Global | Name      | Type             | Variable/Constant | Purpose                                                                                                                                                                       |
|---------|--------------------------|--------------|-----------|------------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Main.py |                          | Global       | root      | Tk() Object      | Variable          | It contains the root window for the GUI as it will be written with Tkinter                                                                                                    |
| Main.py | Validate login()         | Local        | username  | String           | Variable          | Takes in and stores user input of username for login                                                                                                                          |
| Main.py | Validate login()         | Local        | Password  | String           | Variable          | Takes in and stores user input of password for login, this will then be hashed and so should produce the same output as the hash stored in the password field for the record. |
| Main.py | Validate login()         | Local        | Member    | Member () Object | Variable          | It stores all relevant information of the user such as their username, email, and role, fetched from the database after a search for its username.                            |
| Main.py | Add Member()             | Local        | username  | String           | Variable          | Stores username entry to add a new member into the database system.                                                                                                           |
| Main.py | Add Member()             | Local        | password  | String           | Variable          | Stores password entry to add a new member into the database system. Before it is added to the system it must be hashed and then the hash is entered into the system.          |
| Main.py | Add Member()             | Local        | email     | String           | Variable          | Stores email entry to add a new member into the database system. This must be checked to see whether the email is unique                                                      |
| Main.py | Get Member by Username() | Local        | username  | String           | Variable          | Stores user entry to search for a member in the database system.                                                                                                              |
| Main.py | Get Member by Username() | Local        | Member    | Member () Object | Variable          | Stores the fields returned by the SQL query search as an Object of the member class so it can be used.                                                                        |
| Main.py | Delete Member()          | Local        | Member_ID | Integer          | Variable          | Stores the unique user id entry dictating the user to be deleted.                                                                                                             |
| Main.py | Add Book()               | Local        | title     | String           | Variable          | Stores title entry to add a new book into the database system.                                                                                                                |
| Main.py | Add Book()               | Local        | Author    | String           | Variable          | Stores author entry to add a new book into the database system.                                                                                                               |
| Main.py | Add Book()               | Local        | Genre     | String           | Variable          | Stores genre entry to add a new book into the database system.                                                                                                                |
| Main.py | Add Book()               | Local        | ISBN      | String           | Variable          | Stores ISBN entry to add a new book into the database system, however this must be checked to see whether it is unique.                                                       |
| Main.py | Delete Book()            | Local        | Book_ID   | Integer          | Variable          | Stores the unique book id entry dictating which book to be deleted.                                                                                                           |
| Main.py | Fetch All Books()        | Local        | title     | String           | Variable          | Stores title entry of one book in the database to be fed into a member object title attribute.                                                                                |

|         |                        |        |                  |               |          |                                                                                                                                      |
|---------|------------------------|--------|------------------|---------------|----------|--------------------------------------------------------------------------------------------------------------------------------------|
| Main.py | Fetch All Books()      | Local  | Author           | String        | Variable | Stores author entry of one book in the database to be fed into a member object author attribute.                                     |
| Main.py | Fetch All Books()      | Local  | Genre            | String        | Variable | Stores genre entry of one book in the database to be fed into a member object genre attribute.                                       |
| Main.py | Fetch All Books()      | Local  | ISBN             | String        | Variable | Stores ISBN entry of one book in the database to be fed into a member object ISBN attribute.                                         |
| Main.py | Fetch All Books()      | Local  | Book             | Book() Object | Variable | Stores the fields returned by the SQL query as an Object of the books class so it can be appended to the end of a list of all books. |
| Main.py | Fetch All Books()      | Local  | Books            | List          | Variable | Contains a list of all books, stored as objects of the Book Class, to be returned so it can be used.                                 |
| Main.py | Search Book by Title() | Local  | title            | String        | Variable | Stores title entry to search for a book in the database system.                                                                      |
| Main.py | Loan Book()            | Local  | Book_ID          | String        | Variable | Stores id of the book, which is going to be checked out, so it can be adjusted in the database.                                      |
| Main.py | Loan Book()            | Local  | Member_ID        | String        | Variable | Stores id of the member who is checking out the book so the loan can be attributed to them.                                          |
| Main.py | Loan Book()            | Local  | Loan Date        | Integer       | Variable | Store the current UNIX time stamp to note when the book was loaned out.                                                              |
| Main.py | Loan Book()            | Local  | Due Date         | Integer       | Variable | Stores the UNIX time stamp to note when the book should be returned.                                                                 |
| Main.py | Return Book()          | Local  | Book_ID          | String        | Variable | Stores id of the book, which is going to be returned, so it can be adjusted in the database.                                         |
| Main.py | Get All Member Loans() | Local  | Member_ID        | String        | Variable | Stores id of the member so an SQL query can return all of those user's loans.                                                        |
| Main.py | Get All Member Loans() | Local  | Loans            | List          | Variable | Contains all of the active loans of a particular user and returns that list.                                                         |
| Main.py | Sort Books by genre()  | Local  | Sorted Books     | List          | Variable | Contains all of the books which have the desired genre.                                                                              |
| Main.py | Sort Books by genre()  | Local  | Genre            | String        | Variable | Stores the user entry for the genre filter for sorting the books by genre.                                                           |
| Main.py | Library app            | Global | Is High Contrast | Boolean       | Variable | Boolean (Yes/No) variable which is used to toggle between the normal and high visibility colour scheme                               |

| <b>Class</b>                                                          | <b>Attribute/Method</b>         | <b>Private/Public</b> | <b>Type</b> | <b>Purpose</b>                                                                                                                                                                     |
|-----------------------------------------------------------------------|---------------------------------|-----------------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Book                                                                  | Title attribute                 | Public                | string      | Contains the title of the book which is being looked at.                                                                                                                           |
| Book                                                                  | Author attribute                | Public                | string      | Contains the author of the book which is being looked at.                                                                                                                          |
| Book                                                                  | Genre attribute                 | Public                | string      | Contains the genre of the book which is being looked at.                                                                                                                           |
| Book                                                                  | ISBN attribute                  | Public                | string      | Contains the unique ISBN of the book which is being looked at.                                                                                                                     |
| Member                                                                | Username attribute              | Public                | string      | Contains the username of a member, either logged in or a new member being added.                                                                                                   |
| Member                                                                | Email attribute                 | Public                | string      | Contains the unique email address of a member, either logged in or a new member being added.                                                                                       |
| Member                                                                | Hashed password attribute       | Bytes                 | string      | Can only be pulled out of the database, used for comparison against user entry login validation.                                                                                   |
| Member                                                                | Role attribute                  | Public                | string      | Contains the role of a member, for role-based access control.                                                                                                                      |
| DatabaseManager<br>(parent class for Book and Member Manager classes) | conn() method                   | Public                | Method      | A base class to create SQLite connections                                                                                                                                          |
| Database manager                                                      | Create tables() method          | Public                | Method      | Base method to create a database table, but like to be overridden by child classes                                                                                                 |
| Member Manager                                                        | Create tables() method          | Public                | Method      | Creates the member table with id, username, email, hashed password and role fields.                                                                                                |
| Member Manager                                                        | Add Member() method             | Public                | Method      | Method which takes in a new member's information and adds it to the member table.                                                                                                  |
| Member Manager                                                        | Get Member by Username() method | Public                | Method      | Method which takes in a member's username and does an SQL query to search for the user in the member table and if it is found, create a member class object with that information. |
| Member Manager                                                        | Delete Member() method          | Public                | Method      | Method which takes in a member ID and then attempts to find and delete the member from the database.                                                                               |
| Book Manager                                                          | Create tables() method          | Public                | Method      | Creates the books table with id, title, author and ISBN fields. It also creates a loan table with a Book ID, member id, loan date, due date and returned statement fields.         |

|              |                                      |        |         |                                                                                                                                                           |
|--------------|--------------------------------------|--------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Book Manager | Add Book() method                    | Public | Method  | Method which takes in a new book's information and adds it to the book table.                                                                             |
| Book Manager | Delete Book() method                 | Public | Method  | Method which takes in a book ID and then attempts to find and delete the book from the database.                                                          |
| Book Manager | Fetch All Books() method             | Public | Method  | Method which selects everything from the books table and returns them all as objects of the Book class, which are then added to a list of all the books.  |
| Book Manager | Search Book by Title() method        | Public | Method  | Method which takes in a user title entry and does a SQL query for titles LIKE the entered title.                                                          |
| Book Manager | Quick Sort() and Partition() methods | Public | Methods | Methods which do a quick sort to sort the ISBNs of the books so a binary search can be done on them.                                                      |
| Book Manager | Binary search() method               | Public | Method  | Method which finds a book by its ISBN by using a binary search after it has been sorted.                                                                  |
| Book Manager | Merge() and Merge sort() method      | Public | Method  | Method which does a merge sort to sort the books alphabetically for reorganising.                                                                         |
| Book Manager | Sort books by Genre() method         | Public | Method  | Method which returns all books of a certain genre, entered by the user, and is then sorted alphabetically using a quick sort so books can be reorganised. |
| Library App  | Create widgets() method              | Public | Method  | Initialises the GUI layout of the system depending on the user's role.                                                                                    |

## Validation

A main validation in my program is during the add member function. To enforce secure passwords and improve account security, a password entry will be checked to see whether it is at least eight characters long. Another source of validation in my program is that email entries should be checked to see whether a valid email address has been entered in the correct format (such as [abc@xyz.com](mailto:abc@xyz.com)) before a new member can be created. This will be done by using a regular expression comparison, to ensure that the string starts with a valid username part (“abc” in the previous example) and is then followed by the “@” symbol and is then followed by a domain name (such as “xyz” in the above example and then ends with a top-level domain name such as “.com” or “.org.” I must also ensure that there are no duplicate emails or ISBN values in the database, to avoid an SQL Integrity error. To avoid crashing the system a try and except block should be placed here with an appropriate error message being relayed to the user. The system should also check that there is a database, which contains the right tables, before trying to modify or transverse the database, to prevent any errors from occurring from while the program runs. I also need to make sure that certain values such as “book\_id” entry values are of the right type, such as in this instance an integer value rather than a string value. This should also be valid using a try and except block to prevent unrecognised types from passing through. As well as this, edge cases from algorithms such as outputs from books which return nothing, this should instead return a statement that no books were found.

## Testing Plan for Development Cycle

At each stage of my iterative development cycle, each feature of the program to ensure they work correctly for any inputs, including special cases like edge cases. I will ensure that inputs which are valid have do their required function and note whether this does not happen, with screenshots for when I debug. I will also test for incorrect or invalid inputs to ensure there are not any false positives such as users being able to get into the program without the correct login information.

| Test ID | What is being tested                                        | Test Data                                       | Expected Outcome                                                                       | Actual Outcome (and evidence) | Success/Fail |
|---------|-------------------------------------------------------------|-------------------------------------------------|----------------------------------------------------------------------------------------|-------------------------------|--------------|
| T-01.1  | A valid username and password were entered for a librarian. | Username= “KentC”<br>Password=”12345678”        | Librarian “KentC” logged into the system and the librarian management screen is shown. |                               |              |
| T-01.2  | A valid username and password were entered for a member.    | Username= “john_doe”<br>Password=”password123”  | Member “john_doe” is logged into the system and the member management screen is shown. |                               |              |
| T-02.1  | An invalid username was entered.                            | Username=” ClarkK”<br>Password=”12345678”       | Incorrect username or password was entered. – User cannot enter the system.            |                               |              |
| T-02.2  | An invalid password was entered.                            | Username=” KentC”<br>Password = “notmypassword” | Incorrect username or password was entered. – User cannot enter the system.            |                               |              |

|        |                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                    |  |  |
|--------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
| T-02.3 | Empty Strings were entered for the login page.                              | Username=""<br>Password=""                                                                                                                                                                                                                                                                                                                                                                     | Incorrect username or password was entered. – User cannot enter the system.                                                                                                                                                                                        |  |  |
| T-03   | On initial run, a books and members database are created.                   | None                                                                                                                                                                                                                                                                                                                                                                                           | A new books and member database file is created.                                                                                                                                                                                                                   |  |  |
| T-04.1 | A member can be added to the system, when provided with normal information. | Username="test"<br>Email= <a href="mailto:test@test.com">test@test.com</a><br>Password="12345678"                                                                                                                                                                                                                                                                                              | Member added successfully.                                                                                                                                                                                                                                         |  |  |
| T-04.2 | A duplicate email is provided to the add member.                            | Username="test"<br>Email=" <a href="mailto:test@test.com">test@test.com</a> "<br>Password="12345678"                                                                                                                                                                                                                                                                                           | You should get an error message stating that the email is not unique.                                                                                                                                                                                              |  |  |
| T-04.3 | An invalid email is entered to the system.                                  | Username="test"<br>Email="@abc.com"<br>Password="12345678"<br><br>Username="test"<br>Email= <a href="mailto:test.com">test.com</a><br>Password="12345678"<br><br>Username="test"<br>Email= <a href="mailto:test@.com">test@.com</a><br>Password="12345678"                                                                                                                                     | An error messaging showing that an invalid email address was entered.                                                                                                                                                                                              |  |  |
| T-04.4 | The user's entered password is less than 8 characters.                      | (less than 8 characters)<br>Username="test"<br>Email=" <a href="mailto:test@abc.com">test@abc.com</a> "<br>Password="1234"<br><br>(exactly 8 characters)<br>Username="test"<br>Email= <a href="mailto:test@abc.com">test@abc.com</a><br>Password="12345678"<br><br>(Edge case: 7 characters)<br>Username="test"<br>Email= <a href="mailto:test@abc.com">test@abc.com</a><br>Password="1234567" | Return that your entered "password is less than 8 characters".<br><br>Password is greater than 8 characters so provided other inputs are okay, member should be added.<br><br>Password is smaller than 8 characters so print (Password is less than 8 characters). |  |  |
| T-05.1 | A valid user id is entered to delete a member from the database.            | User ID="13"                                                                                                                                                                                                                                                                                                                                                                                   | User number 13 has been deleted from the system. This should be checked by trying to run the program again.                                                                                                                                                        |  |  |
| T-05.2 | An invalid user id is entered to be deleted.                                | User ID ="9999"                                                                                                                                                                                                                                                                                                                                                                                | There is no user with that ID.                                                                                                                                                                                                                                     |  |  |

|        |                                                                                          |                                                                                                                                     |                                                                                                                                                                                                                                                                 |  |  |
|--------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
| T-05.3 | A user id entry of an invalid type (such as string) is entered.                          | User ID =" David"                                                                                                                   | The member ID should be a number.                                                                                                                                                                                                                               |  |  |
| T-06.1 | A book can be added to the system, when provided with normal information.                | Title= "The Hunger Games"<br>Author = "Suzanne Collins"<br>Genre = "Dystopian"<br>ISBN= "9780439023528"                             | The book should be added to the books table                                                                                                                                                                                                                     |  |  |
| T-06.2 | A duplicate email is provided to the add book function.                                  | Title= "The Hunger Games 2"<br>Author = "Suzanne Collins"<br>Genre = "Dystopian"<br>ISBN= "9780439023528"<br><br>#Note after T-06.1 | You should get an error message stating that the ISBN is not unique.                                                                                                                                                                                            |  |  |
| T-07.1 | A valid Book id is entered to delete a book from the database.                           | Book ID="8"                                                                                                                         | Book number 8 has been deleted from the system. This should be checked by trying to run the program again.                                                                                                                                                      |  |  |
| T-07.2 | An invalid book id is entered to be deleted.                                             | Book ID ="9999"                                                                                                                     | There is no book with that ID.                                                                                                                                                                                                                                  |  |  |
| T-07.3 | A book id entry of an invalid type (such as string) is entered.                          | Book ID =" The Hunger games"                                                                                                        | The book ID should be a number.                                                                                                                                                                                                                                 |  |  |
| T-08.1 | The user enters a valid book title, and the right book is returned.                      | Title=" To Kill a Mockingbird"                                                                                                      | Information about the book, its ID number, title, author, genre, ISBN, whether it is checked in or not.                                                                                                                                                         |  |  |
| T-08.2 | The user enters part of a book title and the books that match that keyword are returned. | Title=" of"                                                                                                                         | Information (see above) about the following books:<br>"The Life of Pi"<br>"The Lord of the Rings"<br>"The Picture of Dorian Gray"<br>"The Adventures of Huckleberry Finn"<br>"The Chronicles of Narnia"<br>(These are the only books with "of" in their title). |  |  |
| T-08.3 | The user enters a book not in the title.                                                 | Title = "Superman"                                                                                                                  | This book has not been found.                                                                                                                                                                                                                                   |  |  |
| T-09.1 | The user enters a correct book and member id when loaning out a book.                    | Book ID ="1"<br><br>Member ID ="1"                                                                                                  | Book loaned out successfully.                                                                                                                                                                                                                                   |  |  |

|        |                                                                                            |                                                                                          |                                                                                                                                                                        |  |  |
|--------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
| T-09.2 | The user attempts to loan out a book which is already loaned out.                          | Book ID= "1"<br>(after checking it is loaned out after test T-07.1)<br><br>Member ID="2" | This book is already loaned out.                                                                                                                                       |  |  |
| T-09.3 | The user does not enter a Book ID.                                                         | Book ID = ""<br>Member ID ="1"                                                           | The Book ID must be a number.                                                                                                                                          |  |  |
| T-09.4 | The user does not enter a member ID.                                                       | Book ID = "2"<br>Member ID =""                                                           | The member ID must be a number.                                                                                                                                        |  |  |
| T-09.5 | The user enters an incorrect member ID                                                     | Book ID ="2"<br>Member ID ="9999" (a member ID which does not exist)                     | This is not a valid member.                                                                                                                                            |  |  |
| T-09.6 | The user enters an incorrect Book ID                                                       | Book ID == "9999" (a book which does not exist)<br>Member ID==1                          | This is not a valid Book ID.                                                                                                                                           |  |  |
| T-09.7 | The loan due date calculated should accurately be 14 days from the loan date               | None                                                                                     | A new loan has a due date 14 days after a loan date.                                                                                                                   |  |  |
| T-10.1 | The user enters a valid genre, and the right books are filtered.                           | Genre=" Fiction"                                                                         | Information on the following books:<br>"The Catcher in the Rye"<br>"The Great Gatsby"<br>"To Kill a Mockingbird"<br>"The Old Man and the Sea"<br>"The Sun also Rises". |  |  |
| T-10.2 | The user does not enter a genre.                                                           | Genre=""                                                                                 | All books should be outputted in alphabetical order.                                                                                                                   |  |  |
| T-10.3 | The user enters a genre that is not found in the database                                  | Genre = "Horror"                                                                         | No books found of this genre.                                                                                                                                          |  |  |
| T-11   | Depending on the user, a different menu scheme should be                                   | Login on both a privileged librarian account and a member account.                       | The librarian account should have access to the full features whereas the member should have access to a more restricted feature list.                                 |  |  |
| T-12   | The system should be able to switch between the high visibility mode and the normal theme. | Click the high visibility mode button                                                    | One click should toggle the high visibility mode on, and the widgets should change and when clicked again it should switch back.                                       |  |  |

|        |                                                                                                                                     |                                      |                                                                   |  |  |
|--------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|-------------------------------------------------------------------|--|--|
| T-13.1 | A valid ISBN is provided when searching by ISBN.                                                                                    | ISBN= "9780446310789"                | Information on that book (title, author, genre ISBN) is returned. |  |  |
| T-13.2 | An ISBN which is not in the book table is entered when searching by ISBN.                                                           | ISBN="9780525559474"                 | This ISBN cannot be found in the table.                           |  |  |
| T-14.1 | A book with a valid loan is entered when attempting to return a book.                                                               | BookID ="23"                         | This book has been successfully returned.                         |  |  |
| T-14.2 | A book which does not have a valid loan is entered when attempting to return a book                                                 | Book ID = "14"                       | This book does not have an active loan.                           |  |  |
| T-14.3 | An invalid book ID is entered when trying to return a book.                                                                         | Book ID = "99999"                    | This book does not have an active loan.                           |  |  |
| T-14.4 | A book ID of an invalid type is entered when trying to return a book.                                                               | Book ID = "The Chronicles of Narnia" | The Book ID should be an integer                                  |  |  |
| T-15.1 | When a valid member ID is entered into the view member loans feature all of their current and previous loans are shown.             | Member ID ="1"                       | Information about their current and previous loans                |  |  |
| T-15.2 | When a user who has not taken out any loans is entered into the feature, it is printed that this user has not had any loans before. | Member ID ="3"                       | Printed that this user has not had any loans before.              |  |  |
| T-15.3 | When a user id of an invalid type is entered into the view members loans function                                                   | Member ID ="David"                   | The member ID should be a number                                  |  |  |

## Post Development Testing with Required Data

After doing my development testing and finishing my program, I will contact my stakeholders to gain feedback on their experience with the program. This is likely to help to find any errors which I did not know about myself as well as help to achieve a program that better suits my stakeholder's needs as their requirements may be different than mine.

This will be achieved using a survey where I will let my stakeholders use my program and then give their opinion in relation to the following questions on a 1–5 star ranking system based on the requirements from my analysis section. My stakeholders will also be given the opportunity to provide any additional feedback if they wish.

| Requirement No. | Requirement                                                                                                                                    | Success Criteria                                                                                                                                                                                             | Stakeholder Question                                                                                                                        |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 1               | A clean, simple to use GUI                                                                                                                     | The GUI should include Intuitive navigation and should be consistent, making it easy for users to understand how to use it. It should also avoid clutter. This should be evidenced by positive user feedback | On a ranking from 1-5 stars how intuitive did the Graphical User Interface feel, and did you have any issues navigating the user interface? |
| 2               | The user should be able to monitor what books they have taken out and how long they have until they must return it.                            | This would include having a user system, which would link the books they have checked out to the user as well as logging their check out date.                                                               | On a ranking from 1-5 stars how do you rate the view member loans feature?                                                                  |
| 3               | New books and old books and media should be able to be catalogued and removed and media types should have their associated information logged. | A database should be created and easily editable containing all books and information: author, release date ISBN etc.                                                                                        | How would you rank the add and remove books feature of the program and why?                                                                 |
| 4               | Should be able to search for different books                                                                                                   | A working search function for books                                                                                                                                                                          | How would you rank the different searching features of the program on a scale from 1-5 and why?                                             |
| 5               | Should be able to sort the books into different categories.                                                                                    | Books should be able to be sorted into different categories, such as genre, author, release date etc.                                                                                                        | How would you rank the different sorting features of books in the program on a scale from 1-5 and why?                                      |

|   |                                               |                                                                                                                                                                                                                           |                                                                                                       |
|---|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| 6 | Books should be able to be checked in and out | Users must be able to check in and out books, with a loan date and return date being generated. The same book should not be able to be checked out multiple times, unless there are multiple quantities of the same book. | How would you rank the loan book and return book features of the program on a scale from 1-5 and why? |
|---|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|

# Development

## Development Cycle Part I

### Code Development

To begin, I created the login system for the program itself. Initially I coded the logic for the system, using a test case username and password which were hard coded into the program code. I then took in user entries for the username and password, using a command line user interface as I had not designed the graphical user interface yet. Following this I then used the bcrypt module to create a hash of the password (line 5). However, I faced an issue when trying to compare the two hashed values. I initially tried to use an if statement, where I hashed the user entry password and if it matched the stored hash password and the username matched the username entry it would then print that the user is welcome to the system. However, even if I entered the right password, it would always print that an invalid username or password was entered into the system. This ended up being due to the comparison which I did. This is because each time the hash password function (line 5 and line 10) it would generate a new hash with a new salt meaning that they would never match as it mean that the hash would always be unique. To fix this, I used a different method, the bcrypt.checkpw() function instead. This method extracts the salt from the hashed password and then uses that salt to hash the user's input which would then generate the same hash and so they hashes would match. Following this, the logic was correct and if the right user information were entered, users could be correctly entered into the system, passing tests T-01.1, T-02.1, T-02.2 and T-02.3.

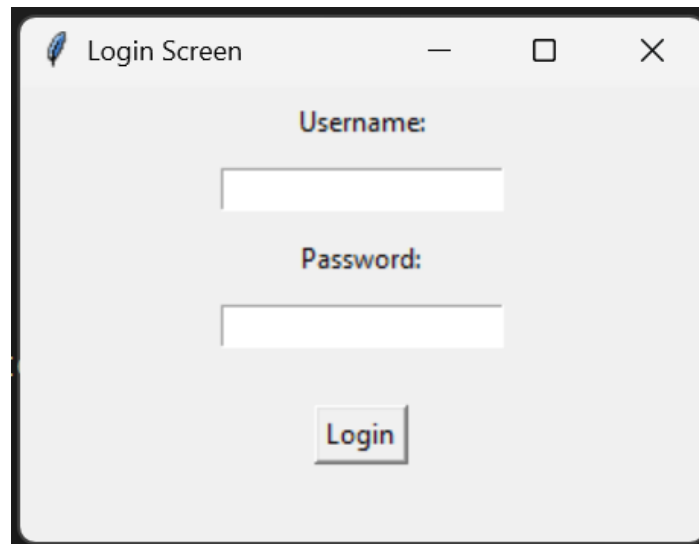
```
1 import bcrypt
2
3 username = "test"
4 password = "12345678"
5 hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
6
7 entry_username = input("Enter your username: ")
8 entry_password = input("Enter your password: ")
9
10 if entry_username == username and bcrypt.hashpw(entry_password.encode('utf-8'), bcrypt.gensalt()) == hashed_password:
11     print("Welcome to the system")
12 else:
13     print("Invalid username or password")
```

Incorrect method, error on line 10

```
1 import bcrypt
2
3 username = "test"
4 password = "12345678"
5
6 hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
7
8 entry_username = input("Enter your username: ")
9 entry_password = input("Enter your password: ")
10 if bcrypt.checkpw(entry_password.encode('utf-8'), hashed_password) == True and entry_username == username:
11     print("welcome to the system")
12 else:
13     print("invalid username or password")
```

Correct method, error fixed at line 9

Following this, I used the Tkinter module to create a basic login menu for the login screen. It consists of a 300 by 200 window with the title login screen, and then text user inputs for the username and password and then a log in button which when clicked runs the validate login function, which contained the same logic as the function above. Rather than printing in the terminal, the program now shows a message box with an either a blue tick and the “welcome to the system” as before or an error message with a red cross with the “incorrect username or password” as seen before. I then did another test to see whether it passed the same tests as before (tests T-01.1, T-02.1, T-02.2 and T-02.3).



The Login Menu

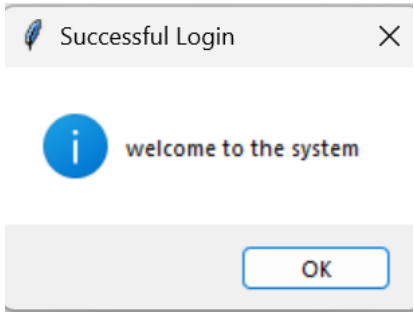
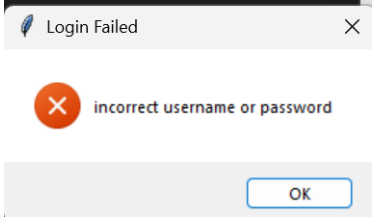
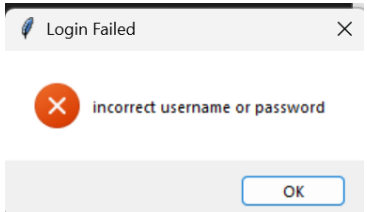
```

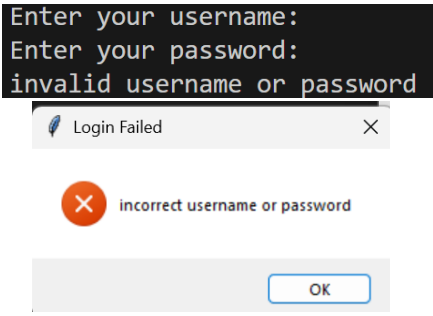
1  import bcrypt
2  import tkinter as tk
3  from tkinter import messagebox
4
5  username = "test" #hardcoded test username
6  password = "12345678" #hardcode test password
7  hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
8  def validate_login():
9      entry_username = entered_username.get()
10     entry_password = entered_password.get()
11     #hashes the user entered password using the salt from the hashed password
12     if bcrypt.checkpw(entry_password.encode('utf-8'), hashed_password) == True and entry_username == username:
13         messagebox.showinfo("Successful Login", "welcome to the system".format(username))
14
15     else:
16         messagebox.showerror("Login Failed", "incorrect username or password")
17
18  root = tk.Tk()
19  root.title("Login Screen")
20  root.geometry("300x200")
21
22  # username entry
23  label_username = tk.Label(root, text="Username:")
24  label_username.pack(padx=5, pady=5)
25  entered_username = tk.Entry(root)
26  entered_username.pack(padx=5, pady=5)
27
28  # password entry
29  label_password = tk.Label(root, text="Password:")
30  label_password.pack(padx= 5,pady=5)
31  entered_password = tk.Entry(root)
32  entered_password.pack(padx=5, pady=5)
33
34  #login button
35  login_button = tk.Button(root, text="Login", command=validate_login)
36  login_button.pack(pady=20)
37  #tkinter main loop
38  root.mainloop()

```

Validate Login page

## Testing Table and Evidence

| Test ID | What is being tested                                        | Test Data                               | Expected Outcome                              | Actual Outcome (and evidence)                                                                                                                                                                                                                                                                      | Success/Fail |
|---------|-------------------------------------------------------------|-----------------------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| T-01.1  | A valid username and password were entered for a librarian. | Username="test"<br>Password="12345678"  | "Welcome to the system" should be printed     | <p>When correct user information is entered, the user is let into the system?</p> <pre>Enter your username: test Enter your password: 12345678 welcome to the system</pre>                                       | Success      |
| T-02.1  | An invalid username was entered.                            | Username="test1"<br>Password="12345678" | "Incorrect username or password was entered." | <p>When an incorrect username is entered it printed that an incorrect username or password was submitted.</p> <pre>Enter your username: test1 Enter your password: 12345678 invalid username or password</pre>  | Success      |
| T-02.2  | An invalid password was entered.                            | Username="test"<br>Password="1234"      | "Incorrect username or password was entered." | <p>When an incorrect password is entered, it printed that an incorrect username or password was submitted.</p> <pre>Enter your username: test Enter your password: 1234 invalid username or password</pre>     | Success      |

|        |                                                |                            |                                                                             |                                                                                                                                                                                              |         |
|--------|------------------------------------------------|----------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| T-02.3 | Empty Strings were entered for the login page. | Username=""<br>Password="" | Incorrect username or password was entered. – User cannot enter the system. | <p>When no information is submitted, it printed that an incorrect username or password was submitted.</p>  | Success |
|--------|------------------------------------------------|----------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|

## Key Stage Review

This is the beginning of the development process of my program. While important process has been made in beginning the development process, I still do not have a valid database system, nor any of the required library specific features. It is also important to note that I was not able to fully test T-01.1, and I could not test T-01.2 as I had only one hard coded username and password. It is therefore important to incorporate a database so any user can be logged in to the system, using their appropriate passwords.

As a result of this stage, I have made progress in Success criteria S001 as I have created part of the graphical user interface as well as making progress on the login page, Success Criteria S005 as most of the login functionality is complete.

| Success Criteria ID | Success Criteria                                                                                                                                    | Relevant Feature ID | Further Justification                                                                                                                                                                                                                                                                                             |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S001                | There will be a graphical user interface where every button works.                                                                                  | F1                  | This is fundamental to making the program easy and intuitive to use. Using a different interface such as a command line interface may be hard to understand and grasp for members and librarians and a natural user interface is likely to be inconvenient to use and so slow down the productivity of librarians |
| S002                | There should be a database containing tables for books, users and loans                                                                             | F2                  | Having a database with these features allows the data to be easily portable, in one or two files. This is also the easiest to edit and so it is easy to keep a well-maintained store of all required information                                                                                                  |
| S003                | There should be a function to search for books by name and by their ISBN and then the book and relevant information should be outputted to the user | F3                  | This enables members and librarians to check what books are in stock as well as information such as the books name or genre, essential to the operation of a library management system.                                                                                                                           |
| S004                | There should be a function to sort books by genre and output them alphabetically                                                                    | F3                  | This allows members to search for books with their desired genre, but also to help the organisation of the library as the books are organised by genre then in alphabetical order                                                                                                                                 |
| S005                | There should be a log in page                                                                                                                       | F4                  | This aims to protect sensitive information or functions from unauthorised users and allows to dictate between member and librarian users – note Feature F5.                                                                                                                                                       |
| S006                | There should be Role Based Access control (between normal                                                                                           | F5                  | Primarily successful through success criteria S005. This is essential as there are features that librarians                                                                                                                                                                                                       |

|      |                                                                                                                                                                                                                                                               |    |                                                                                                                                                                                                                                     |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      | members and privileged librarian accounts) which have access to different features                                                                                                                                                                            |    | should be able to access to enable them to do their work that would be useless or even cause security risks for normal member users to have access to such as adding or deleting books or members.                                  |
| S007 | There should be a feature to add or remove books                                                                                                                                                                                                              | F6 | Fundamental to the job of a librarian, adding and removing books as well as its metadata such as the name of the author and its genre. This is necessary to improve the database and enable users to see which books are available. |
| S008 | There should be a feature to borrow and return books. It should check whether the book is in the database and prevent already checked out books from being checked out again. It should also log which member logged out the book according to its member ID. | F7 | Also fundamental to the operation of a library. Enables users to check books in and out themselves, relieving this duty from librarians. Also, it will add to a loans table, allowing librarians to check loans.<br>Note- also F8   |
| S009 | There should be a feature to add and remove new members                                                                                                                                                                                                       | F9 | This allows for new members to be added to the system, particularly at the start of the school year such as new sixth form students                                                                                                 |

### Key

Green = Complete

Orange = Progress has been made

Red = Not done

## Development Cycle Part 2

### Code Development

In this section I began the creation of my database management systems. As I am going to include two different managers (Member Manager for members) and (Book manager for books and loans) I began by creating a base parent class called Database Manager. This removes the need to copy different methods into the two manager systems. This database manager contained a base method for creating the database table which did not have any code as the different managers would generate two different tables. It also contained the method to connect to the sqlite table using the `sqlite3.connect(db_name)` method to connect to the table which is passed through as a parameter and included the method to close the connection between the table, called “close.”

```
#parent class for database managers
class DatabaseManager:
    def __init__(self, db_name):
        #connects to database which is passed in a parameter
        self.conn = sqlite3.connect(db_name)
        self.create_tables()
    def create_tables(self):
        pass
    #closes connection with database
    def close(self):
        self.conn.close()
```

Database manager class

Following this I created the Member manager child class which inherits from the database manager as a parent. I then created the new `create_tables()` method which would override the method inherited from the database manager. This method creates a cursor and creates a table if it doesn't exist into a members database file with fields for a member ID (which is the primary key, an integer and is set to automatically increment when a new member is added), the username, which must be text which is unique and is not empty, an email which also must be unique and not empty, a password field which contains the hashed user password (although this hashed password would be stored in bytes), and a role text file which is either librarian or member. I decided to keep this as a text file rather than making it a Boolean True or false statement to check whether the user is a privileged librarian or not to allow for different roles with different privileges to be added in the future. I then ran the program to check whether the database was created correctly with the right fields and then manually added some users to allow to incorporate testing with the login system.

| id | username   | email            | password                                                 | role      |
|----|------------|------------------|----------------------------------------------------------|-----------|
| 1  | john_doe   | john@doe.com     | 36,50,98,36,49,50,36,113,65,89,120,82,70,117,65,78,5...  | member    |
| 2  | jane_smith | jane@smith.com   | 36,50,98,36,49,50,36,119,121,107,121,50,114,110,98,...   | member    |
| 3  | alice      | alice@gmail.com  | 36,50,98,36,49,50,36,72,48,111,55,77,68,116,120,72,8...  | member    |
| 4  | tobi       | tobi@testing.com | 36,50,98,36,49,50,36,50,73,85,79,121,84,111,120,57,1...  | member    |
| 5  | alex       | alex@bacon.com   | 36,50,98,36,49,50,36,107,49,97,55,113,89,116,55,69,7...  | member    |
| 7  | KentC      | clark@kent.com   | 36,50,98,36,49,50,36,71,71,72,54,98,47,106,80,113,88,... | librarian |

Visual representation of the members table (note the password is stored in bytes.

To succeed with Test T-01.1 and T-01.2 as well as satisfying my basic stakeholder requirements I need to have a working login system with the ability for different users to sign in, which I was not able to implement in Development Cycle Part I. To implement the feature of allowing different users to log into the system, I created a new method of the Member Manager system to grab information on the user based on searching for their username. I also created a member class which contains attributes of the users' username, email, hashed password and role. The get member by username method works by searching the database for the record containing the username entered and returning everything (all the fields) from the member table. This information is then used to create a

member class object I then adjusted the validate login function to then get the associated member based on the username entered, as well as removing the hardcoded username and password. However, I faced an issue while testing when a username entered is not in the database. This would mean that the returned member object was null as there was no information pulled from the database, meaning that it did not have any attributes causing an None type error when comparing the two passwords. To fix this I encased the if statement with another if statement which checked whether the member was not equal to None, in that case checking whether the password matched and otherwise printing that the login failed. I also realised that this means that the “and entry\_username == get\_member.username” logic in the if statement is not necessary as if this if statement is even being run, the username must match A username in the database and so it only needs to compare whether the password matches, which is done in the first half of that if statement.

```
def validate_login():
    entry_username = entered_username.get()
    entry_password = entered_password.get()
    member_manager = MemberManager("members2.db")
    get_member = member_manager.get_member_by_username(entry_username)

    #hashes the user entered password using the salt from the hashed password
    if bcrypt.checkpw(entry_password.encode('utf-8'), get_member.hashed_password) == True and entry_username == get_member.username:
        messagebox.showinfo("Successful Login", "{} has logged into the system, your role is {}".format(get_member.username, get_member.role))
    else:
        messagebox.showerror("Login Failed", "incorrect username or password")
```

Original incorrect update to the validate login function, with support for the members table

```
def validate_login():
    #take username and password entry
    entry_username = entered_username.get()
    entry_password = entered_password.get()
    # create object of the member manager class and connect to the members table
    member_manager = MemberManager("members2.db")
    get_member = member_manager.get_member_by_username(entry_username)

    if get_member != None:
        #hashes the user entered password using the salt from the hashed password
        if bcrypt.checkpw(entry_password.encode('utf-8'), get_member.hashed_password) == True:
            messagebox.showinfo("Successful Login", "{} has logged into the system, your role is {}".format(get_member.username, get_member.role))
        else:
            messagebox.showerror("Login Failed", "incorrect username or password")
    else:
        messagebox.showerror("Login Failed", "incorrect username or password")
```

Updated version validate login function, with support for the members table and alterations to logic

```
class Member:
    def __init__(self, username, email, hashed_password, role):
        self.username = username
        self.email = email
        self.hashed_password = hashed_password
        self.role = role # 'member' or 'librarian'
```

Member Class

```
class DatabaseManager:
    def __init__(self, db_name):
        #connects to database which is passed in a parameter
        self.conn = sqlite3.connect(db_name)
        self.create_tables()
    def create_tables(self):
        pass
    #closes connection with database
    def close(self):
        self.conn.close()
```

Database Class

```

class MemberManager(DatabaseManager):
    #create the library member table if needed, particularly when run for the first time, prevents error when modifying a database which does not exist.
    def create_tables(self):
        cursor = self.conn.cursor()
        cursor.execute('''CREATE TABLE IF NOT EXISTS members
        (id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE NOT NULL,
        email TEXT UNIQUE NOT NULL,
        password TEXT NOT NULL,
        role TEXT NOT NULL)''')
        self.conn.commit()

    #function to search the database for a certain user and create a member object with their information
    def get_member_by_username(self, username):
        cursor = self.conn.cursor()
        cursor.execute('''SELECT * FROM members WHERE username = ?''', (username,))
        #creates a list of the user information
        row = cursor.fetchone()
        if row != None: # if the user is found create a member object with their information
            return Member(row[1], row[2], row[3], row[4])
        return None # if no user

```

Member Manager class which inherits from database manager however overrides the base create tables class with its own. Also contains the get\_member\_by\_username() method.

I then began to create the add member and delete member functions of my program. This was also done in the terminal as I had not programmed the GUI yet. These are both methods of the member manager class. The add member method works by taking in user entries for the username, email, and password for a new user and then attempts to insert this into the members table. However, I faced an issue whether the program would crash if a username or email which is already in the database is used as an entry. This is because it caused an integrity error as the username and email address are supposed to be unique. To fix this, I wrapped the code in a try and except block, and so if an integrity error is raised, it returns to the user that the username or email address they had entered is not unique, prompting them to change it. Otherwise, the user is correctly added to the system, with the user ID being grabbed from the database and a successful addition message being printed along with the new user's id. In the main program, it creates a new member manager object and then takes in user entries for the username email and password. When a valid email address and password is entered, it will then hash the password and create a new member object with this information. It is important to note that because this is a "add\_member" method, users will automatically get set the member role and must be manually adjusted to the librarian role. I also used the inbuilt python function .strip() to remove any unneeded whitespace from the username and email entries and then tested function to see whether users were added to the system.

|    |         |                  |                                                          |        |
|----|---------|------------------|----------------------------------------------------------|--------|
| 16 | ParkerP | parkerp@test.com | 36,50,98,36,49,50,36,88,97,55,73,80,49,122,79,51,53,5... | member |
|----|---------|------------------|----------------------------------------------------------|--------|

Successful test for test T-04.I

```

def add_member(self, member):
    #connects to database with cursor c
    cursor = self.conn.cursor()
    try:
        # add new user into the members database, using hashed password
        cursor.execute('''INSERT INTO members (username, email, password, role) VALUES (?, ?, ?, ?)''',
        (member.username, member.email, member.hashed_password, member.role))
        self.conn.commit()
        #grabs the primary key from the last row, i.e. the row that was just added.
        user_id = cursor.lastrowid
        return "The member {} has been added to the system. Their Member ID= {}".format(member.username, user_id)
    except sqlite3.IntegrityError: return "The email address or username you have entered is not unique"

```

Member manager add\_member()

```
def add_member():
    member_manager=MemberManager("members2.db")
    username = input("Enter username: ").strip() #remove any whitespace
    email = input("Enter email: ").strip()
    password = input("Enter password")

    #regular expression, to validate whether an email address was entered
    email_regex = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'

    #ensuring valid email and password
    email_check=False
    password_check = False

    while email_check== False:
        if re.match(email_regex, email)== None:
            print("Invalid Email", "You have entered an invalid email address.")
            email = input("Enter email: ").strip()
        else:
            email_check =True
    while password_check == False:
        if len(password) < 8: # make sure the password is at least 8 characters long
            print("Weak Password", "Your password should be at least 8 characters long.")
            password = input("Enter password")
        else:
            password_check= True
```

In line add\_member() function

I then developed the delete member function, which works in a similar way to the add member function with a method within the member manager class as well as an inline function. The delete member method works by taking in a member ID as a user entry and then attempting the SQL query to 'DELETE from members WHERE id = member\_ID' This query will either return one row, as the member ID is unique or none at all, when there is not a member with that member ID, so it will return a statement that the user has entered an incorrect member id, or will commit the changes and then return that the user has been successfully deleted from the table. The inline delete member function then handles taking in the member id user entry. However, I came into an issue when testing test T-05.3. This is because the program would crash when the member ID entry was not an integer number as the type casting to an integer would not work. To fix this, I wrapped the user entry in a try and except block so if the user does not enter an ID of a valid type (integer) it would print that the member ID should be a number to avoid the value error.

| id | username   | email            | password                                                | role   |
|----|------------|------------------|---------------------------------------------------------|--------|
| 1  | john_doe   | john@doe.com     | 36,50,98,36,49,50,36,113,65,89,120,82,70,117,65,78,5... | member |
| 2  | jane_smith | jane@smith.com   | 36,50,98,36,49,50,36,119,121,107,121,50,114,110,98,...  | member |
| 3  | alice      | alice@gmail.com  | 36,50,98,36,49,50,36,72,48,111,55,77,68,116,120,72,8... | member |
| 4  | tobi       | tobi@testing.com | 36,50,98,36,49,50,36,50,73,85,79,121,84,111,120,57,1... | member |

Before deleting member 3 (Username=Alice) Test T-05.1

| id | username   | email            | password                                                | role   |
|----|------------|------------------|---------------------------------------------------------|--------|
| 1  | john_doe   | john@doe.com     | 36,50,98,36,49,50,36,113,65,89,120,82,70,117,65,78,5... | member |
| 2  | jane_smith | jane@smith.com   | 36,50,98,36,49,50,36,119,121,107,121,50,114,110,98,...  | member |
| 4  | tobi       | tobi@testing.com | 36,50,98,36,49,50,36,50,73,85,79,121,84,111,120,57,1... | member |

Member 3 (Alice) Successfully deleted passing Test T-05.I

```
def delete_member(self, member_id): #delete member from database
    cursor = self.conn.cursor()
    cursor.execute(''DELETE FROM members WHERE id = ?'', (member_id,))
    #if no rows containing the member was found, the user does not exist in the database
    if cursor.rowcount == 0:
        return "Incorrect member ID entered"
    else:
        self.conn.commit()
        return "User {} has been deleted".format(member_id)
```

Member Manager delete\_member() method

```
def delete_member():
    # code for deleting memeber from member database using member ID
    member_manager=MemberManager("members2.db")

    try: member_id = int(input("Enter member ID: "))
    except ValueError:
        # if the user has not entered a valid integer
        print("The member ID should be a number.")
        return
    result = member_manager.delete_member(member_id)
    print(result)
```

In line delete\_member() function, note the try and except blue to avoid value error

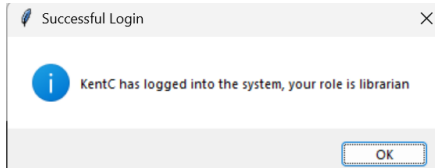
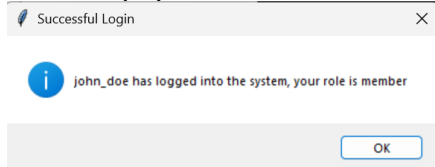
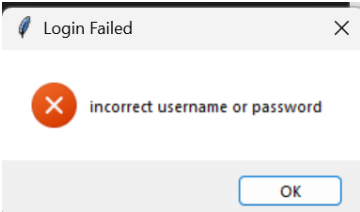
## Validation

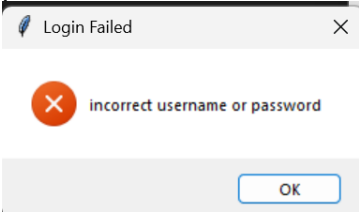
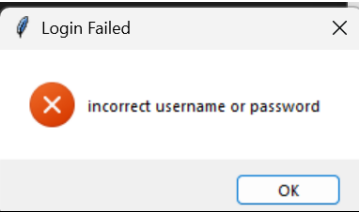
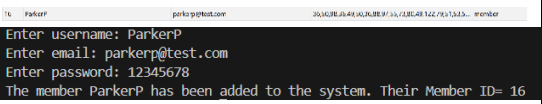
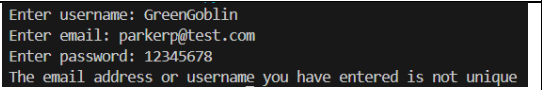
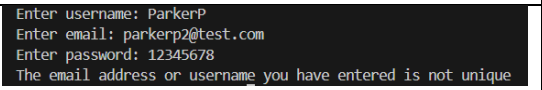
The first bit of validation within this solution is to check whether a user's email input in the add member function is a valid email address. This is validated using regular expression to ensure a valid email address is entered. The expression checks first whether there is text at the beginning of the string and then checks for the presence of the "@" symbol. It then checks for text after the "@" symbol, such as "Gmail" or "Hotmail" and then checks for the presence of a full stop and then a top-level domain such as ".com" or ".org." If this does not match it returns None but if it does it will return a match object. If the match ==None, it will print that the user has entered an invalid email and makes sure that the user enters a valid email before a user is added. This is evidenced using test T-04.3.

Furthermore, to ensure secure password policy, the user's entry for a password within the add user function is to check whether it is smaller than 8 characters. If the password that the user entered is smaller than 8 characters, it prompts the user by saying all passwords must be at least 8 characters long and will keep saying this until the user enters a valid password.

I also ensured that the user entered a member ID of a valid type using a try and except block to avoid a value error. This would return that the user must enter an integer number, to ensure that the user has entered a valid integer.

## Testing Table and Evidence

| Test ID | What is being tested                                        | Test Data                                     | Expected Outcome                                               | Actual Outcome (and evidence)                                                                                                                                                                  | Success/Fail |
|---------|-------------------------------------------------------------|-----------------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| T-01.1  | A valid username and password were entered for a librarian. | Username="KentC"<br>Password="12345678"       | "KentC" has signed into the system, their role is "librarian." | When correct user information is entered, the user is let into the system and their role is displayed.<br> | Success      |
| T-01.2  | A valid username and password were entered for a member     | Username="john_doe"<br>Password="password123" | "john_doe" has signed into the system, their role is "member"  | When correct user information is entered, the user is let into the system and their role is displayed.<br> | Success      |
| T-02.1  | An invalid username was entered.                            | Username="test1"<br>Password="12345678"       | "Incorrect username or password was entered."                  | When an incorrect username is entered it printed that an incorrect username or password was submitted.<br> | Success      |

|          |                                                                             |                                                                              |                                                                              |                                                                                                                                                                                               |         |
|----------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| T-02.2   | An invalid password was entered.                                            | Username= "KentC"<br>Password = "1234"                                       | "Incorrect username or password was entered."                                | When an incorrect password is entered, it printed that an incorrect username or password was submitted.<br> | Success |
| T-02.3   | Empty Strings were entered for the login page.                              | Username=""<br>Password=""                                                   | Incorrect username or password was entered. – User cannot enter the system.  | When no information is submitted, it printed that an incorrect username or password was submitted<br>       | Success |
| T-04.1   | A member can be added to the system, when provided with normal information. | Username = "ParkerP"<br>Email= "parkerp@test.com"<br>Password= "12345678"    | Member "ParkerP" added. successfully. Their user ID ="" (with valid user id) | <br>User was successfully added to the members table.                                                       | Success |
| T-04.2   | A duplicate email is provided to the add member.                            | Username= "GreenGoblin"<br>Email= "parkerp@test.com"<br>Password= "12345678" | "The email address or username you have entered is not unique."              |                                                                                                           | Success |
| T-04.2.1 | A duplicate username is provided to the add member                          | Username = "ParkerP"<br>Email= Parkerp2@test.com<br>Password= "12345678"     | "The email address or username you have entered is not unique."              |                                                                                                           | Success |

|        |                                                                  |                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                  |         |
|--------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| T-04.3 | An invalid email is entered to the system.                       | Username="ParkerP3"<br>Email="@abc.com"<br>Password="12345678"<br><br>Username="ParkerP3"<br>Email= <a href="#">test.com</a><br>Password="12345678"<br><br>Username="ParkerP3"<br>Email= <a href="#">test@.com</a><br>Password="12345678"                                                                                             | Prompted to enter a valid email before the user can progress.                                                                                                                                                                                                                  | <pre> Enter username: ParkerP3 Enter email: @abc.com Enter password: 12345678 You have entered an invalid email address. Enter email: test.com You have entered an invalid email address. Enter email: test@.com You have entered an invalid email address. Enter email: </pre> Matched expected.                                | Success |
| T-04.4 | The user's entered password is less than 8 characters.           | (less than 8 characters)<br>Username="test"<br>Email= <a href="#">test@xyz.com</a><br>Password="1234"<br><br>(exactly 8 characters)<br>Username="test"<br>Email= <a href="#">test@xyz.com</a><br>Password="12345678"<br><br>(Edge case: 7 characters)<br>Username="test"<br>Email= <a href="#">test@xyz.com</a><br>Password="1234567" | Return that your entered "password is less than 8 characters".<br><br>Password is greater than or equal to 8 characters so provided other inputs are okay, member should be added.<br><br>Password is smaller than 8 characters so print (Password is less than 8 characters). | <pre> Enter username: StarkT Enter email: test@xyz.com Enter password: 1234 Your password should be at least 8 characters long. Enter password: 1234567 Your password should be at least 8 characters long. Enter password: 12345678 The member StarkT has been added to the system. Their Member ID= 18 </pre> Expected result. | Success |
| T-05.1 | A valid user id is entered to delete a member from the database. | User ID="3"                                                                                                                                                                                                                                                                                                                           | User 3 has been deleted from the system. This should be checked by trying to                                                                                                                                                                                                   | <pre> Enter member ID: 3 User 3 has been deleted </pre> Validated by checking members table, seen above.<br><br>Expected result                                                                                                                                                                                                  | Success |

|        |                                                                 |                  |                                   |                                                                                                        |         |
|--------|-----------------------------------------------------------------|------------------|-----------------------------------|--------------------------------------------------------------------------------------------------------|---------|
|        |                                                                 |                  | run the program again.            |                                                                                                        |         |
| T-05.2 | An invalid user id is entered to be deleted.                    | User ID ="9999"  | Incorrect member ID entered       | <div>Enter member ID: 9999</div> <div>Incorrect member ID entered</div> <p>Expected Result</p>         | Success |
| T-05.3 | A user id entry of an invalid type (such as string) is entered. | User ID ="David" | The member ID should be a number. | <div>Enter member ID: David</div> <div>The member ID should be a number.</div> <p>Expected Result.</p> | Success |

## Key Stage Review

I have made significant process in getting a finished program, having completed the members functionality and adding a member's database support. However, next I will need to implement support for the different features relating to books and loans including the creation and integration for those features. Furthermore, I will also need to develop the graphical user interface for the system for my stakeholders to interact with, although it is important to have the logic behind the user interface working correctly first before its integration.

I have made progress towards Success Criteria S002, with the creation of a member/ users table as well as beginning to implement Role Based Access Control in the members database and login page (Success Criteria S006), although not complete. From completing this Development Cycle, I have completed Success Criteria S005, as the login page is complete, as well as S010 as the functionality to add or remove books is complete.

| Success Criteria ID | Success Criteria                                                                                                                                                                                                                                              | Relevant Feature ID | Further Justification                                                                                                                                                                                                                                                                                             |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S001                | There will be a graphical user interface where every button works.                                                                                                                                                                                            | F1                  | This is fundamental to making the program easy and intuitive to use. Using a different interface such as a command line interface may be hard to understand and grasp for members and librarians and a natural user interface is likely to be inconvenient to use and so slow down the productivity of librarians |
| S002                | There should be a database containing tables for books, users and loans                                                                                                                                                                                       | F2                  | Having a database with these features allows the data to be easily portable, in one or two files. This is also the easiest to edit and so it is easy to keep a well-maintained store of all required information                                                                                                  |
| S003                | There should be a function to search for books by name and by their ISBN and then the book and relevant information should be outputted to the user                                                                                                           | F3                  | This enables members and librarians to check what books are in stock as well as information such as the books name or genre, essential to the operation of a library management system.                                                                                                                           |
| S004                | There should be a function to sort books by genre and output them alphabetically                                                                                                                                                                              | F3                  | This allows members to search for books with their desired genre, but also to help the organisation of the library as the books are organised by genre then in alphabetical order                                                                                                                                 |
| S005                | There should be a log in page                                                                                                                                                                                                                                 | F4                  | This aims to protect sensitive information or functions from unauthorised users and allows to dictate between member and librarian users – note Feature F5.                                                                                                                                                       |
| S006                | There should be Role Based Access control (between normal members and privileged librarian accounts) which have access to different features                                                                                                                  | F5                  | Primarily successful through success criteria S005. This is essential as there are features that librarians should be able to access to enable them to do their work that would be useless or even cause security risks for normal member users to have access to such as adding or deleting books or members.    |
| S007                | There should be a feature to add or remove books                                                                                                                                                                                                              | F6                  | Fundamental to the job of a librarian, adding and removing books as well as its metadata such as the name of the author and its genre. This is necessary to improve the database and enable users to see which books are available.                                                                               |
| S008                | There should be a feature to borrow and return books. It should check whether the book is in the database and prevent already checked out books from being checked out again. It should also log which member logged out the book according to its member ID. | F7                  | Also fundamental to the operation of a library. Enables users to check books in and out themselves, relieving this duty from librarians. Also, it will add to a loans table, allowing librarians to check loans.<br>Note- also F8                                                                                 |

|      |                                                         |    |                                                                                                                                      |
|------|---------------------------------------------------------|----|--------------------------------------------------------------------------------------------------------------------------------------|
| S009 | There should be a feature to add and remove new members | F9 | This allows for new members to be added to the system, particularly at the start of the school year such as new sixth form students. |
|------|---------------------------------------------------------|----|--------------------------------------------------------------------------------------------------------------------------------------|

### Key

Green = Complete

Orange = Progress has been made

Red = Not done

## Development Cycle Part 3

### Code Development

In this cycle, I developed the Book manager functionality. This began by creating the Book manager class which takes the database manager as a parent, just like Member manager. I then created the new create tables function which overrides the creates tables function in the database manager. In this, when called it will create the books table if it does not exist. Just like the members table it will have a primary id key which is unique and autoincrementing. However, it also has fields for the title of a book, a field for its author, a field for the genre of the book, and a text field for the ISBN of the book, which should be unique. There is also a Boolean (True/False / 1/0) field dictating whether the book is checked in or out, meaning whether it is available for loans or not. I then ran the code to make sure it worked, and a books database file was created.

| id | title | author | genre | isbn | checked_in |
|----|-------|--------|-------|------|------------|
|----|-------|--------|-------|------|------------|

Books table

```
class BookManager(DatabaseManager):
    #create the library books tables if needed, particularly when run for the first time, prevents error when modifying a database which does not exist.
    def create_tables(self):
        cursor = self.conn.cursor()
        cursor.execute('''CREATE TABLE IF NOT EXISTS books
        (id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT,
        author TEXT,
        genre TEXT,
        isbn TEXT UNIQUE,
        checked_in INTEGER DEFAULT 1)''')
        self.conn.commit()
```

Create table function, creating books table

Following from this, I implemented the add book and delete book functionality. These functions are almost identical to the related add member and delete member functions and are only adjusted to take in values for a books, title, author, genre and ISBN in the add book function and the book id in the delete book function. This also meant I had to create a new Book class which has its own title, author, genre and ISBN attributes. I also had to adjust the try and except block for the add book function as it would crash if a duplicate ISBN were entered into system as it had to be unique.

|    |                  |                 |           |               |   |
|----|------------------|-----------------|-----------|---------------|---|
| 33 | The Hunger Games | Suzanne Collins | Dystopian | 9780439023528 | 1 |
|----|------------------|-----------------|-----------|---------------|---|

New Book added (Test T-06.I)

|   |                |                 |                    |               |   |
|---|----------------|-----------------|--------------------|---------------|---|
| 7 | Fahrenheit 451 | Ray Bradbury    | Dystopian          | 9781451673319 | 1 |
| 8 | Moby-Dick      | Herman Melville | Adventure          | 9781503280786 | 1 |
| 9 | War and Peace  | Leo Tolstoy     | Historical Fiction | 9781400079988 | 1 |

An extract of the books table before Book 8 is deleted

|   |                |              |                    |               |   |
|---|----------------|--------------|--------------------|---------------|---|
| 7 | Fahrenheit 451 | Ray Bradbury | Dystopian          | 9781451673319 | 1 |
| 9 | War and Peace  | Leo Tolstoy  | Historical Fiction | 9781400079988 | 1 |

After Book 8 is deleted, passing Test T-07.I

```
def add_book(self, newbook): #add book to database
    cursor = self.conn.cursor()
    #attempt to add the book to the books table error from sqlite3 integrity error as the ISBN (the book) is already in the database
    try:
        cursor.execute('INSERT INTO books (title, author, genre, isbn) VALUES (?, ?, ?, ?)',
            (newbook.title, newbook.author, newbook.genre, newbook.isbn))
        self.conn.commit()
        book_id = cursor.lastrowid
        return "The user {} has been added to the system. Their Member ID= {}".format(newbook.title ,book_id)
    except sqlite3.IntegrityError:#ISBN must be unique
        return "You have entered an ISBN which is already in the database."
```

#### Book Manager add book method

```
def delete_book(self, delete_book_id): #delete book from database using a search of the book id
    cursor = self.conn.cursor()
    cursor.execute('DELETE FROM books WHERE id = ?', (delete_book_id,))
    #if no rows containing the book ID was found, the book does not exist in the database
    if cursor.rowcount == 0:
        return "Incorrect Book ID"
    else:
        self.conn.commit()
        return "Book {} has been deleted".format(delete_book_id)
```

#### Book Manager delete book method

```
def add_book():
    book_manager=BookManager("books2.db")
    #takes in user entry
    title= input("Enter the title of the book: ")
    author=input("Enter the author of the book: ")
    genre = input("Enter the genre of the book: ")
    isbn = input("Enter the ISBN of the book: ")
    #creates new book object
    book = Book(title, author, genre, isbn)
    #attempts to insert it into the database
    result = book_manager.add_book(book)
    print (result)
```

#### Inline add book function

```
def delete_book():
    book_manager=BookManager("books2.db")
    try: book_id = int(input("Enter book ID: "))
    except ValueError:
        # if the user has not entered a valid integer
        print("The book ID should be a number.")
        return
    result = book_manager.delete_book(book_id)
    print(result)
```

#### Inline delete book function

```
#defines book class with public attributes for title, author, genre and ISBN value (all entered as parameters)
class Book:
    def __init__(self, title, author, genre, isbn):
        self.title = title
        self.author = author
        self.genre = genre
        self.isbn = isbn
```

#### Book Class

Following this, I began to create the different sorting functions to complete Success Criteria S004 and S005. This began with a search book by title method of the book manager. This function works by taking in a user input for a title of a book and calling an SQL query to return the everything, all records and their fields which match that title. I used the LIKE query and encased the user entry using %title% to make the query search allow for wildcards. This query rather than being strict and searching exactly the user entry as a title, takes in a book and enters it like data as a parameter and allows for specific key words to be returned. For example, if 'harry' was entered, all books containing 'Harry' such as the various Harry Potter books will be returned rather than just searching for a book with the title 'harry'. This will then return a 2d list of filtered books which match the description containing the books which match the keyword. Within the inline function, it will then go through the 2d list and will print the different attributes of each book, for example book[i][0] contains the ID, book[i][1] contains the title of the book, book[i][2], contains the author of the book, book[i][3] contains the genre of the book, book[i][4] contains the ISBN and book[i][5] contains the status. As the status is a Boolean (1 or 0) value, the status is then changed to being checked in or out for clarify this to the user easier. If no books were found the function returns that the book is not found.

```
def search_book_by_title(self, title):# search for books according to the title
    cursor = self.conn.cursor()
    #LIKE so doesnt have to be a direct match (allows for wildcards)
    cursor.execute('''SELECT * FROM books WHERE title LIKE ?''', ('%' + title + '%',))
    filteredbooks = cursor.fetchall()
    return filteredbooks
```

Book Manager search book by title method

```
def search_book_by_title():
    # code to search for a book according to its title
    book_manager=BookManager("books2.db")
    booktitle = input("Enter Book Title: ")
    #2d list containing filtered books
    filteredbooks = book_manager.search_book_by_title(booktitle)#find the book() using a sql like query, function from book manager
    if filteredbooks == []:#if book not found and so list is empty
        print("No books with this keyword")
    else:
        for i in range (len(filteredbooks)):
            #checks the status of the current book
            if filteredbooks[i][5]== 1:
                status="Checked in"
            else:
                status = "Checked Out"
            print( "ID: {}, Title: {}, Author: {}, Genre: {}, ISBN: {}, Status: {}".format(
                filteredbooks[i][0],filteredbooks[i][1],filteredbooks[i][2],filteredbooks[i][3],filteredbooks[i][4],status))
```

Inline search book by title function

```
Enter Book Title: the hobbit
ID: 6, Title: The Hobbit, Author: J.R.R. Tolkien, Genre: Fantasy, ISBN: 9780547928227, Status: Checked In
```

Search by Title function, with the title “the hobbit”, and its book information is being printed passing

Passing Test T-08.1

```
Enter Book Title: the
ID: 3, Title: The Great Gatsby, Author: F. Scott Fitzgerald, Genre: Fiction, ISBN: 9780743273565, Status: Checked In
ID: 5, Title: The Catcher in the Rye, Author: J.D. Salinger, Genre: Fiction, ISBN: 9780316769488, Status: Checked In
ID: 6, Title: The Hobbit, Author: J.R.R. Tolkien, Genre: Fantasy, ISBN: 9780547928227, Status: Checked In
ID: 10, Title: The Odyssey, Author: Homer, Genre: Epic Poetry, ISBN: 9780140268867, Status: Checked In
ID: 12, Title: The Brothers Karamazov, Author: Fyodor Dostoevsky, Genre: Philosophical Fiction, ISBN: 9780374528379, Status: Checked In
ID: 13, Title: The Divine Comedy, Author: Dante Alighieri, Genre: Epic Poetry, ISBN: 9780142437223, Status: Checked In
ID: 14, Title: The Iliad, Author: Homer, Genre: Epic Poetry, ISBN: 9780140275360, Status: Checked In
ID: 16, Title: The Road, Author: Cormac McCarthy, Genre: Post-Apocalyptic, ISBN: 9780307387899, Status: Checked In
ID: 17, Title: The Alchemist, Author: Paulo Coelho, Genre: Fantasy, ISBN: 9780062315007, Status: Checked In
ID: 18, Title: The Book Thief, Author: Markus Zusak, Genre: Historical Fiction, ISBN: 9780375842207, Status: Checked In
ID: 19, Title: The Kite Runner, Author: Khaled Hosseini, Genre: Historical Fiction, ISBN: 9781594631931, Status: Checked In
ID: 21, Title: The Handmaid's Tale, Author: Margaret Atwood, Genre: Dystopian, ISBN: 9780385490818, Status: Checked In
ID: 22, Title: The Lord of the Rings, Author: J.R.R. Tolkien, Genre: Fantasy, ISBN: 9780544003415, Status: Checked In
ID: 23, Title: The Picture of Dorian Gray, Author: Oscar Wilde, Genre: Gothic Fiction, ISBN: 9780141439570, Status: Checked In
ID: 25, Title: The Adventures of Huckleberry Finn, Author: Mark Twain, Genre: Adventure, ISBN: 9780486280615, Status: Checked In
```

Extract printed from the search by title method, using the keyword “the” (note not all books fit in the screenshot)

Passing Test T-08.2

I then created a search by ISBN function to enable the librarians to find specific books and link them together. This will be done using a binary search. To incorporate this. I had to also create a fetch all books function. This contains an SQL query to fetch all books from the books table and creates a list of books objects. I then attempted to do a binary search using this list of books. This works by taking in the list of books and assigning an initial low pointer to the first index in the list and a high index at the end of the list it then calculates the middle of the list using  $(\text{High} + \text{low} // 2)$  such as if  $\text{low} = 0$  and  $\text{high} = 8 + 0 // 2 = 4$  and so the mid-point is 4. And then, while low is smaller than or equal to high, meaning while there is more than one book in the sublist, it compares the target ISBN to the ISBN stored at the midpoint. If the target ISBN is greater than the ISBN at the midpoint then you cut off the smaller half of the list, smaller than the midpoint and then recalculate the midpoint. If then target ISBN is smaller than the ISBN at the midpoint, then you cut off the larger half of the list, where the ISBN is larger than the midpoint, by setting high to mid and then recalculating mid. This is then done until there are no more items in the list or if the target ISBN is equal to the midpoint, where the book object is then returned. However, after implementing this binary search I forgot to add a sort to sort the books ascending order. This mean that sometimes even though the ISBN was in the table it would not be returned. This is because if they are not in order, then the assumption that the books cut off are not the target book cannot be confirmed to be true. This as a result lead to me having to add a quick sort to sort all the books, when the list of books is created from the fetch all books function to make sure the binary search works correctly.

The quick sort function works using recursion. It works by setting the first book in the list as a pivot and then looping through the list and moving all books with an ISBN lower than the pivot to the left side of the pivot, by incrementing the left wall and swapping the current book with the one at the left wall which moves smaller elements to the left, and all books with higher ISBN's to the right side of the pivot, swapping the pivot with left wall. It then returns the index of the pivot so the recursion can continue. Once the quick sort function is run, it will return the list of sorted books into the binary search function allowing for it to work correctly.

```
def fetch_all_books(self):
    cursor = self.conn.cursor()
    cursor.execute(''SELECT * FROM books'')# return everything (*) from the books database
    rows = cursor.fetchall()
    #initialises empty list of books and then appends new book objects to the end of that list
    all_books=[]
    for i in range(len(rows)):
        all_books.append(Book(rows[i][1], rows[i][2], rows[i][3], rows[i][4]))

    return all_books
```

Book Manager method to fetch all books()

```
def binary_search(self, isbn):
    allbooks = self.fetch_all_books() # grabs all books, making sure they're all objects
    low=0
    high = len(allbooks) - 1
    allbooks = self.quick_sort(allbooks,low,high) # sorts all the books for isbn (binary search must be in order)
    while low <= high:
        mid = (low + high) // 2
        if allbooks[mid].isbn == isbn: #if the mid index isbn = target, it is found
            return allbooks[mid]
        elif allbooks[mid].isbn < isbn:#cut off lower half of the array
            low = mid + 1
        else:
            high = mid - 1 #cut of larger half of the array
    return None
```

Book Manager method for a binary search

```

def quick_sort(self, books, low, high):
    if (low < high):
        #like a pre order traversal
        pivot=self.partition(books,low, high) # get the index of the pivot
        self.quick_sort(books,low, pivot-1) # sort the left subset
        self.quick_sort(books, pivot+1, high)# sort the right subset
    return books

def partition(self,books,low, high):
    pivot=books[low] #first book is the pivot
    leftwall=low#boundary between elements smaller and greater than the pivot
    for i in range(low+1,high+1):
        if books[i].isbn < pivot.isbn:# if curent element is less than the pivot move it to the left of the leftwall
            leftwall +=1
            #swap the current element and the element at index leftwall
            temp=books[i]
            books[i]=books[leftwall]
            books[leftwall]=temp
    #swap the pivot element with the element at the left wall
    temp=books[low]
    books[low] = books[leftwall]
    books[leftwall]=temp
    #return index of the pivot
    return leftwall

```

Book Manager method to quick sort, so the binary search works as it must be ordered.

```

def search_book_by_isbn():
    book_manager=BookManager("books2.db")
    searched_isbn = input("Enter ISBN: ")
    book = book_manager.binary_search(searched_isbn)

    if book == None:# if book is not found
        print( "No book found with that ISBN.")
    else:
        print( "Title: {}, Author: {}, Genre: {}, ISBN: {}\n".format(book.title,book.author,book.genre,book.isbn))

```

Inline search book by ISBN function

I then started to produce the search members alphabetically by genre. This is useful as it allows the librarian to accurately place the books in their correct order when organising the library as well as helping users find new books to read in their favourite genre. This works by taking in a specific user's genre and then using the fetch all books which I wrote before to get a 2d list of all the books. It then passes this list into the search book by genre method of the book database as a parameter along with the target genre and checks every book linearly to see whether their genre matches the target genre, adding it to a new list of filtered books. Once this is returned to the original function, a merge sort is then done to order the books in alphabetical order for the librarian to then use as each book and its information is printed to the user.

```

def sort_books_alphabetically_by_genre():
    book_manager=BookManager("books2.db")
    genre = input("Enter Genre")
    books = book_manager.fetch_all_books()# fetch the list of all books in the database
    if genre != "":
        books=book_manager.sort_books_by_genre(genre)#filter for all books of that genre

        sortedbooks = book_manager.merge_sort(books,0, len(books)-1)#merge sort to get it into alphabetical order so it can be placed

        # if there are no books within that genre
        if sortedbooks == []:
            print( "No books found of the genre {}".format(genre))
        else:
            # print all of the books and its data in order
            for i in range (len(sortedbooks)):
                print( "Title: {}, Author: {}, Genre: {}, ISBN: {}\n".format
                    (sortedbooks[i].title, sortedbooks[i].author,sortedbooks[i].genre,sortedbooks[i].isbn))

```

In line sort books alphabetically by genre function

```
def sort_books_by_genre(self, genre):
    books= self.fetch_all_books()#grabs all the books
    sortedbooks=[]
    for i in range (len(books)):#linear search, if the book's genre matches the target genre add it to the sortedbooks list
        if books[i].genre == genre:
            sortedbooks.append(books[i])
    return sortedbooks
```

#### Book Manager filter books by genre method

The merge sort function works by taking the list of filtered books, the starting index (low) and the ending index (high) as parameters. Its base case is where low is greater than or equal to high, meaning it is already sorted. It then finds the middle index, and recursively divides the left side, until it is a single book, meaning it is sorted, and then combines its back up, merging the two sorted halves and then recursively doing the same with the right half. The merge function helps to merge two already sorted halves by taking the list of books, the middle pointer and the high pointer, creating two sublists (a left sublist from the low index to mid and a right sublist from mid+1 to high). It then compares the elements of left and right in order of their title and then copies the smaller one into the original list, after one list is sorted, it adds the remaining books to the other list.

```
def merge(self,sortedbooks, low, mid, high):
    left = sortedbooks[low:mid + 1] #create temporary list for left half
    right=sortedbooks[mid+1:high+1] #create temporary list for right half
    loc1 = 0 #pointer for left array
    loc2 = 0 # pointer for right array
    locM = low # pointer for merged array

    while loc1 < len(left) and loc2 < len(right):
        #if the current array in the left half is smaller than the one in the right
        if left[loc1].title.strip().lower() <= right[loc2].title.strip().lower():
            sortedbooks[locM] = left[loc1] #put in the merged array
            loc1 += 1 # increment the pointer in the left half
        else:
            sortedbooks[locM] = right[loc2]#put in the merged array
            loc2 += 1 #increment the pointer in the right half
            locM += 1 #increment pointer in merged array
    #if there are any remaining add to merged array
    while loc1 < len(left):
        sortedbooks[locM] = left[loc1]
        locM += 1
        loc1 += 1
    while loc2 < len(right):
        sortedbooks[locM] = right[loc2]
        loc2 += 1
        locM += 1

def merge_sort(self,sortedbooks, low, high):
    if high > low:
        mid = (low + high) // 2
        #post order
        self.merge_sort(sortedbooks, low, mid,)#sort left half recursively
        self.merge_sort(sortedbooks, mid + 1, high,) #sort right half recursively
        self.merge(sortedbooks, low, mid, high)#combine both halves
    return sortedbooks
```

#### Book Manager merge and merge sort methods

When comparing the two books within the merge sort I used the `.strip()` and `.lower()` functions to remove whitespace and make all the characters in the book lowercase to ensure that the merge sort compares correctly. For example, when a book called “Zoo” and a book called “apple” are compared, as the ‘z’ is capitalised, it may be shown to be lower than the ‘a’ in apple however, the `.lower()` function makes the ‘z’ lowercase and so it is accurately placed after the book “apple.”

```
Enter GenreFiction
Title: The Catcher in the Rye, Author: J.D. Salinger, Genre: Fiction, ISBN: 9780316769488

Title: The Great Gatsby, Author: F. Scott Fitzgerald, Genre: Fiction, ISBN: 9780743273565

Title: The Old Man and the Sea, Author: Ernest Hemingway, Genre: Fiction, ISBN: 9780684801223

Title: The Sun Also Rises, Author: Ernest Hemingway, Genre: Fiction, ISBN: 9780743297332

Title: To Kill a Mockingbird, Author: Harper Lee, Genre: Fiction, ISBN: 9780446310789
```

Test for books of the genre: Fiction. Passes Test T-10.1

However, I came into an issue. If a librarian wanted all books regardless of genre to be sorted alphabetically, this would not work. If they do not enter a string, then no books would match, meaning an empty list will be returned by the sorted books and so no books will be returned. To fix this, after a genre entry is taken in, it will only sort them alphabetically if the genre is not equal to “”, meaning they have entered a genre, to help pass test T-10.2.

```
Title: 1984, Author: George Orwell, Genre: Dystopian, ISBN: 9780451524935

Title: Anna Karenina, Author: Leo Tolstoy, Genre: Historical Fiction, ISBN: 9780143035008

Title: Crime and Punishment, Author: Fyodor Dostoevsky, Genre: Psychological Fiction, ISBN: 9780143058144

Title: Fahrenheit 451, Author: Ray Bradbury, Genre: Dystopian, ISBN: 9781451673319

Title: Frankenstein, Author: Mary Shelley, Genre: Gothic Fiction, ISBN: 9780486282114

Title: Life of Pi, Author: Yann Martel, Genre: Adventure, ISBN: 9780156027328

Title: Pride and Prejudice, Author: Jane Austen, Genre: Romance, ISBN: 9780486284736

Title: The Adventures of Huckleberry Finn, Author: Mark Twain, Genre: Adventure, ISBN: 9780486280615

Title: The Alchemist, Author: Paulo Coelho, Genre: Fantasy, ISBN: 9780062315007

Title: The Book Thief, Author: Markus Zusak, Genre: Historical Fiction, ISBN: 9780375842207

Title: The Brothers Karamazov, Author: Fyodor Dostoevsky, Genre: Philosophical Fiction, ISBN: 9780374528379

Title: The Catcher in the Rye, Author: J.D. Salinger, Genre: Fiction, ISBN: 9780316769488

Title: The Chronicles of Narnia, Author: C.S. Lewis, Genre: Fantasy, ISBN: 9780064471190

Title: The Divine Comedy, Author: Dante Alighieri, Genre: Epic Poetry, ISBN: 9780142437223

Title: The Great Gatsby, Author: F. Scott Fitzgerald, Genre: Fiction, ISBN: 9780743273565

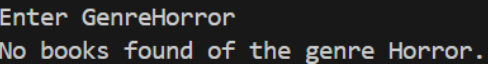
Title: The Handmaid's Tale, Author: Margaret Atwood, Genre: Dystopian, ISBN: 9780385490818
```

All books returned after adjustments made, passing Test T-10.2

## Testing Table and Evidence

| Test ID | What is being tested                                                      | Test Data                                                                                                                                                                                                                                  | Expected Outcome                                                                                           | Actual Outcome (and evidence)                                                                                                                                                                                                                                                                                     | Success/Fail |
|---------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| T-06.1  | A book can be added to the system, when provided with normal information. | Title= "The Hunger Games"<br>Author = "Suzanne Collins"<br>Genre = "Dystopian"<br>ISBN= "9780439023528"                                                                                                                                    | The book should be added to the books table                                                                | <div> Enter the title of the book: The Hunger Games<br/> Enter the author of the book: Suzanne Collins<br/> Enter the genre of the book: Dystopian<br/> Enter the ISBN of the book: 9780439023528<br/> The user The Hunger Games has been added to the system. Their Member ID= 33 </div> <p>Expected result.</p> | Success      |
| T-06.2  | A duplicate ISBN is provided to the add book function.                    | Title= "The Hunger Games 2"<br>Author = "Suzanna Collins"<br>Genre = "Science Fiction"<br>ISBN= "9780439023528"<br><br>#note1 Title, Author and Genre changed to note that integrity error due to duplicate ISBN<br>#note2 do after T-06.1 | You should get an error message stating that the ISBN is not unique.                                       | <div> Enter the title of the book: The Hunger Games 2<br/> Enter the author of the book: Suzanna Collins<br/> Enter the genre of the book: Science Fiction<br/> Enter the ISBN of the book: 9780439023528<br/> You have entered an ISBN which is already in the database </div> <p>Expected result.</p>           | Success      |
| T-07.1  | A valid Book id is entered to delete a book from the database.            | Book ID="8"                                                                                                                                                                                                                                | Book number 8 has been deleted from the system. This should be checked by trying to run the program again. | <div> Enter number4<br/> Enter book ID: 8<br/> Book 8 has been deleted </div> <p>Expected result, effect on database shown above</p>                                                                                                                                                                              | Success      |
| T-07.2  | An invalid book id is entered to be deleted.                              | Book ID ="9999"                                                                                                                                                                                                                            | Incorrect Book ID                                                                                          | <div> Enter book ID: 9999<br/> Incorrect Book ID </div> <p>Expected Result</p>                                                                                                                                                                                                                                    | Success      |
| T-07.3  | A book id entry of an invalid type (such as string) is entered.           | Book ID = "The Hunger Games"                                                                                                                                                                                                               | The book ID should be a number.                                                                            | <div> Enter book ID: The Hunger Games<br/> The book ID should be a number. </div> <p>Expected Result</p>                                                                                                                                                                                                          | Success      |

|        |                                                                                          |                      |                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |         |
|--------|------------------------------------------------------------------------------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| T-08.1 | The user enters a valid book title, and the right book is returned.                      | Title "The Hobbit"   | Information about the book, its ID number, title, author, genre, ISBN, whether it is checked in or not.                                                                                                                          | <div>Enter Book Title: the hobbit</div> <div>ID: 6, Title: The Hobbit, Author: J.R.R. Tolkien, Genre: Fantasy, ISBN: 9780547928227, Status: Checked In</div> <div>Expected Result.</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Success |
| T-08.2 | The user enters part of a book title and the books that match that keyword are returned. | Title="the"          | Information (see above) about books containing "the" in their name such as:<br>"The Life of Pi"<br>"The Lord of the Rings"<br>"The Picture of Dorian Gray"<br>"The Adventures of Huckleberry Finn"<br>"The Chronicles of Narnia" | <div>Enter Book Title: the</div> <div>ID: 3, Title: The Great Gatsby, Author: F. Scott Fitzgerald, Genre: Fiction, ISBN: 978043273565, Status: Checked In</div> <div>ID: 5, Title: The Catcher in the Rye, Author: J.D. Salinger, Genre: Fiction, ISBN: 978031670489, Status: Checked In</div> <div>ID: 6, Title: The Hobbit, Author: J.R.R. Tolkien, Genre: Fantasy, ISBN: 9780547928227, Status: Checked In</div> <div>ID: 10, Title: The Odyssey, Author: Homer, Genre: Epic Poetry, ISBN: 9780140268867, Status: Checked In</div> <div>ID: 12, Title: The Brothers Karamazov, Author: Fyodor Dostoevsky, Genre: Philosophical Fiction, ISBN: 9780374526379, Status: Checked In</div> <div>ID: 13, Title: The Divine Comedy, Author: Dante Alighieri, Genre: Epic Poetry, ISBN: 9780140453723, Status: Checked In</div> <div>ID: 14, Title: The Iliad, Author: Homer, Genre: Epic Poetry, ISBN: 9780140275300, Status: Checked In</div> <div>ID: 16, Title: The Road, Author: Cormac McCarthy, Genre: Post-Apocalyptic, ISBN: 9780307387899, Status: Checked In</div> <div>ID: 17, Title: The Alchemist, Author: Paulo Coelho, Genre: Fantasy, ISBN: 9780062315007, Status: Checked In</div> <div>ID: 18, Title: The Book Thief, Author: Markus Zusak, Genre: Historical Fiction, ISBN: 9780375842207, Status: Checked In</div> <div>ID: 19, Title: The Kite Runner, Author: Khaled Hosseini, Genre: Historical Fiction, ISBN: 9781596331931, Status: Checked In</div> <div>ID: 21, Title: The Handmaid's Tale, Author: Margaret Atwood, Genre: Dystopian, ISBN: 9780385490818, Status: Checked In</div> <div>ID: 22, Title: The Lord of the Rings, Author: J.R.R. Tolkien, Genre: Fantasy, ISBN: 9780544003415, Status: Checked In</div> <div>ID: 23, Title: The Picture of Dorian Gray, Author: Oscar Wilde, Genre: Gothic Fiction, ISBN: 9780141439570, Status: Checked In</div> <div>ID: 25, Title: The Adventures of Huckleberry Finn, Author: Mark Twain, Genre: Adventure, ISBN: 9780486280615, Status: Checked In</div> <div>Expected result.</div> <div>#Note larger image above (page 74)</div> | Success |
| T-08.3 | The user enters a book not in the title.                                                 | Title = "Superman"   | This book has not been found.                                                                                                                                                                                                    | <div>Enter Book Title: Superman</div> <div>No books with this keyword</div> <div>Expected result</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Success |
| T-13.1 | A valid ISBN is provided when searching by ISBN                                          | ISBN="9780446310789" | Information on that book (title, author, genre ISBN) is returned.                                                                                                                                                                | <div>Enter ISBN: 9780446310789</div> <div>Title: To Kill a Mockingbird, Author: Harper Lee, Genre: Fiction, ISBN: 9780446310789</div> <div>Expected result</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Success |
| T-13.2 | An ISBN which is not in the book table is entered when searching by ISBN                 | ISBN="9780525559474" | This ISBN cannot be found in the table.                                                                                                                                                                                          | <div>Enter ISBN: 9780525559474</div> <div>No book found with that ISBN.</div> <div>Expected result.</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Success |
| T-10.1 | The user enters a valid genre, and the right books are filtered.                         | Genre="Fiction"      | Information on the following books:<br>"The Catcher in the Rye"<br>"The Great Gatsby"<br>"To Kill a Mockingbird"<br>"The Old Man and the Sea"<br>"The Sun also Rises"                                                            | <div>Enter Genre: Fiction</div> <div>Title: The Catcher in the Rye, Author: J.D. Salinger, Genre: Fiction, ISBN: 9780316704888</div> <div>Title: The Great Gatsby, Author: F. Scott Fitzgerald, Genre: Fiction, ISBN: 978043272565</div> <div>Title: The Old Man and the Sea, Author: Ernest Hemingway, Genre: Fiction, ISBN: 9780684801233</div> <div>Title: The Sun Also Rises, Author: Ernest Hemingway, Genre: Fiction, ISBN: 9780743297332</div> <div>Title: To Kill a Mockingbird, Author: Harper Lee, Genre: Fiction, ISBN: 9780446310789</div> <div>Also shown above (Page 77)</div> <div>Expected result.</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Success |

|        |                                                           |                  |                                                      |                                                                                                                                  |         |
|--------|-----------------------------------------------------------|------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|---------|
| T-10.2 | The user does not enter a genre.                          | Genre=""         | All books should be outputted in alphabetical order. |  <p>Expected result.<br/>Also shown page 77</p> | Success |
| T-10.3 | The user enters a genre that is not found in the database | Genre = "Horror" | No books found of this genre.                        |  <p>Expected result.</p>                       | Success |

## Key Stage Review

This development cycle was quite successful which is good as it was an essential part of the program. I successfully incorporate the books table, along with the book class. I then created methods to add or remove books from the table, as well as the various ways to sort and filter the table for a specific book or range of books. However, I still need to incorporate the loan books and return books feature, which will have to include a new loans table which I will need to develop, likely as another table of the books database for convenience. In the future, I will also have to develop a graphical user interface for all the different features as well as include accessibility features, however I will do this when the logic of the program is done, to make its integration easier.

As a result of this cycle, I have made further progress in achieving success criteria S002, however I have completed Success Criteria S003, S004 and S007

| Success Criteria ID | Success Criteria                                                                                                                                    | Relevant Feature ID | Further Justification                                                                                                                                                                                                                                                                                             |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S001                | There will be a graphical user interface where every button works.                                                                                  | F1                  | This is fundamental to making the program easy and intuitive to use. Using a different interface such as a command line interface may be hard to understand and grasp for members and librarians and a natural user interface is likely to be inconvenient to use and so slow down the productivity of librarians |
| S002                | There should be a database containing tables for books, users and loans                                                                             | F2                  | Having a database with these features allows the data to be easily portable, in one or two files. This is also the easiest to edit and so it is easy to keep a well-maintained store of all required information                                                                                                  |
| S003                | There should be a function to search for books by name and by their ISBN and then the book and relevant information should be outputted to the user | F3                  | This enables members and librarians to check what books are in stock as well as information such as the books name or genre, essential to the operation of a library management system.                                                                                                                           |
| S004                | There should be a function to sort books by genre and output them alphabetically                                                                    | F3                  | This allows members to search for books with their desired genre, but also to help the organisation of the library as the books are organised by genre then in alphabetical order                                                                                                                                 |

|      |                                                                                                                                                                                                                                                               |    |                                                                                                                                                                                                                                                                                                                |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S005 | There should be a log in page                                                                                                                                                                                                                                 | F4 | This aims to protect sensitive information or functions from unauthorised users and allows to dictate between member and librarian users – note Feature F5.                                                                                                                                                    |
| S006 | There should be Role Based Access control (between normal members and privileged librarian accounts) which have access to different features                                                                                                                  | F5 | Primarily successful through success criteria S005. This is essential as there are features that librarians should be able to access to enable them to do their work that would be useless or even cause security risks for normal member users to have access to such as adding or deleting books or members. |
| S007 | There should be a feature to add or remove books                                                                                                                                                                                                              | F6 | Fundamental to the job of a librarian, adding and removing books as well as its metadata such as the name of the author and its genre. This is necessary to improve the database and enable users to see which books are available.                                                                            |
| S008 | There should be a feature to borrow and return books. It should check whether the book is in the database and prevent already checked out books from being checked out again. It should also log which member logged out the book according to its member ID. | F7 | Also fundamental to the operation of a library. Enables users to check books in and out themselves, relieving this duty from librarians. Also, it will add to a loans table, allowing librarians to check loans. Note- also F8                                                                                 |
| S009 | There should be a feature to add and remove new members                                                                                                                                                                                                       | F9 | This allows for new members to be added to the system, particularly at the start of the school year such as new sixth form students.                                                                                                                                                                           |

## Key

Green = Complete

Orange = Progress has been made

Red = Not done

## Development Cycle Part 4

### Code Development

In this section I aim to complete the loan and return loan functionality of the library management system. This began with creating a loans table. This was done by adjusting the Book Manager create tables method to also create a loans table. Just like the member and books table, the loans table also has an automatically incrementing loan ID as its primary key. However, it also has fields for book ID's and member ID's which are both foreign keys from the book table ID and member table ID respectively. This is used to link a loan to its book as well as attribute a loan to a specific member. The loans table also contains fields for the loan date and the due date of the loan as well as a Boolean value field to state whether the loan has been completed, whether the book has been returned or not. This is automatically 0, and changes to 1 when the book is returned. After this, I ran the code, which I then checked to see whether it generated the loans table correctly.

```
class BookManager(DatabaseManager):
    #create the library books and loans tables if needed, particularly when run for the first time, prevents error when modifying a database which does not exist.
    def create_tables(self):
        cursor = self.conn.cursor()
        cursor.execute('''CREATE TABLE IF NOT EXISTS books
        (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT,
            author TEXT,
            genre TEXT,
            isbn TEXT UNIQUE,
            checked_in INTEGER DEFAULT 1)''')

        cursor.execute('''CREATE TABLE IF NOT EXISTS loans
        (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            book_id INTEGER,
            member_id INTEGER,
            loan_date TEXT,
            due_date TEXT,
            returned INTEGER DEFAULT 0,
            FOREIGN KEY(book_id) REFERENCES books(id),
            FOREIGN KEY(member_id) REFERENCES members(id))''')
```

Updated Book Manager create tables method

| id | book_id | member_id | loan_date | due_date | returned |
|----|---------|-----------|-----------|----------|----------|
|----|---------|-----------|-----------|----------|----------|

Loans table

Following this I began to develop the loan books function. Firstly, it takes in a member ID and book ID, and then will check the books table, created in development cycle 3, to check whether the submitted book ID is “checked in,” meaning to say it is either True if the book is available for loan, or False if it is out on loan. I then used the datetime module to take in today’s date, assigning that to the loan date and then calculating the due date, 14 days from now using the time delta class from the datetime module. It then attempts to insert this into the loans table, logging the book id member id, loan date and due date and setting the returned to 0. It will then also change the check in field in the books table to 0, so another loan of the same book cannot be taken out again before it is returned. In the Book Manager method, if no book is found, this means the book is not in the books table and so the user has enabled an invalid book ID which is returned to the user.

Within the inline function, I also had to ensure that the user entered a valid book and member id, an integer type or it would crash when trying to check the book ID. However, I forgot to add validation for the member ID, which allowed for any integer to be assigned as a member ID regardless of whether it is a valid member in the system. To fix this, I created a new method of the member Manager database called get member by ID. This works identically to the get member by username method however takes in the member ID instead. If it returns None, then the member is not in the database and so the user has entered an invalid member.

| id | book_id | member_id  | loan_date  | due_date   | returned |
|----|---------|------------|------------|------------|----------|
| 1  | 1       | 1          | 2025-04-10 | 2025-04-24 | 1        |
| 2  | 1       | 1          | 2025-04-13 | 2025-04-27 | 0        |
| 3  | 2       | 9999999999 | 21-04-2025 | 05-05-2025 | 0        |

Screenshot of the loans table showing any user can be member ID could be entered.

```
def get_member_by_id(self, member_id):
    cursor = self.conn.cursor()
    cursor.execute(''SELECT * FROM members WHERE id = ?'', (member_id,))
    #creates a list of the user information
    row = cursor.fetchone()
    if row != None: # if the user is found create a member object with their infomation
        return Member(row[1], row[2], row[3], row[4])
    return None # if no user
```

To fix this issue, the get member\_by\_id method was developed to check whether a valid member id was entered.

```
Enter Book ID: 1
Enter Member ID: 9999999999999999
You have entered an incorrect member ID
```

When a member is not found in the database a loan cannot occur.

```
Enter Book ID: 23
Enter Member ID: 8
You have successfully loaned out a book, your loan is due on 05-05-2025.
```

When a member is found, and a valid book is entered, a loan occurs.

```

def loan_book(self, book_id, member_id):
    cursor = self.conn.cursor()
    #SQL query to see whether the book is out on loan
    cursor.execute('''SELECT checked_in FROM books WHERE id = ?''', (book_id,))
    book = cursor.fetchone()
    #the user has entered an incorrect book id, the book is not in the database
    if book == None:
        return "You have entered an incorrect book id."
    #if the checked in field is 0/False, then the book is currently on loan
    if book[0] == 0:
        return "This book is currently out on loan."

    loan_date = datetime.today().strftime("%d-%m-%Y") # the loan date, today
    due_date = (datetime.today() + timedelta(days=14)).strftime("%d-%m-%Y") # 14 days from loan date

    cursor.execute('''INSERT INTO loans (book_id, member_id, loan_date, due_date) VALUES (?, ?, ?, ?)''',
        (book_id, member_id, loan_date, due_date))
    # adds the book, member, loan date and due date in the loans table
    cursor.execute('''UPDATE books SET checked_in = 0 WHERE id = ?''', (book_id,)) #change book to loaned out
    self.conn.commit()
    return "You have successfully loaned out a book, your loan is due on {}".format(due_date)

```

Book Manager loan book method

```

def loan_book():
    # code/function for loaning a book
    book_manager=BookManager("books2.db")
    member_manager = MemberManager("members2.db")
    try: book_id = int(input("Enter Book ID: "))
    except ValueError:
        print("The book ID should be a number.")
        return
    try: member_id = int(input("Enter Member ID: "))
    except ValueError:
        print("The member ID should be a number.")
        return
    is_valid_user= member_manager.get_member_by_id(member_id)
    if is_valid_user==None:
        print("You have entered an incorrect member ID")
        return
    result = book_manager.loan_book(book_id, member_id)
    print(result)

```

Inline loan book function

I then created a return book function. This works by taking in an entry Book ID and then attempting to find the loan in the loans table where the book id equals the entry book ID and where returned = 0, where it has an active loan. This should only be able to return at maximum 1 row, as there should only be one loan for a specific book at one time. If this loan is found it changes the returned state of the book to 1, making it an inactive loan, as well as changing the checked\_in state of the book back to 1, so another loan of the book can be taken out. If a row is not returned, this means there isn't an active loan for this book and so this is returned to the user.

```
def return_book(self, book_id): #Function to return a loaned book
    cursor = self.conn.cursor()
    cursor.execute('UPDATE loans SET returned = 1 WHERE book_id = ? AND returned = 0', (book_id,))
    #if not found in the loans database or has been returned already
    if cursor.rowcount == 0:
        return "This book is not currently loaned out."
    else:
        #change loan returned field to 1/True
        cursor.execute('UPDATE books SET checked_in = 1 WHERE id = ?', (book_id,)) #change book to returned
        self.conn.commit()
        return "You have successfully returned the book with book id {}".format(book_id)
```

Book Manager return books method.

```
def return_book():
    # function for returning a book
    book_manager=BookManager("books2.db")
    try: book_id = int(input("Enter Book ID"))
    except ValueError:
        print("The book ID should be a number.")
        return
    result = book_manager.return_book(book_id)
    print(result)
```

Inline return book function

Finally, I created a function to return all of a user's loans. This works by taking in a member ID and doing a SQL query to search for all book of that user's loans. The book ID of these loans are then used to join the information in the loans table with information about the book, such as its title and author. All these loans are then return in a list which are then linearly printed out to the user. However, I had an issue as the returned field contains a 1 or a 0 which may be confusing to a user or librarian to see whether their book is returned or checked in. Therefore, within the for loop I checked that index of that specific loan to see whether it is a 1 or 2 and assigned a text indicator of Returned or Not returned for clarity.

```
def view_member_loans():#function to view all loans from one member
    book_manager=BookManager("books2.db")
    try: member_id = int(input("Enter Member ID: "))
    except ValueError:
        print("The member ID should be a number.")
        return
    loans = book_manager.get_member_loans(member_id)
    if loans == []:#if the loans list is empty
        print("There are no loans for this member.")
    else:
        for i in range(len(loans)):
            #references the specific returned books field and gives a text indicator/ "status"
            if loans[i][4] == 1:
                status= "Returned"
            else:
                status = "Not Returned"
            print("Book ID: {}, Title: {}, Author: {}, Due Date: {}, Status: {}\n"
                  .format(loans[i][0], loans[i][1],loans[i][2],loans[i][3],status))
```

Inline view member loans function

```
def get_member_loans(self, member_id):#get a log of all the user's loans, (current and previous)
    cursor = self.conn.cursor()
    #return the id of the book, its title, its author, the due date and whether it has been returned
    #join both the loans and books table, with the book id as the foreign key
    cursor.execute('''SELECT books.id, books.title, books.author, loans.due_date, loans.returned
FROM loans
JOIN books ON loans.book_id = books.id
WHERE loans.member_id = ?''', (member_id,))
    loans = cursor.fetchall()
    return loans
```

Book Manager get member loans method

Book ID: 1, Title: To Kill a Mockingbird, Author: Harper Lee, Due Date: 2025-04-24, Status: Returned

Book ID: 1, Title: To Kill a Mockingbird, Author: Harper Lee, Due Date: 2025-04-27, Status: Not Returned

Test to see a user's loans, note one book has already been returned and another has not been returned.

## Testing Table and Evidence

| Test ID | What is being tested                                                         | Test Data                                                                             | Expected Outcome                                     | Actual Outcome (and evidence)                                                                                                                                                                                                                                                                                                                 | Success/Fail |
|---------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| T-09.1  | The user enters a correct book and member id when loaning out a book.        | Book ID = "23"<br>Member ID = "8"                                                     | Book loaned out successfully.                        | <div> Enter Book ID: 23<br/> Enter Member ID: 8<br/> You have successfully loaned out a book, your loan is due on 05-05-2025. </div> Expected result.                                                                                                                                                                                         | Success      |
| T-09.2  | The user attempts to loan out a book which is already loaned out.            | Book ID = "23" (after checking it is loaned out after test T-09.1)<br>Member ID = "2" | This book is already loaned out.                     | <div> Enter Book ID: 23<br/> Enter Member ID: 8<br/> This book is currently out on loan. </div> Expected result                                                                                                                                                                                                                               | Success      |
| T-09.3  | The user does not enter a Book ID.                                           | Book ID = ""<br>Member ID = "1"                                                       | The Book ID must be a number.                        | <div> Enter Book ID:<br/> The book ID should be a number. </div> Expected result                                                                                                                                                                                                                                                              | Success      |
| T-09.4  | The user does not enter a member ID.                                         | Book ID = "2"<br>Member ID = ""                                                       | The member ID must be a number.                      | <div> Enter Book ID: 2<br/> Enter Member ID:<br/> The member ID should be a number. </div> Expected result                                                                                                                                                                                                                                    | Success      |
| T-09.5  | The user enters an incorrect member ID                                       | Book ID = "2"<br>Member ID = "9999" (a member ID which does not exist)                | This is not a valid member.                          | <div> Enter Book ID: 1<br/> Enter Member ID: 9999999999999999<br/> You have entered an incorrect member ID </div> Expected result                                                                                                                                                                                                             | Success      |
| T-09.6  | The user enters an incorrect Book ID                                         | Book ID = "9999" (a book which does not exist)<br>Member ID = "1"                     | This is not a valid Book ID.                         | <div> Enter Book ID: 9999<br/> Enter Member ID: 2<br/> You have entered an incorrect book id. </div> z<br>Expected result                                                                                                                                                                                                                     | Success      |
| T-09.7  | The loan due date calculated should accurately be 14 days from the loan date | None                                                                                  | A new loan has a due date 14 days after a loan date. | <div> Loan date = 21/04/2025<br/> <div> Enter Book ID: 23<br/> Enter Member ID: 8<br/> You have successfully loaned out a book, your loan is due on 05-05-2025. </div> <div> 14 days from today </div> <div> Monday<br/> 5 May 2025 </div> </div> Expected<br>#Note successfully took account for going into next month (from April into May) | Success      |

|        |                                                                                                                                     |                                      |                                                      |                                                                                                                                                                                                                                                                                        |         |
|--------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| T-14.1 | A book with a valid loan is entered when attempting to return a book                                                                | BookID ="23"                         | This book has been successfully returned.            | <div> Enter Book ID23<br/> You have successfully returned the book with book id 23 </div> <div> 23      8      21-04-2025      05-05-2025      1 </div> <p>Expected result.<br/>#bottom screenshot shows the loan log was updated for returned to equal 1</p>                          | Success |
| T-14.2 | A book which does not have a valid loan is entered when attempting to return a book                                                 | Book ID = "14"                       | This book does not have an active loan.              | <div> Enter Book ID14<br/> This book is not currently loaned out. </div> <p>Expected result.</p>                                                                                                                                                                                       | Success |
| T-14.3 | An invalid book ID is entered when trying to return a book                                                                          | Book ID = "99999"                    | This book does not have an active loan.              | <div> Enter Book ID9999999<br/> This book is not currently loaned out. </div> <p>Expected result.</p>                                                                                                                                                                                  | Success |
| T-14.4 | A book ID of an invalid type is entered when trying to return a book                                                                | Book ID = "The Chronicles of Narnia" | The Book ID should be an integer                     | <div> Enter Book IDThe Chronicles of Narnia<br/> The book ID should be a number. </div> <p>Expected result.</p>                                                                                                                                                                        | Success |
| T-15.1 | When a valid member ID is entered into the view member loans feature all of their current and previous loans are shown.             | Member ID ="1"                       | Information about their current and previous loans   | <div> Book ID: 1, Title: To Kill a Mockingbird, Author: Harper Lee, Due Date: 2025-04-24, Status: Returned<br/> Book ID: 1, Title: To Kill a Mockingbird, Author: Harper Lee, Due Date: 2025-04-27, Status: Not Returned </div> <p>Expected result.<br/>#note shown above, page 86</p> | Success |
| T-15.2 | When a user who has not taken out any loans is entered into the feature, it is printed that this user has not had any loans before. | Member ID ="3"                       | Printed that this user has not had any loans before. | <div> Enter Member ID: 3<br/> There are no loans for this member. </div> <p>Expected result</p>                                                                                                                                                                                        | Success |
| T-15.3 | When a user id of an invalid type is entered into the view members loans function                                                   | Member ID ="David"                   | The member ID should be a number                     | <div> Enter Member ID: David<br/> The member ID should be a number. </div> <p>Expected result.</p>                                                                                                                                                                                     | Success |

## Key Stage Review

The completion of Development Cycle 4 marks the completion of the functions and logic of the library management system. This means the next stage will have a focus of integrating these functions with a graphical user interface as well as incorporating accessibility features into the program. Following this I will need to complete further testing for robustness. With the creation of the loans table, I have completed Success Criteria S002. The creation of the loan book, return book and view member loans functions, means that Success Criteria S008 is also complete.

| Success Criteria ID | Success Criteria                                                                                                                                                                                                                                              | Relevant Feature ID | Further Justification                                                                                                                                                                                                                                                                                             |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S001                | There will be a graphical user interface where every button works.                                                                                                                                                                                            | F1                  | This is fundamental to making the program easy and intuitive to use. Using a different interface such as a command line interface may be hard to understand and grasp for members and librarians and a natural user interface is likely to be inconvenient to use and so slow down the productivity of librarians |
| S002                | There should be a database containing tables for books, users and loans                                                                                                                                                                                       | F2                  | Having a database with these features allows the data to be easily portable, in one or two files. This is also the easiest to edit and so it is easy to keep a well-maintained store of all required information                                                                                                  |
| S003                | There should be a function to search for books by name and by their ISBN and then the book and relevant information should be outputted to the user                                                                                                           | F3                  | This enables members and librarians to check what books are in stock as well as information such as the books name or genre, essential to the operation of a library management system.                                                                                                                           |
| S004                | There should be a function to sort books by genre and output them alphabetically                                                                                                                                                                              | F3                  | This allows members to search for books with their desired genre, but also to help the organisation of the library as the books are organised by genre then in alphabetical order                                                                                                                                 |
| S005                | There should be a log in page                                                                                                                                                                                                                                 | F4                  | This aims to protect sensitive information or functions from unauthorised users and allows to dictate between member and librarian users – note Feature F5.                                                                                                                                                       |
| S006                | There should be Role Based Access control (between normal members and privileged librarian accounts) which have access to different features                                                                                                                  | F5                  | Primarily successful through success criteria S005. This is essential as there are features that librarians should be able to access to enable them to do their work that would be useless or even cause security risks for normal member users to have access to such as adding or deleting books or members.    |
| S007                | There should be a feature to add or remove books                                                                                                                                                                                                              | F6                  | Fundamental to the job of a librarian, adding and removing books as well as its metadata such as the name of the author and its genre. This is necessary to improve the database and enable users to see which books are available.                                                                               |
| S008                | There should be a feature to borrow and return books. It should check whether the book is in the database and prevent already checked out books from being checked out again. It should also log which member logged out the book according to its member ID. | F7                  | Also fundamental to the operation of a library. Enables users to check books in and out themselves, relieving this duty from librarians. Also, it will add to a loans table, allowing librarians to check loans. Note- also F8                                                                                    |
| S009                | There should be a feature to add and remove new members                                                                                                                                                                                                       | F9                  | This allows for new members to be added to the system, particularly at the start of the school year such as new sixth form students.                                                                                                                                                                              |

Key Green = Complete Orange = Progress has been made Red = Not done

## Development Cycle Part 5

### Code Development

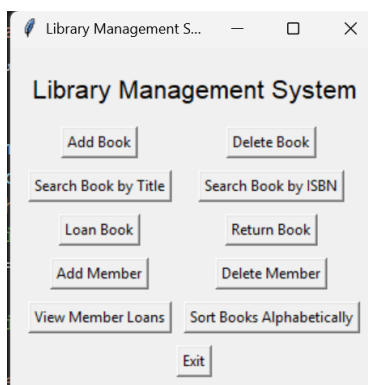
In this section I developed the Graphical user interface for the library management system. In the previous development cycles, my program had a simple terminal-based user interface where the different features were entered using a user entry. The problem with this is that it is a very unintuitive and confusing way to interact with the method as well as consisting of a large sequence of if statements which makes it bad for readability.

```
menu=int(input("Enter number"))
if menu==1:
    add_member()
if menu== 2:
    delete_member()
if menu== 3:
    add_book()
if menu== 4:
    delete_book()
if menu== 5:
    search_book_by_title()
if menu== 6:
    search_book_by_isbn()
if menu==7:
    sort_books_alphabetically_by_genre()
if menu==8:
    loan_book()
if menu==9:
    return_book()
if menu ==10:
    view_member_loans()
if menu== 11:
    quit()
```

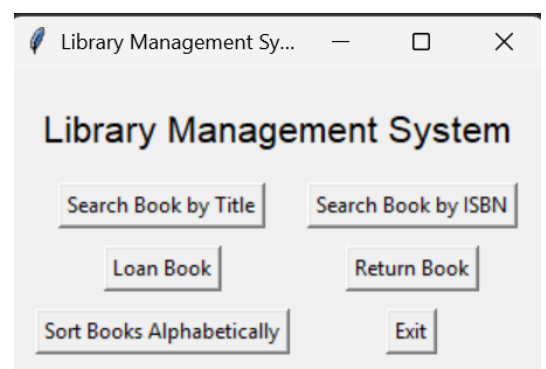
Original Terminal-based user interface

### Main Page

To create the main page, I created a new class called the LibraryApp to manage the main application screen. This class contains a method to the main screen widgets. To improve the Role-Based Access control, the library app takes in the role from the validate login function as an attribute of the LibraryApp class. Within the create widgets method, if the role is Librarian, then buttons are created for: Add Book, Delete Book, Add Member, Delete Member, View Member Loans, Search Book by Title, Search Book By ISBN, Loan Book, Return Book, Sort Books Alphabetically by Genre and Exit as they have access to every function. In comparison if a member is logged in, then only the Search Book by Title, Search Book by ISBN, Loan Book, Return Book, Sort Books Alphabetically by genre and Exit buttons are implemented as they are unauthorised to access the other features of the management system.



Librarian GUI



Member GUI

However, I had an issue getting the main page to open after the user has successfully entered their username and password. After adjusting the validate login to open the library App page if the password matches the hashed password, while the library app opens, the log in page does not close. To fix this, I had to call the `root.destroy()` function to close the login page, before opening the library app, as otherwise this would close both the login page and the library app. The library app main page is then opened using an open library app function which creates a new library root window, creating an object of the library app class called app, which takes in the user role, and then starts a new main loop.

```
def validate_login():
    #take username and password entry
    entry_username =entered_username.get()
    entry_password=entered_password.get()
    # create object of the member manager class and connect to the members table
    member_manager = MemberManager("members2.db")
    get_member = member_manager.get_member_by_username(entry_username)

    if get_member != None:
        #hashes the user entered password using the salt from the hashed password
        if bcrypt.checkpw(entry_password.encode('utf-8'), get_member.hashed_password) == True:
            messagebox.showinfo("Successful Login", "{} has logged into the system, your role is {}".format(get_member.username,get_member.role))

        else:
            messagebox.showerror("Login Failed", "incorrect username or password")
    else:
        messagebox.showerror("Login Failed", "incorrect username or password")
```

Old validate login method

```
def validate_login():
    #take username and password entry
    entry_username =entered_username.get()
    entry_password=entered_password.get()
    # create object of the member manager class and connect to the members table
    member_manager = MemberManager("members2.db")
    get_member = member_manager.get_member_by_username(entry_username)

    if get_member != None:
        #hashes the user entered password using the salt from the hashed password
        if bcrypt.checkpw(entry_password.encode('utf-8'), get_member.hashed_password) == True:
            messagebox.showinfo("Successful Login", "{} has logged into the system, your role is {}".format(get_member.username,get_member.role))
            root.destroy()

            #open library app window and pass in the user's role
            open_library_app(get_member.role)
        else:
            messagebox.showerror("Login Failed", "incorrect username or password")
    else:
        messagebox.showerror("Login Failed", "incorrect username or password")
```

New validate login method which creates a opens the library app and closes the login page.

## Buttons and Functions

On the click of each of the buttons it will run the different inline functions from before. As a result, I had to convert these functions to methods of the library class. I also made the member manager and book manager object instantiations attributes of the library app class, so they do not need to be run every time a new command is run.

```
def create_widgets(self):
    if self.role == 'librarian': # will only show if on a librarian account
        self.frame.pack(padx=10, pady=10)
        tk.Label(self.frame, text="Library Management System", font=("Arial", 16)).grid(row=0, columnspan=2, pady=10)
        tk.Button(self.frame, text="Add Book", command=self.add_book).grid(row=1, column=0, padx=5, pady=5)
        tk.Button(self.frame, text="Delete Book", command=self.delete_book).grid(row=1, column=1, padx=5, pady=5)
        tk.Button(self.frame, text="Add Member", command=self.add_member).grid(row=4, column=0, padx=5, pady=5)
        tk.Button(self.frame, text="View Member Loans", command=self.view_member_loans).grid(row=5, column=0, padx=5, pady=5)
        tk.Button(self.frame, text="Delete Member", command=self.delete_member).grid(row=4, column=1, padx=5, pady=5)
        tk.Button(self.frame, text="Search Book by Title", command=self.search_book_by_title).grid(row=2, column=0, padx=5, pady=5)
        tk.Button(self.frame, text="Search Book by ISBN", command=self.search_book_by_isbn).grid(row=2, column=1, padx=5, pady=5)
        tk.Button(self.frame, text="Loan Book", command=self.loan_book).grid(row=3, column=0, padx=5, pady=5)
        tk.Button(self.frame, text="Return Book", command=self.return_book).grid(row=3, column=1, padx=5, pady=5)
        tk.Button(self.frame, text="Sort Books Alphabetically", command=self.sort_books_alphabetically_by_genre).grid(row=5, column=1, padx=5, pady=5)
        tk.Button(self.frame, text="Exit", command=self.root.destroy).grid(row=6, columnspan=2, padx=5, pady=5)

    elif self.role == 'member': # will only show if on a member account
        self.frame.pack(padx=10, pady=10)
        tk.Label(self.frame, text="Library Management System", font=("Arial", 16)).grid(row=0, columnspan=2, pady=10)
        tk.Button(self.frame, text="Search Book by Title", command=self.search_book_by_title).grid(row=1, column=0, padx=5, pady=5)
        tk.Button(self.frame, text="Search Book by ISBN", command=self.search_book_by_isbn).grid(row=1, column=1, padx=5, pady=5)
        tk.Button(self.frame, text="Loan Book", command=self.loan_book).grid(row=2, column=0, padx=5, pady=5)
        tk.Button(self.frame, text="Return Book", command=self.return_book).grid(row=2, column=1, padx=5, pady=5)
        tk.Button(self.frame, text="Sort Books Alphabetically", command=self.sort_books_alphabetically_by_genre).grid(row=3, column=0, padx=5, pady=5)
        tk.Button(self.frame, text="Exit", command=self.root.destroy).grid(row=3, column=1, padx=5, pady=5)

def add_member(self):
```

Library app, create widgets method. Note RBAC changing which widgets are shown depending on the user's role.

## Alterations to individual functions

Rather than a terminal system, each function needs to have their own graphic user interface page. As a result, its previous logic will be wrapped in a submit function which will be run when the user clicks the appropriate submit button. In addition to this, typical inputs are replaced by a .get() with entry fields, like the login page. In addition to this, rather than printing successful notifications or errors in the terminal, it was changed to use a message.showinfo() or message.showerror() box instead. As well as this, during this cycle, I took the time to change some messages and errors to more clear and grammatically correct sentences to improve user experience. However, in some of the search book functions when testing I had issues where the books from a previous search were not deleted making the search function overcrowded and useless. Therefore, to fix this I used the result\_text.delete(1.0, tk.End) command to clear the old results before populating the results table with new results. In addition to this, the logic for the add member function was drastically changed. As the GUI would crash or close if an invalid email or password were entered, there is no longer a requirement for the password and email match variables and so this does not need to be encased in a while loop, error message boxes can instead just be shown to the user. Furthermore, during testing I discovered that for adding members and books, there was not a test to see whether user entries were not blank and so a user could store a book with no information. To fix this, I used an if statement to check whether any of the user entry fields were blank, and if they were, then to show an error message and so prevent the book from being created. As well as this, during the login, to improve security I replaced the user entry for their password with asterisks to prevent other people from seeing the password. I then used the same thing for the add member password entry.

```
def add_book(self):
    def submit():
        #takes in user entry
        title = title_entry.get()
        author = author_entry.get()
        genre = genre_entry.get()
        isbn = isbn_entry.get()
        if title=="" or author=="" or genre=="" or isbn=="":
            messagebox.showerror("Invalid Input", "You must enter entries for every field.")
            return
        #creates new book object
        book = Book(title, author, genre, isbn)
        #attempts to insert it into the database
        result = self.book_manager.add_book(book)
        messagebox.showinfo("Result", result)
        add_book_window.destroy()

    add_book_window = tk.Toplevel(self.root)
    add_book_window.title("Add Book")

    tk.Label(add_book_window, text="Title").grid(row=0, column=0, padx=5, pady=5)
    title_entry = tk.Entry(add_book_window)
    title_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Label(add_book_window, text="Author").grid(row=1, column=0, padx=5, pady=5)
    author_entry = tk.Entry(add_book_window)
    author_entry.grid(row=1, column=1, padx=5, pady=5)

    tk.Label(add_book_window, text="Genre").grid(row=2, column=0, padx=5, pady=5)
    genre_entry = tk.Entry(add_book_window)
    genre_entry.grid(row=2, column=1, padx=5, pady=5)

    tk.Label(add_book_window, text="ISBN").grid(row=3, column=0, padx=5, pady=5)
    isbn_entry = tk.Entry(add_book_window)
    isbn_entry.grid(row=3, column=1, padx=5, pady=5)

    tk.Button(add_book_window, text="Submit", command=submit).grid(row=4, columnspan=2, pady=10)
```

Library App add book method and its user interface. Note the if statement to check for the presence of user entries

```
def delete_book(self):
    # function to delete book from database
    def submit():
        try: book_id = int(book_id_entry.get())
        except ValueError:
            #if the user did not enter an integer
            messagebox.showerror("Invalid Input", "Book ID must be a number.")
            return
        #calls delete book function in book manager to remove the book id from the database
        result = self.book_manager.delete_book(book_id)
        messagebox.showinfo("Result", result)
        delete_book_window.destroy()

    delete_book_window = tk.Toplevel(self.root)
    delete_book_window.title("Delete Book")

    tk.Label(delete_book_window, text="Book ID").grid(row=0, column=0, padx=5, pady=5)
    book_id_entry = tk.Entry(delete_book_window)
    book_id_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Button(delete_book_window, text="Submit", command=submit).grid(row=1, columnspan=2, pady=10)
```

Library App delete book method and its user interface. Almost identical to original function.

```
def search_book_by_title(self):
    # code to search for a book according to its title
    def submit():
        booktitle = title_entry.get()
        result.delete(1.0, tk.END) # clear old results
        #2d list containing filtered books
        filteredbooks = self.book_manager.search_book_by_title(booktitle) #find the book() using a sql like query, function from book manager
        if filteredbooks == []: #if book not found and so list is empty
            result.insert(tk.END, "No books found.")
        else:
            for i in range(len(filteredbooks)):
                #checks the status of the current book
                if filteredbooks[i][5] == 1:
                    status = "Checked in"
                else:
                    status = "Checked Out"
                result.insert(tk.END, "ID: {}, Title: {}, Author: {}, Genre: {}, ISBN: {}, Status: {}\n".format(filteredbooks[i][0], filteredbooks[i][1], filteredbooks[i][2], filteredbooks[i][3], filteredbooks[i][4], status))

    search_book_window = tk.Toplevel(self.root)
    search_book_window.title("Search Book by Title")

    tk.Label(search_book_window, text="Title").grid(row=0, column=0, padx=5, pady=5)
    title_entry = tk.Entry(search_book_window)
    title_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Button(search_book_window, text="Submit", command=submit).grid(row=1, columnspan=2, pady=10)

    result = tk.Text(search_book_window, width=60, height=15)
    result.grid(row=2, columnspan=2, padx=5, pady=5)
```

Library App search book by title method and its user interface. Almost identical to original function. User interface consists of a title search and a massive text box for the result and a submit button.

```

def search_book_by_isbn(self):
    def submit():
        searched_isbn = isbn_entry.get()
        book = self.book_manager.binary_search(searched_isbn)
        result.delete(1.0, tk.END)
        if book == None: # if book is not found
            result.insert(tk.END, "No book found with that ISBN.")
        else:
            result.insert(tk.END, "Title: {}, Author: {}, Genre: {}, ISBN: {}\n"
                              .format(book.title, book.author, book.genre, book.isbn))

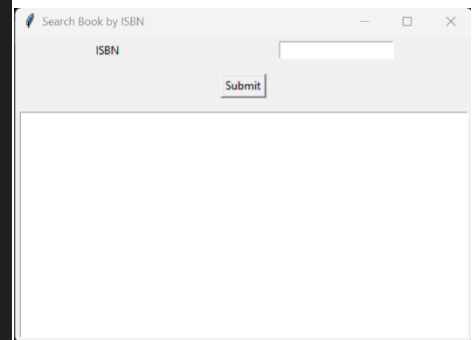
    search_book_window = tk.Toplevel(self.root)
    search_book_window.title("Search Book by ISBN")

    tk.Label(search_book_window, text="ISBN").grid(row=0, column=0, padx=5, pady=5)
    isbn_entry = tk.Entry(search_book_window)
    isbn_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Button(search_book_window, text="Submit", command=submit).grid(row=1, columnspan=2, pady=10)

    result = tk.Text(search_book_window, width=60, height=15)
    result.grid(row=2, columnspan=2, padx=5, pady=5)

```



Library App search book by ISBN method and its user interface. Almost identical to original function. User interface consists of an ISBN search and a massive text box for the result and a submit button.

```

def sort_books_alphabetically_by_genre(self):
    def submit():
        genre = genre_entry.get().strip()
        books = self.book_manager.fetch_all_books() # fetch the list of all books in the database
        if genre != "":
            books = self.book_manager.sort_books_by_genre(genre) # filter for all books of that genre

        sortedbooks = self.book_manager.merge_sort(books, 0, len(books)-1) # merge sort to get it into alphabetical order so it can be placed

        result.delete(1.0, tk.END)

        # if there are no books within that genre
        if sortedbooks == []:
            result.insert(tk.END, "No books found of the genre {}".format(genre))
        else:
            # print all of the books and its data in order
            for i in range(len(sortedbooks)):
                result.insert(tk.END, "Title: {}, Author: {}, Genre: {}, ISBN: {}\n"
                              .format(sortedbooks[i].title, sortedbooks[i].author, sortedbooks[i].genre, sortedbooks[i].isbn))

    sort_books_window = tk.Toplevel(self.root)
    sort_books_window.title("Sort Books Alphabetically")

    tk.Label(sort_books_window, text="Genre").grid(row=0, column=0, padx=5, pady=5)
    genre_entry = tk.Entry(sort_books_window)
    genre_entry.grid(row=0, column=1, padx=5, pady=5)
    tk.Button(sort_books_window, text="Sort Books", command=submit).grid(row=1, columnspan=2, pady=10)

    # text widget for the results
    result = tk.Text(sort_books_window, width=60, height=15)
    result.grid(row=2, columnspan=2, padx=5, pady=5)

```



Library App sort book alphabetically by genre method and its user interface. Almost identical to original function. User interface consists of a genre search and a massive text box for the result and a submit button.

```

def loan_book(self):
    # code/function for loaning a book
    def submit():
        try: book_id = int(book_id_entry.get())
        except ValueError:
            messagebox.showerror("Invalid Entry", "The book ID should be a number.")
            return
        try: member_id = int(member_id_entry.get())
        except ValueError:
            messagebox.showerror("Invalid Entry", "The member ID should be a number.")
            return

        is_valid_user = self.member_manager.get_member_by_id(member_id)
        if is_valid_user == None:
            messagebox.showerror("Invalid Input", "You have entered an incorrect Member ID.")
            return

        result = self.book_manager.loan_book(book_id, member_id)
        messagebox.showinfo("Result", result)
        loan_book_window.destroy()

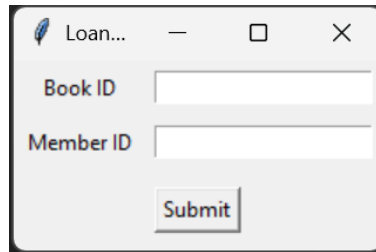
    loan_book_window = tk.Toplevel(self.root)
    loan_book_window.title("Loan Book")

    tk.Label(loan_book_window, text="Book ID").grid(row=0, column=0, padx=5, pady=5)
    book_id_entry = tk.Entry(loan_book_window)
    book_id_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Label(loan_book_window, text="Member ID").grid(row=1, column=0, padx=5, pady=5)
    member_id_entry = tk.Entry(loan_book_window)
    member_id_entry.grid(row=1, column=1, padx=5, pady=5)

    tk.Button(loan_book_window, text="Submit", command=submit).grid(row=2, columnspan=2, pady=10)

```



Library App return book method and its user interface. Almost identical to original function. User interface consists of a Book ID entry and a submit button

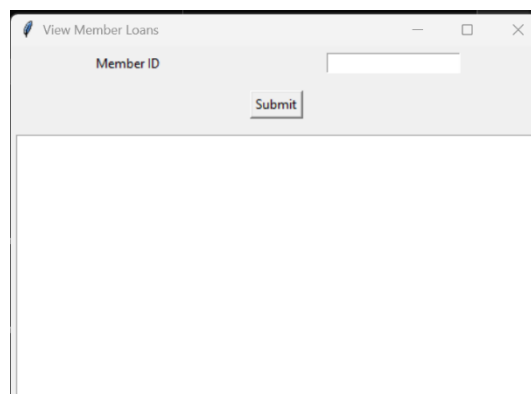
```
def view_member_loans(self):#function to view all loans from one member
def submit():
    try: member_id = int(member_id_entry.get())
    except ValueError:
        messagebox.showerror("Invalid Entry", "The Member ID should be a number.")
        return
    loans = self.book_manager.get_member_loans(member_id)
    result.delete(1.0, tk.END)
    if loans == []:#if the loans list is empty
        result.insert(tk.END, "There are no loans for this member.")
    else:
        for i in range(len(loans)):
            #references the specific returned books field and gives a text indicator/ "status"
            if loans[i][4] == 1:
                status= "Returned"
            else:
                status = "Not Returned"
            result.insert(tk.END, "Book ID: {}, Title: {}, Author: {}, Due Date: {}, Status: {}\n".
                           format(loans[i][0], loans[i][1],loans[i][2],loans[i][3],status))

view_loans_window = tk.Toplevel(self.root)
view_loans_window.title("View Member Loans")

tk.Label(view_loans_window, text="Member ID").grid(row=0, column=0, padx=5, pady=5)
member_id_entry = tk.Entry(view_loans_window)
member_id_entry.grid(row=0, column=1, padx=5, pady=5)

tk.Button(view_loans_window, text="Submit", command=submit).grid(row=1, columnspan=2, pady=10)

result = tk.Text(view_loans_window, width=60, height=15)
result.grid(row=2, columnspan=2, padx=5, pady=5)
```



Library App view member loans method and its user interface. Almost identical to original function. User interface consists of a Member ID entry and a submit button and a massive text box to print results.

```

def add_member(self):
    def submit():
        username = name_entry.get().strip() #remove any whitespace
        email = email_entry.get().strip()
        password = password_entry.get()

        if username == "" or email == "":
            messagebox.showerror("Invalid Entry", "You need to enter an entry for all fields.")
        #regular expression, to validate whether an email address was entered
        email_regex = r'^[a-zA-Z0-9_%.+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,})$'

        #ensuring valid username and password
        if re.match(email_regex, email)==None:
            messagebox.showerror("Invalid Email", "You have entered an invalid email address.")
            return

        if len(password) < 8: # make sure the password is at least 8 characters long
            messagebox.showerror("Weak Password", "Your password should be at least 8 characters long.")
            return

        #hash the entered password
        hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())

        # creates a new member class
        member = Member(username, email, hashed_password)

        # add the member to the database
        result = self.member_manager.add_member(member)
        messagebox.showinfo("Result", result)

        add_member_window.destroy()

    # create new window
    add_member_window = tk.Toplevel(self.root)
    add_member_window.title("Add Member")

    tk.Label(add_member_window, text="Username").grid(row=0, column=0, padx=5, pady=5)
    name_entry = tk.Entry(add_member_window)
    name_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Label(add_member_window, text="Email").grid(row=1, column=0, padx=5, pady=5)
    email_entry = tk.Entry(add_member_window)
    email_entry.grid(row=1, column=1, padx=5, pady=5)

    tk.Label(add_member_window, text="Password").grid(row=2, column=0, padx=5, pady=5)
    password_entry = tk.Entry(add_member_window, show="*")
    password_entry.grid(row=2, column=1, padx=5, pady=5)

    tk.Button(add_member_window, text="Submit", command=submit).grid(row=3, columnspan=2, pady=10)

```

Library App add member method and its user interface. Note that the while loop was removed as the page will only close when a successful member is added, or the page is closed. Note also that only the username and email is checked to see whether it has an entry, as if the password is empty, it would get blocked as no characters is smaller than 8. User interface consists of a username, email and password entry box and a submit button.

```

def delete_member(self):
    # code for deleting memeber from member database using member ID
    def submit():
        try: member_id = int(member_id_entry.get())
        except ValueError:
            messagebox.showerror("Invalid Entry", "The member ID should be a number.")
            return
        result = self.member_manager.delete_member(member_id)
        messagebox.showinfo("Result", result)
        delete_member_window.destroy()

    delete_member_window = tk.Toplevel(self.root)
    delete_member_window.title("Delete Member")

    tk.Label(delete_member_window, text="Member ID").grid(row=0, column=0, padx=5, pady=5)
    member_id_entry = tk.Entry(delete_member_window)
    member_id_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Button(delete_member_window, text="Submit", command=submit).grid(row=1, columnspan=2, pady=10)

```

Library App delete member method and its user interface. The User interface consists of a member ID box and a submit button.

## High visibility mode

To improve the useability of my program, particularly as one of my stakeholders is visually impaired and another has protanopia I developed a button to switch the theme of the app from black text on white background to yellow text on black background. This works by having its own button on the main page which on click changes an attribute of the library class which is a Boolean value to note whether high contrast is on or not (True / False). If high contrast is True, then since the user has clicked it then it runs a function to disable the high contrast theme and set high contrast to false. On the next click, since high contrast is False, then the enable high contrast function is run and high contrast is changed to True. The enable and disable high contrast functions work by recursively attempting to change the background of every widget in the window to black or the system grey as well as the text to white or yellow. However, since not every widget can be configured, I had to wrap this in a try and except block which will thusly attempt to configure the widget or pass as otherwise this would crash the GUI. However, I encountered a major failure. While the background of the main page can be changed to black, I was not able to convert the background of any widgets, such as add book or delete book to change colour to black, however the text was able to be changed to yellow.

```
def toggle_high_contrast(self):
    #swaps between high visibility/contrast theme and normal theme
    if self.is_high_contrast==True:
        self.disable_high_contrast()
        self.is_high_contrast= False
    else:
        self.enable_high_contrast()
        self.is_high_contrast= True

def enable_high_contrast(self):
    apply_high_contrast(self.root)
    self.root.configure(background="black")
    self.frame.configure(bg="black")

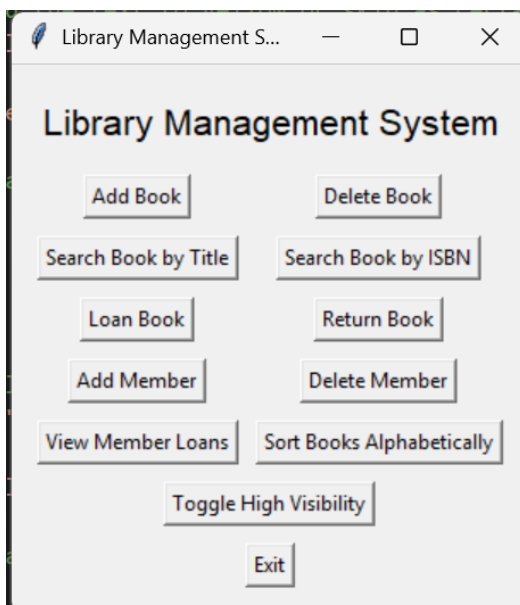
def disable_high_contrast(self):
    apply_normal_contrast(self.root)
    self.root.configure(background="SystemButtonFace")
    self.frame.configure(bg="SystemButtonFace")
```

```
def apply_high_contrast(widget):
    #changes all items in the program to yellow text with a black background
    #Try to change the background to black and the text to yellow or skip as certain widgets do not support this
    try: widget.configure(bg="black", fg="yellow")
    except: pass
    try: widget.configure(insertbackground="yellow")
    except: pass
    #recursively apply this colour scheme to all widgets
    children=widget.winfo_children()
    for i in range (len(children)):
        apply_high_contrast(children[i])

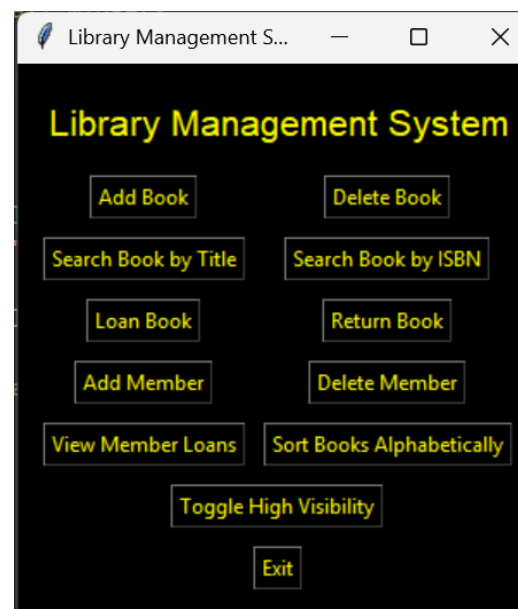
def apply_normal_contrast(widget):
    #return all the items in the program to black text on white background
    try: widget.configure(bg="SystemButtonFace", fg="black")
    except: pass
    try: widget.configure(insertbackground="black")
    except: pass
    #recursively apply this colour scheme to all widgets
    children=widget.winfo_children()
    for i in range (len(children)):
        apply_normal_contrast(children[i])
```

Toggle high contrast and enable/disable high contrast methods.

Recursive apply high contrast and apply normal contrast functions.

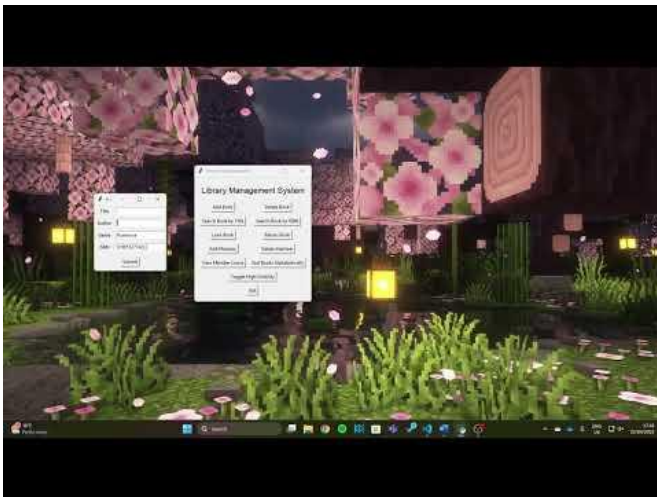


Normal Theme

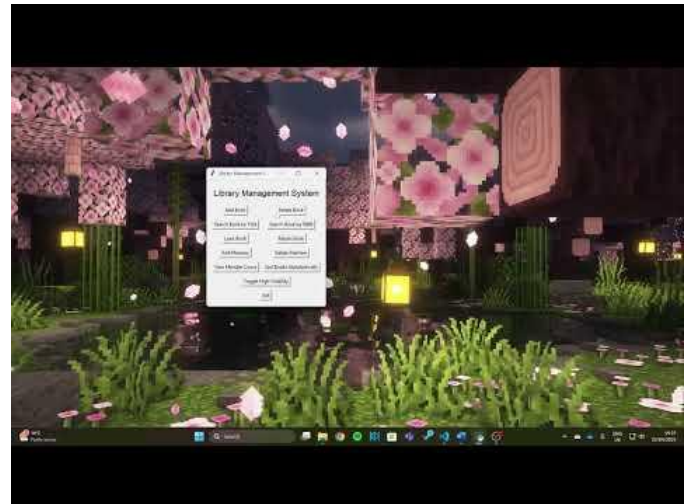


High Visibility Theme

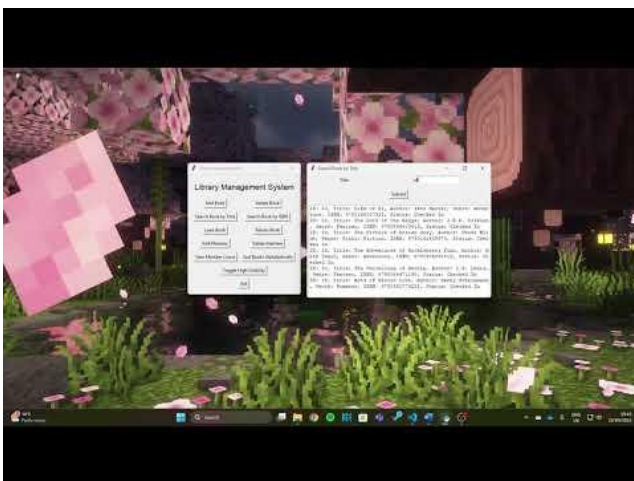
## Test Videos



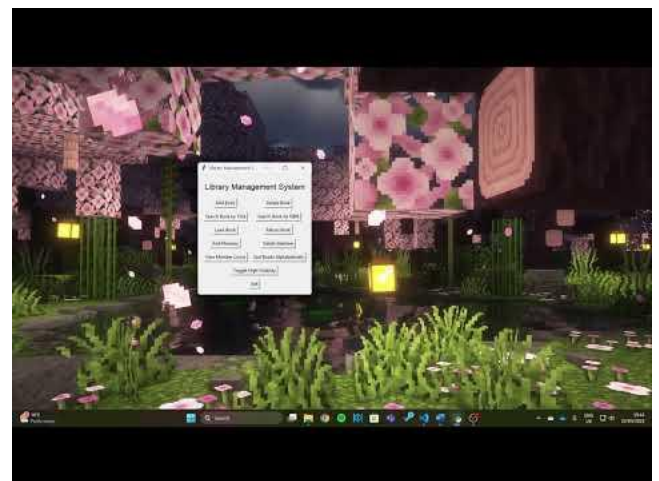
Add book test, Passes Test T-06.1 and T-06.2



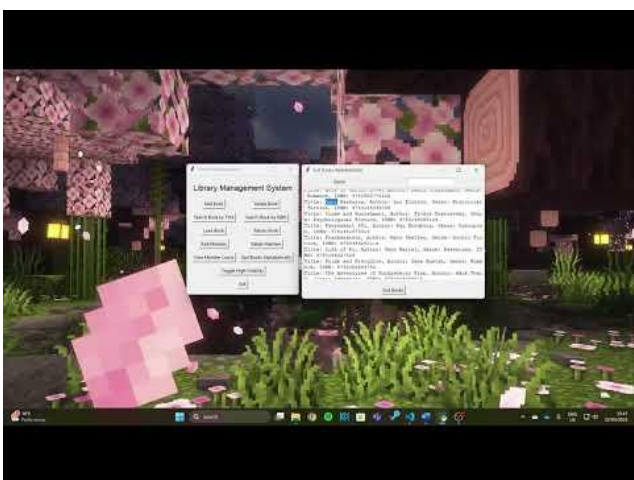
Delete book test, Passes Test T-07.1 and T-07.2 and T-07.3



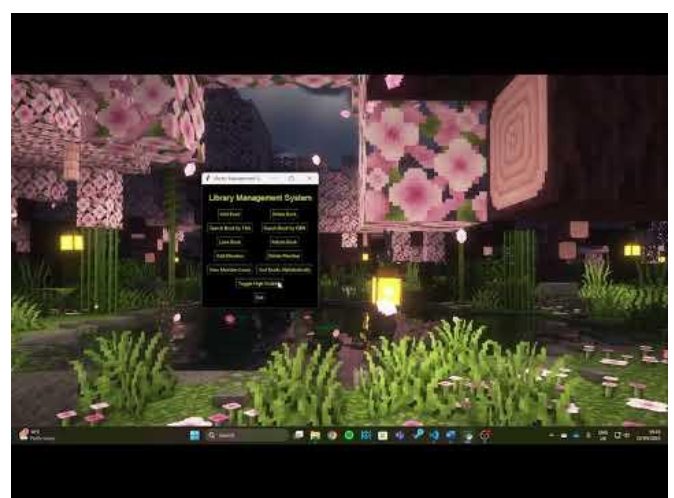
Search book by title test, Passes Test T-08.1 and T-08.2 and T-08.3



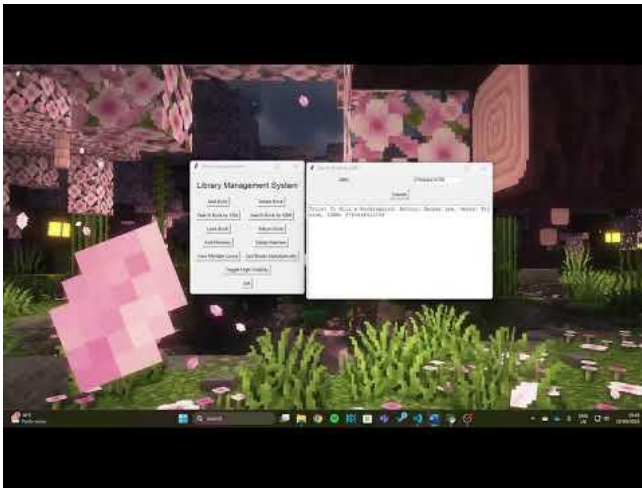
Loan book test, Passes Test T-09.1 and T-09.2 and T-09.3, T-09.4, T-09.5, T-09.6 and T-09.7.



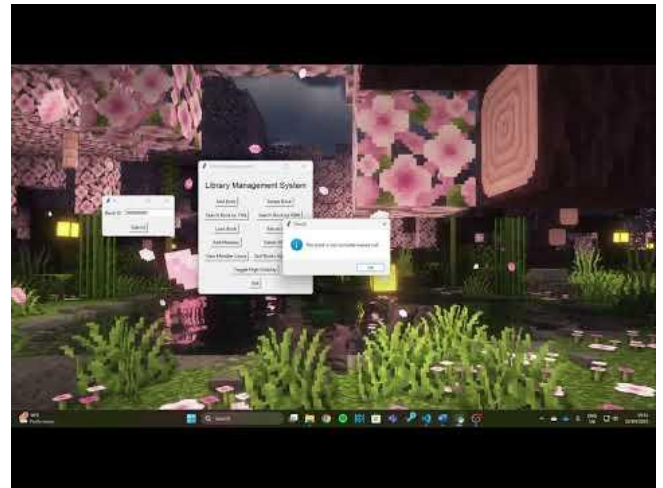
Sort Books Alphabetically by Genre test, Passes Test T-10.1 and T-10.2 and T-10.3,



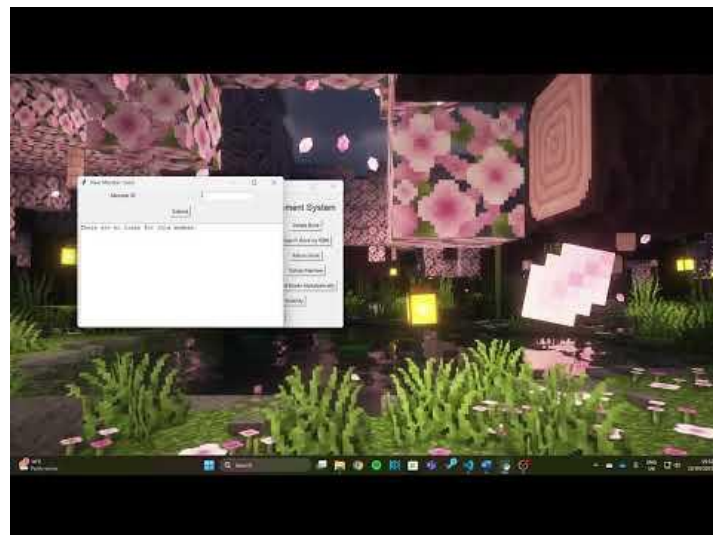
Toggle High Visibility Test. Partially Passes Test T-12



Search Book by ISBN test, Passes Test T-13.1 and T-13.2

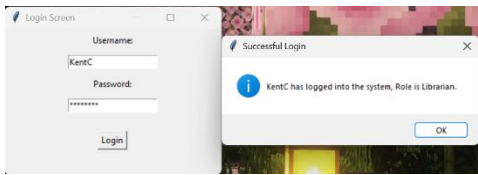
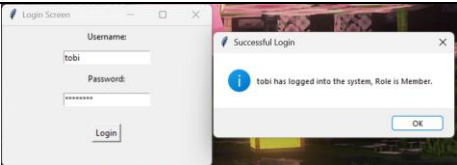
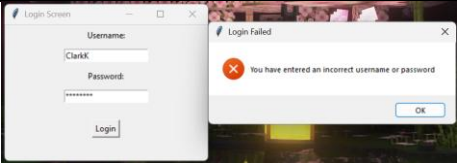
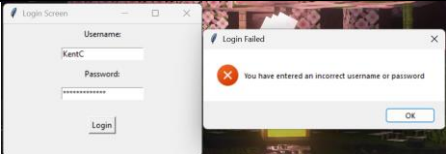
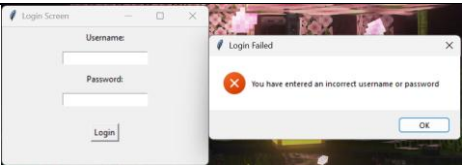


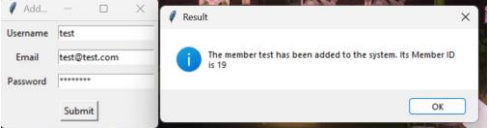
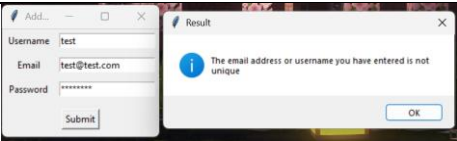
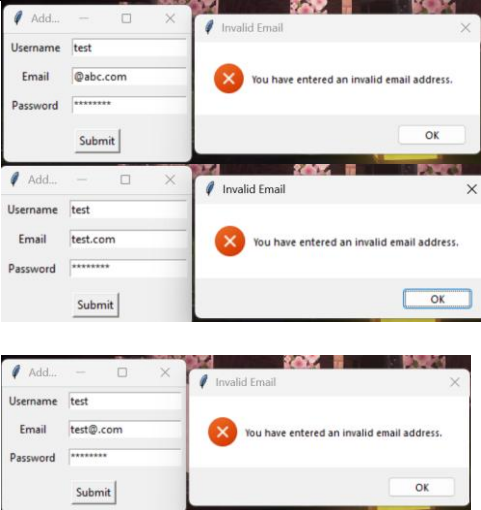
Return Book test, Passes Test T-14.1 and T-14.2 and T-14.3 and T-14.4

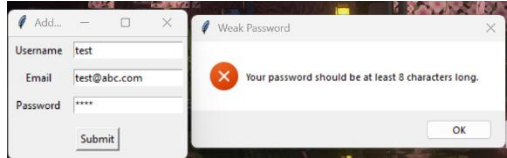
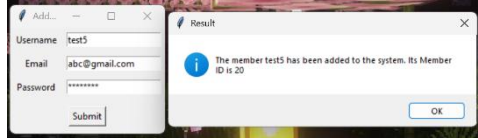
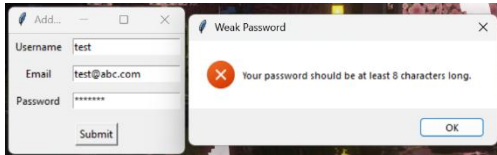
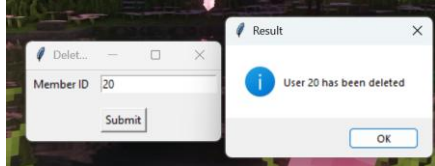
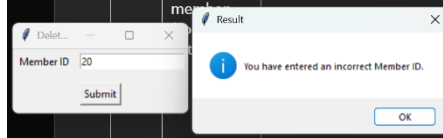
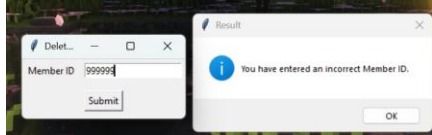
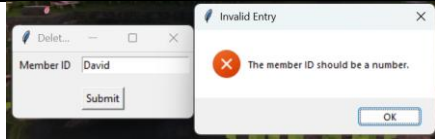


View member loans test, Passes Test T-15.1 and T-15.2 and T-15.3

## Testing Table and Evidence

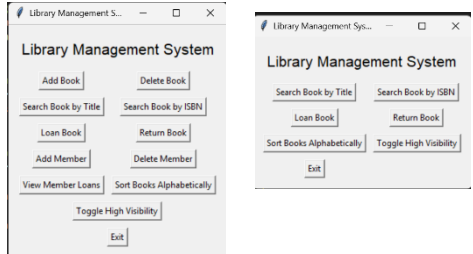
| Test ID | What is being tested                                        | Test Data                                       | Expected Outcome                                                                       | Actual Outcome (and evidence)                                                                               | Success /Fail |
|---------|-------------------------------------------------------------|-------------------------------------------------|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|---------------|
| T-01.1  | A valid username and password were entered for a librarian. | Username= "KentC"<br>Password="12345678"        | Librarian "KentC" logged into the system and the librarian management screen is shown. |  <p>Expected result</p>   | Success       |
| T-01.2  | A valid username and password were entered for a member.    | Username= "tobi"<br>Password="12345678"         | Member "tobi" is logged into the system and the member management screen is shown.     |  <p>Expected result</p>   | Success       |
| T-02.1  | An invalid username was entered.                            | Username=" ClarkK"<br>Password="12345678"       | Incorrect username or password was entered.<br>– User cannot enter the system.         |  <p>Expected result</p>   | Success       |
| T-02.2  | An invalid password was entered.                            | Username=" KentC"<br>Password = "notmypassword" | Incorrect username or password was entered.<br>– User cannot enter the system.         |  <p>Expected result</p> | Success       |
| T-02.3  | Empty Strings were entered for the login page.              | Username=""<br>Password=""                      | Incorrect username or password was entered.<br>– User cannot enter the system.         |  <p>Expected result</p> | Success       |

|        |                                                                             |                                                                                                                                                                                                                                                            |                                                                       |                                                                                                                                                                                                                                                           |         |
|--------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| T-03   | On initial run, a books and members database are created.                   | None                                                                                                                                                                                                                                                       | A new books and member database file is created.                      |  <p>This was testing using a hardcoded username and password, to get into the system, in a virtual environment, a new book database and member database was created.</p> | Success |
| T-04.1 | A member can be added to the system, when provided with normal information. | Username="test"<br>Email= <a href="mailto:test@test.com">test@test.com</a><br>Password="12345678"                                                                                                                                                          | Member added successfully.                                            |  <p>Expected result</p>                                                                                                                                                 | Success |
| T-04.2 | A duplicate email is provided to the add member.                            | Username="test"<br>Email= <a href="mailto:test@test.com">test@test.com</a><br>Password="12345678"                                                                                                                                                          | You should get an error message stating that the email is not unique. |  <p>Expected result</p>                                                                                                                                               | Success |
| T-04.3 | An invalid email is entered to the system.                                  | Username="test"<br>Email="@abc.com"<br>Password="12345678"<br><br>Username="test"<br>Email= <a href="mailto:test.com">test.com</a><br>Password="12345678"<br><br>Username="test"<br>Email= <a href="mailto:test@.com">test@.com</a><br>Password="12345678" | An error messaging showing that an invalid email address was entered. |                                                                                                                                                                       | Success |

|        |                                                                            |                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                         |         |
|--------|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| T-04.4 | The user's entered password is less than 8 characters.                     | <p>(less than 8 characters)<br/>Username="test"<br/>Email="test@abc.com"<br/>Password="1234"</p> <p>(exactly 8 characters)<br/>Username="test"<br/>Email="test@abc.com"<br/>Password="12345678"</p> <p>(Edge case: 7 characters)<br/>Username="test"<br/>Email="test@abc.com"<br/>Password="1234567"</p> | <p>Return that your entered "password is less than 8 characters".</p> <p>Password is greater than or equal to 8 characters so provided other inputs are okay, member should be added.</p> <p>Password is smaller than 8 characters so print (Password is less than 8 characters).</p> |    | Success |
| T-05.1 | A valid user id is entered to delete a member from the database.           | User ID="20"                                                                                                                                                                                                                                                                                             | User number 20 has been deleted from the system. This should be checked by trying to run the program again.                                                                                                                                                                           |                                                                                  | Success |
| T-05.2 | An invalid user id is entered to be deleted.                               | User ID ="9999"                                                                                                                                                                                                                                                                                          | There is no user with that ID.                                                                                                                                                                                                                                                        |                                                                                                                                                                     | Success |
| T-05.3 | A user id entry of an invalid type (such as string) is entered.            | User ID =" David"                                                                                                                                                                                                                                                                                        | The member ID should be a number.                                                                                                                                                                                                                                                     |                                                                                                                                                                     | Success |
| T-06.1 | A book can be added to the system, when provided with normal information . | <p>Title= "Acts of Heroic Love"</p> <p>Author = "Danny Scheinmann"</p> <p>Genre = "Romance"</p> <p>ISBN= "9780552774222"</p>                                                                                                                                                                             | The book should be added to the books table                                                                                                                                                                                                                                           | <p>See Add Book video (page 110)</p> <p>Link:<br/><a href="https://youtu.be/iGqTiXfUC9I">https://youtu.be/iGqTiXfUC9I</a></p>                                                                                                                           | Success |

|        |                                                                                          |                                                                                                                                     |                                                                                                                                                                           |                                                                                                                                   |         |
|--------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|---------|
| T-06.2 | A duplicate email is provided to the add book function.                                  | Title= "The Hunger Games 2"<br>Author = "Suzanne Collins"<br>Genre = "Dystopian"<br>ISBN= "9780439023528"<br><br>#Note after T-06.1 | You should get an error message stating that the ISBN is not unique.                                                                                                      | See Add Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/iGqTiXfUC9I">https://youtu.be/iGqTiXfUC9I</a>             | Success |
| T-07.1 | A valid Book id is entered to delete a book from the database.                           | Book ID="8"                                                                                                                         | Book number 8 has been deleted from the system. This should be checked by trying to run the program again.                                                                | See Delete book video (page 110)<br><br>Link:<br><a href="https://youtu.be/4JhQ_-IMHpM">https://youtu.be/4JhQ_-IMHpM</a>          | Success |
| T-07.2 | An invalid book id is entered to be deleted.                                             | Book ID ="9999"                                                                                                                     | There is no book with that ID.                                                                                                                                            | See Delete book video (page 110)<br>Link:<br><a href="https://youtu.be/4JhQ_-IMHpM">https://youtu.be/4JhQ_-IMHpM</a>              | Success |
| T-07.3 | A book id entry of an invalid type (such as string) is entered.                          | Book ID =" The Hunger games"                                                                                                        | The book ID should be a number.                                                                                                                                           | See Delete book video (page 110)<br>Link:<br><a href="https://youtu.be/4JhQ_-IMHpM">https://youtu.be/4JhQ_-IMHpM</a>              | Success |
| T-08.1 | The user enters a valid book title, and the right book is returned.                      | Title=" To Kill a Mockingbird"                                                                                                      | Information about the book, its ID number, title, author, genre, ISBN, whether it is checked in or not.                                                                   | See Search Book by Title video (page 110)<br><br>Link:<br><a href="https://youtu.be/cBG6LI5wEI0">https://youtu.be/cBG6LI5wEI0</a> | Success |
| T-08.2 | The user enters part of a book title and the books that match that keyword are returned. | Title=" of"                                                                                                                         | Information (see above) about the following books:<br>"The Life of Pi"<br>"The Lord of the Rings"<br>"The Picture of Dorian Gray"<br>"The Adventures of Huckleberry Finn" | See Search Book by Title video (page 110)<br><br>Link:<br><a href="https://youtu.be/cBG6LI5wEI0">https://youtu.be/cBG6LI5wEI0</a> | Success |

|        |                                                                              |                                                                                        |                                                                                 |                                                                                                                                   |         |
|--------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|---------|
|        |                                                                              |                                                                                        | “The Chronicles of Narnia” (These are the only books with “of” in their title). |                                                                                                                                   |         |
| T-08.3 | The user enters a book not in the title.                                     | Title = “Superman”                                                                     | This book has not been found.                                                   | See Search Book by Title video (page 110)<br><br>Link:<br><a href="https://youtu.be/cBG6LI5wEI0">https://youtu.be/cBG6LI5wEI0</a> | Success |
| T-09.1 | The user enters a correct book and member id when loaning out a book.        | Book ID = “1”<br><br>Member ID = “1”                                                   | Book loaned out successfully.                                                   | See Loan Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/IS7d2zvDDe0">https://youtu.be/IS7d2zvDDe0</a>            | Success |
| T-09.2 | The user attempts to loan out a book which is already loaned out.            | Book ID= “1” (after checking it is loaned out after test T-07.1)<br><br>Member ID= “2” | This book is already loaned out.                                                | See Loan Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/IS7d2zvDDe0">https://youtu.be/IS7d2zvDDe0</a>            | Success |
| T-09.3 | The user does not enter a Book ID.                                           | Book ID = “”<br>Member ID = “1”                                                        | The Book ID must be a number.                                                   | See Loan Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/IS7d2zvDDe0">https://youtu.be/IS7d2zvDDe0</a>            | Success |
| T-09.4 | The user does not enter a member ID.                                         | Book ID = “2”<br>Member ID = “”                                                        | The member ID must be a number.                                                 | See Loan Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/IS7d2zvDDe0">https://youtu.be/IS7d2zvDDe0</a>            | Success |
| T-09.5 | The user enters an incorrect member ID                                       | Book ID = “2”<br>Member ID = “9999” (a member ID which does not exist)                 | This is not a valid member.                                                     | See Loan Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/IS7d2zvDDe0">https://youtu.be/IS7d2zvDDe0</a>            | Success |
| T-09.6 | The user enters an incorrect Book ID                                         | Book ID = “9999” (a book which does not exist)<br>Member ID = 1                        | This is not a valid Book ID.                                                    | See Loan Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/IS7d2zvDDe0">https://youtu.be/IS7d2zvDDe0</a>            | Success |
| T-09.7 | The loan due date calculated should accurately be 14 days from the loan date | None                                                                                   | A new loan has a due date 14 days after a loan date.                            | See Loan Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/IS7d2zvDDe0">https://youtu.be/IS7d2zvDDe0</a>            | Success |

|        |                                                                                            |                                                                    |                                                                                                                                                                       |                                                                                                                                     |                 |
|--------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| T-10.1 | The user enters a valid genre, and the right books are filtered.                           | Genre=" Fiction"                                                   | Information on the following books:<br>"The Catcher in the Rye"<br>"The Great Gatsby"<br>"To Kill a Mockingbird"<br>"The Old Man and the Sea"<br>"The Sun also Rises" | See Search Book by Genre video (page 110)<br><br>Link:<br><a href="https://youtu.be/SUiPkDvR_TA">https://youtu.be/SUiPkDvR_TA</a>   | Success         |
| T-10.2 | The user does not enter a genre.                                                           | Genre=""                                                           | All books should be outputted in alphabetical order.                                                                                                                  | See Search Book by Genre video (page 110)<br><br>Link:<br><a href="https://youtu.be/SUiPkDvR_TA">https://youtu.be/SUiPkDvR_TA</a>   | Success         |
| T-10.3 | The user enters a genre that is not found in the database                                  | Genre = "Horror"                                                   | No books found of this genre.                                                                                                                                         | See Search Book by Genre video (page 110)<br><br>Link:<br><a href="https://youtu.be/SUiPkDvR_TA">https://youtu.be/SUiPkDvR_TA</a>   | Success         |
| T-11   | Depending on the user, a different menu scheme should be                                   | Login on both a privileged librarian account and a member account. | The librarian account should have access to the full features whereas the member should have access to a more restricted feature list.                                | <br>Librarian and member pages respectively     | Success         |
| T-12   | The system should be able to switch between the high visibility mode and the normal theme. | Click the high visibility mode button                              | One click should toggle the high visibility mode on, and the widgets should change and when clicked again it should switch back.                                      | See Toggle High Visibility video (page 110)<br><br>Link:<br><a href="https://youtu.be/Tb4sFQgBE5I">https://youtu.be/Tb4sFQgBE5I</a> | Partial Success |

|        |                                                                                           |                                      |                                                                   |                                                                                                                                  |         |
|--------|-------------------------------------------------------------------------------------------|--------------------------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|---------|
| T-13.1 | A valid ISBN is provided when searching by ISBN.                                          | ISBN="9780446310789"                 | Information on that book (title, author, genre ISBN) is returned. | See Search Book by ISBN video (page 111)<br><br>Link:<br><a href="https://youtu.be/XNgG0TIVOfs">https://youtu.be/XNgG0TIVOfs</a> | Success |
| T-13.2 | An ISBN which is not in the book table is entered when searching by ISBN.                 | ISBN="9780525559474"                 | This ISBN cannot be found in the table.                           | See Search Book by ISBN video (page 111)<br><br>Link:<br><a href="https://youtu.be/XNgG0TIVOfs">https://youtu.be/XNgG0TIVOfs</a> | Success |
| T-14.1 | A book with a valid loan is entered when attempting to return a book.                     | Book ID ="23"                        | This book has been successfully returned.                         | See Return Book video (page 111)<br><br>Link:<br><a href="https://youtu.be/SDo_iG9uAB8">https://youtu.be/SDo_iG9uAB8</a>         | Success |
| T-14.2 | A book which does not have a valid loan is entered when attempting to return a book       | Book ID = "14"                       | This book does not have an active loan.                           | See Return Book video (page 111)<br><br>Link:<br><a href="https://youtu.be/SDo_iG9uAB8">https://youtu.be/SDo_iG9uAB8</a>         | Success |
| T-14.3 | An invalid book ID is entered when trying to return a book.                               | Book ID = "99999"                    | This book does not have an active loan.                           | See Return Book video (page 111)<br><br>Link:<br><a href="https://youtu.be/SDo_iG9uAB8">https://youtu.be/SDo_iG9uAB8</a>         | Success |
| T-14.4 | A book ID of an invalid type is entered when trying to return a book.                     | Book ID = "The Chronicles of Narnia" | The Book ID should be an integer                                  | See Return Book video (page 111)<br><br>Link:<br><a href="https://youtu.be/SDo_iG9uAB8">https://youtu.be/SDo_iG9uAB8</a>         | Success |
| T-15.1 | When a valid member ID is entered into the view member loans feature all of their current | Member ID ="1"                       | Information about their current and previous loans                | See View Member Loans video (page 111)<br><br>Link:<br><a href="https://youtu.be/tBKKTH-GD58">https://youtu.be/tBKKTH-GD58</a>   | Success |

|        |                                                                                                                                     |                    |                                                      |                                                                                                                                |         |
|--------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|---------|
|        | and previous loans are shown.                                                                                                       |                    |                                                      |                                                                                                                                |         |
| T-15.2 | When a user who has not taken out any loans is entered into the feature, it is printed that this user has not had any loans before. | Member ID ="3"     | Printed that this user has not had any loans before. | See View Member Loans video (page 111)<br><br>Link:<br><a href="https://youtu.be/tBKKTH-GD58">https://youtu.be/tBKKTH-GD58</a> | Success |
| T-15.3 | When a user id of an invalid type is entered into the view members loans function                                                   | Member ID ="David" | The member ID should be a number                     | See View Member Loans video (page 111)<br><br>Link:<br><a href="https://youtu.be/tBKKTH-GD58">https://youtu.be/tBKKTH-GD58</a> | Success |

## Key Stage Review

This marks the last development stage of my program. I have successfully converted my program to use a graphical user interface as well as improving role-based access control, by not letting members access certain features. However, Success Criteria S001 and S006 is not marked off as complete as this will be evaluated by my stakeholders. Next, I need to do more thorough testing of my program to ensure robustness and functionality.

| Success Criteria ID | Success Criteria                                                                                                                                                                                                                                              | Relevant Feature ID | Further Justification                                                                                                                                                                                                                                                                                             |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S001                | There will be a graphical user interface where every button works.                                                                                                                                                                                            | F1                  | This is fundamental to making the program easy and intuitive to use. Using a different interface such as a command line interface may be hard to understand and grasp for members and librarians and a natural user interface is likely to be inconvenient to use and so slow down the productivity of librarians |
| S002                | There should be a database containing tables for books, users and loans                                                                                                                                                                                       | F2                  | Having a database with these features allows the data to be easily portable, in one or two files. This is also the easiest to edit and so it is easy to keep a well-maintained store of all required information                                                                                                  |
| S003                | There should be a function to search for books by name and by their ISBN and then the book and relevant information should be outputted to the user                                                                                                           | F3                  | This enables members and librarians to check what books are in stock as well as information such as the books name or genre, essential to the operation of a library management system.                                                                                                                           |
| S004                | There should be a function to sort books by genre and output them alphabetically                                                                                                                                                                              | F3                  | This allows members to search for books with their desired genre, but also to help the organisation of the library as the books are organised by genre then in alphabetical order                                                                                                                                 |
| S005                | There should be a log in page                                                                                                                                                                                                                                 | F4                  | This aims to protect sensitive information or functions from unauthorised users and allows to dictate between member and librarian users – note Feature F5.                                                                                                                                                       |
| S006                | There should be Role Based Access control (between normal members and privileged librarian accounts) which have access to different features                                                                                                                  | F5                  | Primarily successful through success criteria S005. This is essential as there are features that librarians should be able to access to enable them to do their work that would be useless or even cause security risks for normal member users to have access to such as adding or deleting books or members.    |
| S007                | There should be a feature to add or remove books                                                                                                                                                                                                              | F6                  | Fundamental to the job of a librarian, adding and removing books as well as its metadata such as the name of the author and its genre. This is necessary to improve the database and enable users to see which books are available.                                                                               |
| S008                | There should be a feature to borrow and return books. It should check whether the book is in the database and prevent already checked out books from being checked out again. It should also log which member logged out the book according to its member ID. | F7                  | Also fundamental to the operation of a library. Enables users to check books in and out themselves, relieving this duty from librarians. Also, it will add to a loans table, allowing librarians to check loans. Note- also F8                                                                                    |
| S009                | There should be a feature to add and remove new members                                                                                                                                                                                                       | F9                  | This allows for new members to be added to the system, particularly at the start of the school year such as new sixth form students.                                                                                                                                                                              |

**Key** Green = Complete Orange = Progress has been made Red = Not done

## Testing of Final Solution for Evaluation

### Testing of final solution for function, robustness, and usability with evidence

#### Protection against SQL Injection

As the library system is reliant on a database, I had to test the system to ensure protection against SQL injection attacks. SQL injection attacks work by trying to inject SQL queries into user inputs to manipulate the database, such as changing the role of different users or modifying or deleting records or attempting to drop the entire table.

To protect against SQL Injection attacks, any SQL queries are parameterised where user entries are entered as parameters. This allows for the SQLite3 module to escape the values before entering this into the SQL command. I tested this by attempting to drop the members table by attempting to add a user with a malicious input. However, this was unsuccessful in dropping the table and attempting to change the user's role and instead new members were just added to the system.

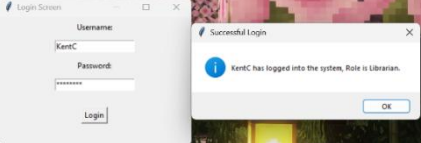
| Test Case                                                                                                                                                                              | Explanation                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Username= testuser'); DROP TABLE members; --<br>Email = <a href="mailto:safe@example.com">safe@example.com</a><br>Password= password123                                                | This attempts to enter a username and in turn drop the entire member table.                                                                  |
| Username= newadmin'; UPDATE members SET role='librarian' WHERE username='tobi'; --<br><br>Email= <a href="mailto:exploit@example.com">exploit@example.com</a><br>Password =password123 | This attempts to enter a malicious username entry and change the role of a different user 'tobi' to librarian so they get privileged access. |

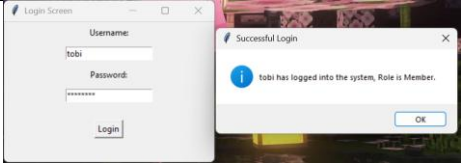
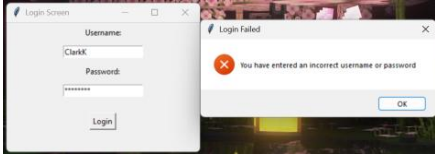
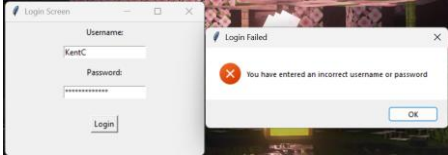
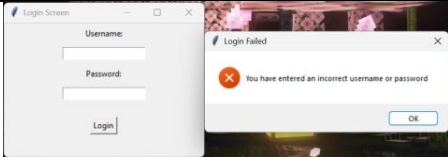
|    |                                                   |                                                              |                                                    |        |
|----|---------------------------------------------------|--------------------------------------------------------------|----------------------------------------------------|--------|
| 22 | testuser'); DROP TABLE members; -                 | <a href="mailto:safe@example.com">safe@example.com</a>       | 3650983649503687988672715399841218067741151065...  | member |
| 23 | newadmin'; UPDATE members SET role='admin' WHE... | <a href="mailto:exploit@example.com">exploit@example.com</a> | 36509836495036781159911476515780991221021109084... | member |

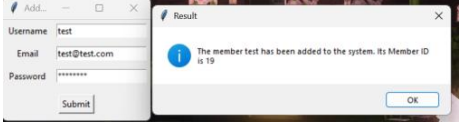
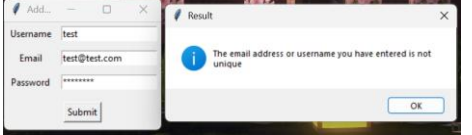
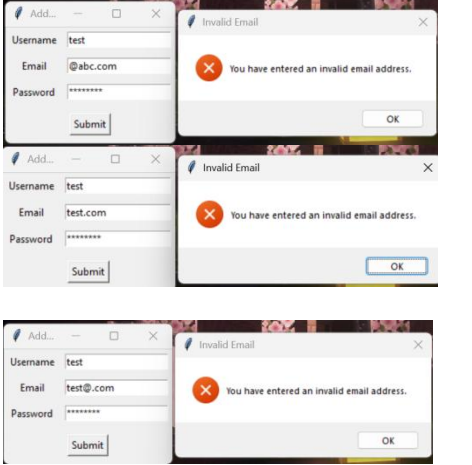
SQL injection was unsuccessful, users were just added to the system with weird usernames

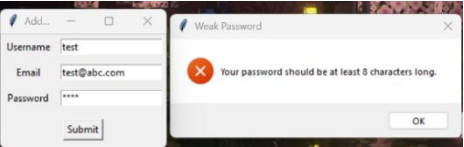
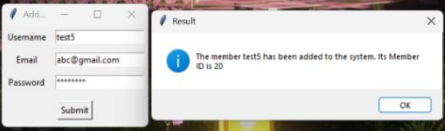
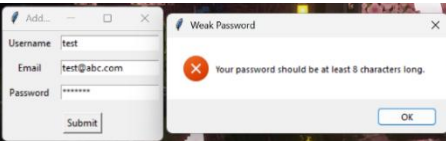
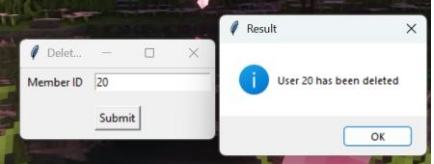
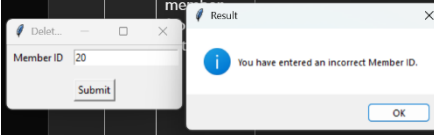
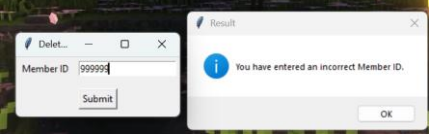
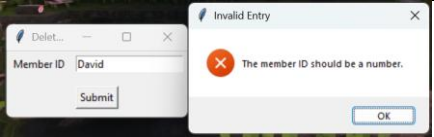
|   |      |                                                        |                                                    |        |
|---|------|--------------------------------------------------------|----------------------------------------------------|--------|
| 4 | tobi | <a href="mailto:tobi@testing.com">tobi@testing.com</a> | 36509836495036507385791218411112057103122102100... | member |
|---|------|--------------------------------------------------------|----------------------------------------------------|--------|

SQL injection was unsuccessful, the user Tobi remained a member.

| Test ID | What is being tested                                        | Test Data                                 | Expected Outcome                                                    | Actual Outcome (and evidence)                                                                               | Success/Fail | Success Criteria |
|---------|-------------------------------------------------------------|-------------------------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|--------------|------------------|
| T-01.1  | A valid username and password were entered for a librarian. | Username= "KentC"<br>Password= "12345678" | Librarian "KentC" logged into the system and the librarian manageme | <br><br>Expected result | Success      | S005             |

|        |                                                          |                                                |                                                                                    |                                                                                                             |         |      |
|--------|----------------------------------------------------------|------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|---------|------|
|        |                                                          |                                                | nt screen is shown.                                                                |                                                                                                             |         |      |
| T-01.2 | A valid username and password were entered for a member. | Username= "tobi"<br>Password= "12345678"       | Member "tobi" is logged into the system and the member management screen is shown. |  <p>Expected result</p>   | Success | S005 |
| T-02.1 | An invalid username was entered.                         | Username="ClarkK"<br>Password="12345678"       | Incorrect username or password was entered. – User cannot enter the system.        |  <p>Expected result</p>   | Success | S005 |
| T-02.2 | An invalid password was entered.                         | Username="KentC"<br>Password = "notmypassword" | Incorrect username or password was entered. – User cannot enter the system.        |  <p>Expected result</p>   | Success | S005 |
| T-02.3 | Empty Strings were entered for the login page.           | Username=""<br>Password=""                     | Incorrect username or password was entered. – User cannot enter the system.        |  <p>Expected result</p> | Success | S005 |

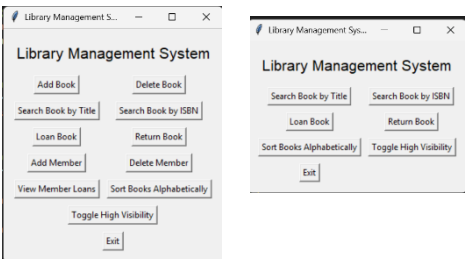
|        |                                                                             |                                                                                                                                                                                                                                                                                            |                                                                       |                                                                                                                                                                                                                                                           |         |      |
|--------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|------|
| T-03   | On initial run, a books and members database are created.                   | None                                                                                                                                                                                                                                                                                       | A new books and member database file is created.                      |  <p>This was testing using a hardcoded username and password, to get into the system, in a virtual environment, a new book database and member database was created.</p> | Success | S002 |
| T-04.1 | A member can be added to the system, when provided with normal information. | Username="test"<br>Email= <a href="mailto:test@test.com">test@test.com</a><br>Password="12345678"                                                                                                                                                                                          | Member added successfully.                                            |  <p>Expected result</p>                                                                                                                                                 | Success | S009 |
| T-04.2 | A duplicate email is provided to the add member.                            | Username="test"<br>Email=" <a href="mailto:test@test.com">test@test.com</a> "<br>Password="12345678"                                                                                                                                                                                       | You should get an error message stating that the email is not unique. |  <p>Expected result</p>                                                                                                                                               | Success | S009 |
| T-04.3 | An invalid email is entered to the system.                                  | Username="test"<br>Email=" <a href="mailto:@abc.com">@abc.com</a> "<br>Password="12345678"<br><br>Username="test"<br>Email= <a href="mailto:test.com">test.com</a><br>Password="12345678"<br><br>Username="test"<br>Email= <a href="mailto:test@.com">test@.com</a><br>Password="12345678" | An error messaging showing that an invalid email address was entered. |                                                                                                                                                                       | Success | S009 |

|        |                                                                  |                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                         |         |      |
|--------|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|------|
| T-04.4 | The user's entered password is less than 8 characters.           | <p>(less than 8 characters)<br/>Username="test"<br/>Email="test@abc.com"<br/>Password="1234"</p> <p>(exactly eight characters)<br/>Username="test"<br/>Email=test@abc.com<br/>Password="12345678"</p> <p>(Edge case: seven characters)<br/>Username="test"<br/>Email=test@abc.com<br/>Password="1234567"</p> | <p>Return that your entered "password is less than 8 characters".</p> <p>Password is greater than or equal to 8 characters so provided other inputs are okay, member should be added.</p> <p>Password is smaller than eight characters so print (Password is less than 8 characters).</p> |    | Success | S009 |
| T-05.1 | A valid user id is entered to delete a member from the database. | User ID="20"                                                                                                                                                                                                                                                                                                 | User number 20 has been deleted from the system. This should be checked by trying to run the program again.                                                                                                                                                                               |                                                                                 | Success | S009 |
| T-05.2 | An invalid user id is entered to be deleted.                     | User ID ="9999"                                                                                                                                                                                                                                                                                              | There is no user with that ID.                                                                                                                                                                                                                                                            |                                                                                                                                                                     | Success | S009 |
| T-05.3 | A user id entry of an invalid type (such as string) is entered.  | User ID ="David"                                                                                                                                                                                                                                                                                             | The member ID should be a number.                                                                                                                                                                                                                                                         |                                                                                                                                                                     | Success | S009 |

|        |                                                                           |                                                                                                                                     |                                                                                                            |                                                                                                                                   |         |      |
|--------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|---------|------|
| T-06.1 | A book can be added to the system, when provided with normal information. | Title= "Acts of Heroic Love"<br>Author = "Danny Scheinmann"<br>Genre = "Romance"<br>ISBN= "9780552774222"                           | The book should be added to the books table                                                                | See Add Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/iGqTiXfUC9I">https://youtu.be/iGqTiXfUC9I</a>             | Success | S007 |
| T-06.2 | A duplicate email is provided to the add book function.                   | Title= "The Hunger Games 2"<br>Author = "Suzanne Collins"<br>Genre = "Dystopian"<br>ISBN= "9780439023528"<br><br>#Note after T-06.1 | You should get an error message stating that the ISBN is not unique.                                       | See Add Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/iGqTiXfUC9I">https://youtu.be/iGqTiXfUC9I</a>             | Success | S007 |
| T-07.1 | A valid Book id is entered to delete a book from the database.            | Book ID="8"                                                                                                                         | Book number 8 has been deleted from the system. This should be checked by trying to run the program again. | See Delete book video (page 110)<br><br>Link:<br><a href="https://youtu.be/4JhQ_-IMHpM">https://youtu.be/4JhQ_-IMHpM</a>          | Success | S007 |
| T-07.2 | An invalid book id is entered to be deleted.                              | Book ID ="9999"                                                                                                                     | There is no book with that ID.                                                                             | See Delete book video (page 110)<br>Link:<br><a href="https://youtu.be/4JhQ_-IMHpM">https://youtu.be/4JhQ_-IMHpM</a>              | Success | S007 |
| T-07.3 | A book id entry of an invalid type (such as string) is entered.           | Book ID =" The Hunger games"                                                                                                        | The book ID should be a number.                                                                            | See Delete book video (page 110)<br>Link:<br><a href="https://youtu.be/4JhQ_-IMHpM">https://youtu.be/4JhQ_-IMHpM</a>              | Success | S007 |
| T-08.1 | The user enters a valid book title, and the right book is returned.       | Title=" To Kill a Mockingbird"                                                                                                      | Information about the book, its ID number, title, author, genre, ISBN, whether it is checked in or not.    | See Search Book by Title video (page 110)<br><br>Link:<br><a href="https://youtu.be/cBG6LI5wEI0">https://youtu.be/cBG6LI5wEI0</a> | Success | S003 |

|        |                                                                                          |                                                                                          |                                                                                                                                                                                                                                                                 |                                                                                                                                   |         |      |
|--------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|---------|------|
| T-08.2 | The user enters part of a book title and the books that match that keyword are returned. | Title=" of"                                                                              | Information (see above) about the following books:<br>"The Life of Pi"<br>"The Lord of the Rings"<br>"The Picture of Dorian Gray"<br>"The Adventures of Huckleberry Finn"<br>"The Chronicles of Narnia"<br>(These are the only books with "of" in their title). | See Search Book by Title video (page 110)<br><br>Link:<br><a href="https://youtu.be/cBG6LI5wEI0">https://youtu.be/cBG6LI5wEI0</a> | Success | S003 |
| T-08.3 | The user enters a book not in the title.                                                 | Title = "Superman"                                                                       | This book has not been found.                                                                                                                                                                                                                                   | See Search Book by Title video (page 110)<br><br>Link:<br><a href="https://youtu.be/cBG6LI5wEI0">https://youtu.be/cBG6LI5wEI0</a> | Success | S003 |
| T-09.1 | The user enters a correct book and member id when loaning out a book.                    | Book ID ="1"<br><br>Member ID ="1"                                                       | Book loaned out successfully.                                                                                                                                                                                                                                   | See Loan Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/IS7d2zvDDe0">https://youtu.be/IS7d2zvDDe0</a>            | Success | S008 |
| T-09.2 | The user attempts to loan out a book which is already loaned out.                        | Book ID= "1"<br>(after checking it is loaned out after test T-07.1)<br><br>Member ID="2" | This book is already loaned out.                                                                                                                                                                                                                                | See Loan Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/IS7d2zvDDe0">https://youtu.be/IS7d2zvDDe0</a>            | Success | S008 |
| T-09.3 | The user does not enter a Book ID.                                                       | Book ID = ""<br>Member ID ="1"                                                           | The Book ID must be a number.                                                                                                                                                                                                                                   | See Loan Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/IS7d2zvDDe0">https://youtu.be/IS7d2zvDDe0</a>            | Success | S008 |
| T-09.4 | The user does not                                                                        | Book ID = "2"<br>Member ID =""                                                           | The member                                                                                                                                                                                                                                                      | See Loan Book video (page 110)                                                                                                    | Success | S008 |

|        |                                                                              |                                                                         |                                                                                                                                                                       |                                                                                                                                   |         |      |
|--------|------------------------------------------------------------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|---------|------|
|        | enter a member ID.                                                           |                                                                         | ID must be a number.                                                                                                                                                  | Link:<br><a href="https://youtu.be/IS7d2zvDDe0">https://youtu.be/IS7d2zvDDe0</a>                                                  |         |      |
| T-09.5 | The user enters an incorrect member ID                                       | Book ID ="2"<br>Member ID ="9999"<br>(a member ID which does not exist) | This is not a valid member.                                                                                                                                           | See Loan Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/IS7d2zvDDe0">https://youtu.be/IS7d2zvDDe0</a>            | Success | S008 |
| T-09.6 | The user enters an incorrect Book ID                                         | Book ID == "9999"<br>(a book which does not exist)<br>Member ID==1      | This is not a valid Book ID.                                                                                                                                          | See Loan Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/IS7d2zvDDe0">https://youtu.be/IS7d2zvDDe0</a>            | Success | S008 |
| T-09.7 | The loan due date calculated should accurately be 14 days from the loan date | None                                                                    | A new loan has a due date 14 days after a loan date.                                                                                                                  | See Loan Book video (page 110)<br><br>Link:<br><a href="https://youtu.be/IS7d2zvDDe0">https://youtu.be/IS7d2zvDDe0</a>            | Success | S008 |
| T-10.1 | The user enters a valid genre, and the right books are filtered.             | Genre=" Fiction"                                                        | Information on the following books:<br>"The Catcher in the Rye"<br>"The Great Gatsby"<br>"To Kill a Mockingbird"<br>"The Old Man and the Sea"<br>"The Sun also Rises" | See Search Book by Genre video (page 110)<br><br>Link:<br><a href="https://youtu.be/SUiPkDvR_TA">https://youtu.be/SUiPkDvR_TA</a> | Success | S004 |
| T-10.2 | The user does not enter a genre.                                             | Genre=""                                                                | All books should be outputted in alphabetical order.                                                                                                                  | See Search Book by Genre video (page 110)<br><br>Link:<br><a href="https://youtu.be/SUiPkDvR_TA">https://youtu.be/SUiPkDvR_TA</a> | Success | S004 |
| T-10.3 | The user enters a genre that is not found in the database                    | Genre = "Horror"                                                        | No books found of this genre.                                                                                                                                         | See Search Book by Genre video (page 110)<br><br>Link:<br><a href="https://youtu.be/SUiPkDvR_TA">https://youtu.be/SUiPkDvR_TA</a> | Success | S004 |

|        |                                                                                            |                                                                    |                                                                                                                                        |                                                                                                                                         |                 |      |
|--------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|-----------------|------|
| T-11   | Depending on the user, a different menu scheme should be                                   | Login on both a privileged librarian account and a member account. | The librarian account should have access to the full features whereas the member should have access to a more restricted feature list. |  <p>Librarian and member pages respectively</p>        | Success         | S006 |
| T-12   | The system should be able to switch between the high visibility mode and the normal theme. | Click the high visibility mode button                              | One click should toggle the high visibility mode on, and the widgets should change and when clicked again it should switch back.       | <p>See Toggle High Visibility video (page 110)</p> <p>Link: <a href="https://youtu.be/Tb4sFQgBE5I">https://youtu.be/Tb4sFQgBE5I</a></p> | Partial Success | N/A  |
| T-13.1 | A valid ISBN is provided when searching by ISBN.                                           | ISBN= "9780446310789"                                              | Information on that book (title, author, genre ISBN) is returned.                                                                      | <p>See Search Book by ISBN video (page 111)</p> <p>Link: <a href="https://youtu.be/XNgG0T1VOfs">https://youtu.be/XNgG0T1VOfs</a></p>    | Success         | S003 |
| T-13.2 | An ISBN which is not in the book table is entered when searching by ISBN.                  | ISBN="9780525559474"                                               | This ISBN cannot be found in the table.                                                                                                | <p>See Search Book by ISBN video (page 111)</p> <p>Link: <a href="https://youtu.be/XNgG0T1VOfs">https://youtu.be/XNgG0T1VOfs</a></p>    | Success         | S003 |
| T-14.1 | A book with a valid loan is entered when attempting to return a book.                      | Book ID ="23"                                                      | This book has been successfully returned.                                                                                              | <p>See Return Book video (page 111)</p> <p>Link: <a href="https://youtu.be/SDo_iG9uAB8">https://youtu.be/SDo_iG9uAB8</a></p>            | Success         | S008 |

|        |                                                                                                                         |                                      |                                                    |                                                                                                                                |         |      |
|--------|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|---------|------|
| T-14.2 | A book which does not have a valid loan is entered when attempting to return a book                                     | Book ID = "14"                       | This book does not have an active loan.            | See Return Book video (page 111)<br><br>Link:<br><a href="https://youtu.be/SDo_iG9uAB8">https://youtu.be/SDo_iG9uAB8</a>       | Success | S008 |
| T-14.3 | An invalid book ID is entered when trying to return a book.                                                             | Book ID = "99999"                    | This book does not have an active loan.            | See Return Book video (page 111)<br><br>Link:<br><a href="https://youtu.be/SDo_iG9uAB8">https://youtu.be/SDo_iG9uAB8</a>       | Success | S008 |
| T-14.4 | A book ID of an invalid type is entered when trying to return a book.                                                   | Book ID = "The Chronicles of Narnia" | The Book ID should be an integer                   | See Return Book video (page 111)<br><br>Link:<br><a href="https://youtu.be/SDo_iG9uAB8">https://youtu.be/SDo_iG9uAB8</a>       | Success | S008 |
| T-15.1 | When a valid member ID is entered into the view member loans feature all of their current and previous loans are shown. | Member ID ="1"                       | Information about their current and previous loans | See View Member Loans video (page 111)<br><br>Link:<br><a href="https://youtu.be/tBKKTH-GD58">https://youtu.be/tBKKTH-GD58</a> | Success | S008 |

|        |                                                                                                                                     |                    |                                                      |                                                                                                                                                              |         |      |
|--------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|------|
| T-15.2 | When a user who has not taken out any loans is entered into the feature, it is printed that this user has not had any loans before. | Member ID ="3"     | Printed that this user has not had any loans before. | See View Member Loans video (page 111)<br><br>Link:<br><a href="https://youtu.be/tBKKTH-GD58">https://youtu.be/tBKKTH-GD58</a>                               | Success | S008 |
| T-15.3 | When a user id of an invalid type is entered into the view members loans function                                                   | Member ID ="David" | The member ID should be a number                     | See View Member Loans video (page 111)<br><br>Link:<br><a href="https://www.youtube.com/watch?v=tBKKTH-GD58">https://www.youtube.com/watch?v=tBKKTH-GD58</a> | Success | S008 |

### Stakeholder feedback

| Stakeholder Questions                                                                                                                       | Average Score /5 |
|---------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| On a ranking from 1-5 stars how intuitive did the Graphical User Interface feel, and did you have any issues navigating the user interface? | 3.67             |
| On a ranking from 1-5 stars how do you rate the view member loans feature?                                                                  | 4.33             |
| How would you rank the add and remove books feature of the program and why?                                                                 | 4.00             |
| How would you rank the different searching features of the program on a scale from 1-5 and why?                                             | 4.67             |
| How would you rank the different sorting features of books in the program on a scale from 1-5 and why?                                      | 4.33             |
| How would you rank the loan book and return book features of the program on a scale from 1-5 and why?                                       | 4.67             |

After doing my post development testing, I contacted my stakeholders to gather feedback through the survey of questions outlined before production started to ensure I met the success criteria of my program, and evaluate the useability, functionality, and effectiveness of the system. I also gave them the opportunity to provide extra details of areas of future improvement.

#### Stakeholder 1- Mrs J Oats

##### 1: GUI Usability (4/5)

“I found that the user interface was clean and very straightforward to use. I appreciated that the buttons were clear, and text was readable. I did not get confused about where to go to carry out tasks. The black text on white background theme was comfortable for long periods of time.”

##### 2: View Member Loans (3/5)

“While this feature did work, some of my work was impeded as I was not able to view every member’s loans and only check individual member’s loans. “

##### 3: Add or remove books (4/5)

“This feature worked well, however it would be helpful if it flagged that a duplicate ISBN was used, before submission rather than afterwards.”

##### 4: Searching features (5/5)

“Searching features work well. I especially like the ability that partial matches can be searched for as it is helpful when I only know part of the books title.”

##### 5: Sorting Features (4/5)

“The sorting features by genre alphabetically was helpful when organising the library however different options like sort by author or by popularity would also be helpful.”

##### 6: Loan / Return Features (5/5)

“This feature is good as it is very fast. It is clear in telling me whether a book is already checked out or not which helps me avoid mistakes.”

#### Stakeholder 2: Theo Ebenezer

##### 1: GUI Usability (4/5)

“The layout of the program was simple and clean, and I did not have trouble finding what I needed. I did not have any trouble interacting with the system, and all text was of high contrast and was not affected by my protanopia.”

##### 2: View Member Loans (5/5)

“This feature was helpful as I could see a log of all the books I have taken out, which helped to keep me on top of things.”

##### 3: Add or remove books (N/A)

User did not have access to this feature.

#### 4: Searching features (5/5)

“The searching features worked well, especially for partial matches. The ISBN feature was too exact, perhaps a future improvement could be suggesting close matches.”

#### 5: Sorting Features (4/5)

“The sorting features worked great however could be improved by adding different filters such as linking to a topic or curriculum for my own school use.”

#### 6: Loan / Return Features (4/5)

“Very straightforward. I like that it was effective in telling me if I have accidentally submitted the wrong book ID.”

### Stakeholder 3 Jeremiah Wilkinson

#### 1: GUI Usability (3/5)

“The overall GUI was well designed and thought out. I thought the high visibility mode was good, but it was not perfect. Some parts of the theme did not work correctly which made some of it hard to see. I customisable font size or window size would make the interface better.”

#### 2: View Member Loans (5/5)

“Very Helpful, I could accurately see all the books that I had loaned which were overdue and when I need to return them.”

#### 3: Add or remove books (N/A)

User did not have access to this feature.

#### 4: Searching features (4/5)

“This feature worked well but was quite text heavy, I believe it could have been improved by breaking each result into cards or separating results.”

#### 5: Sorting Features (5/5)

“The sorting by genre feature worked well and has helped me find new books to read.”

#### 6: Loan / Return Features (5/5)

“Simple and worked effectively.”

## Evaluation

### Success Criteria Evaluation and Test Evidence

| Success Criteria ID | Success Criteria                                                                                                                                                                                                                                              | Pass/Fail       | Test reference                                                 | Further Justification                                                                                                                                                                                                                                                                                                                                                                         |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S001                | There will be a graphical user interface where every button works.                                                                                                                                                                                            | Partial Failure | Every part of the program although specifically T-12           | While my system did have a graphical user interface, it was not perfect. For example, there were issues with the user interface being too small and text falling off the page. As well as this, there was issues with the high visibility theme.                                                                                                                                              |
| S002                | There should be a database containing tables for books, users, and loans                                                                                                                                                                                      | Pass            | Linked to every part of the program although specifically T-03 | This was completely met although there were two database files, for book and loans, and users. This enabled all the different functions of the system to work.                                                                                                                                                                                                                                |
| S003                | There should be a function to search for books by name and by their ISBN and then the book and relevant information should be outputted to the user                                                                                                           | Pass            | T-08.1, T-08.2, T-08.3, T-13.1, T-13.2                         | This feature did work s expected. However, this could be improved by separating different books to make them easier to read.                                                                                                                                                                                                                                                                  |
| S004                | There should be a function to sort books by genre and output them alphabetically                                                                                                                                                                              | Pass            | T-10.1, T-10.2, T-10.3                                         | This feature worked as expected and the sort I used to sort the genre in alphabetical order worked correctly.                                                                                                                                                                                                                                                                                 |
| S005                | There should be a log in page                                                                                                                                                                                                                                 | Pass            | All of T-01 and T-02                                           | This feature worked correctly including safety features such as hiding password entry with asterisks.                                                                                                                                                                                                                                                                                         |
| S006                | There should be Role Based Access control (between normal members and privileged librarian accounts) which have access to different features                                                                                                                  | Partial Pass    | All of Test T-01 and T-11                                      | There was some Role-Based Access control with normal members who do not have access to all features and privileged librarians which have access to all features. However this could have been developed with further different types of members, such as split into different tiers and library assistants. There was also not a feature to change a user and librarian for security reasons. |
| S007                | There should be a feature to add or remove books                                                                                                                                                                                                              | Pass            | All of Test T-06 and T-07                                      | This feature worked as expected, with books being correctly added or deleted from the database without affecting other books.                                                                                                                                                                                                                                                                 |
| S008                | There should be a feature to borrow and return books. It should check whether the book is in the database and prevent already checked out books from being checked out again. It should also log which member logged out the book according to its member ID. | Pass            | All of T-09, T-14 and T-15                                     | This feature worked well with books being able to be loaned out or returned and an individual's loans were able to be printed, however this could have been improved by allowing more than just one user's loans to be shown.                                                                                                                                                                 |

|      |                                                         |      |                      |                                                                                                                                                                                                               |
|------|---------------------------------------------------------|------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S009 | There should be a feature to add and remove new members | Pass | All of T-04 and T-05 | This feature worked as expected, with email checking and password security measures. When deleting a user, they were successfully deleted from the system, without affecting the other users in the database. |
|------|---------------------------------------------------------|------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Key** Green = Complete Orange = Progress has been made Red = Not done

## Partial/Unmet Criteria and Future Work

While overall my stakeholders found the Graphical User Interface of my system functional, there were certain issues with certain features such as the toggle high visibility theme which did not work fully. Future work would include searching to find the background of the different widgets to change the background of these widgets accurately. Furthermore, development into personalisation features such as being able to create your own theme, which would be stored and associated with your account allows users to decide what works for them best. To improve visibility, having a scalable window size would also be developed as I was unable to do this due to the difficulty of its integration. This will also reduce the issue of text entries not completely fitting on the page.

I also only partially passed success criteria S006 which was to implement Role-Based Access Control to the system. Firstly, with future work I would print the user who is signed into the system as well as their role for convenience as well as ensuring data integrity. Furthermore, I would create a wider range of different roles to enforce stricter role control strategies. As well as this, while this was not included at least in part for security reasons, I would then develop a feature to promote or demote different accounts, such as being able to change a year 12 account to a year 13 account, rather than having to do this within the database. Due to its security implications, there must be several checks to ensure only an authorised user can implement this change, like librarians such as my stakeholder Mrs J Oats.

## Evidence of Usability Features

| Feature ID | Feature                                         | Pass/Fail    | Related Tests                                        | Justification                                                                                                                                                      |
|------------|-------------------------------------------------|--------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| F1         | User interaction using graphical user interface | Partial Fail | Every part of the program although specifically T-12 | As noted in Success Criteria S001, the Graphical User Interface did not completely work correctly. Further development is needed to make it more intuitive to use. |

|    |                                                                                         |              |                                                                |                                                                                                                                                                                              |
|----|-----------------------------------------------------------------------------------------|--------------|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| F2 | There should a database of books that is edited by the program.                         | Pass         | Linked to every part of the program although specifically T-03 | This feature worked and a new database is generated on the first boot, if required. This was the backbone of the system and so was an essential feature.                                     |
| F3 | There should be and algorithm used to sort and output books- a search and filter system | Pass         | T-08.1, T-08.2, T-08.3, T-13.1, T-13.2, T-10.1, T-10.2, T-10.3 | My program had features to search a book by title and ISBN and sort by genre alphabetically. However, it could have been improved with other sort functions, such as by popularity or author |
| F4 | There should be user authentication allowing users to sign into the program             | Pass         | All of T-01 and T-02                                           | This function worked properly, with different users stored in the database being able to be signed in.                                                                                       |
| F5 | There should be Role Based Access Control (RBAC)                                        | Partial Pass | All of Test T-01 and T-11                                      | This aims to enforce security so unauthorised users cannot tamper with the system or other users' accounts                                                                                   |
| F6 | Cataloguing books                                                                       | Partial Pass | All of Test T-06 and T-07                                      | This feature was very brief, limited to only being able to add or remove books. To improve, a feature to be able to organise the library which is more developed than just sorting by genre. |
| F7 | Borrowing and returning process                                                         | Pass         | All of T-09, T-14 and T-15                                     | This feature was implemented by allowing the user to enter a book id and member id to loan a book, which marks it as loaned, so duplicate loans cannot be taken.                             |

|    |                                               |      |                      |                                                                                                                                                       |
|----|-----------------------------------------------|------|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| F8 | Stock Monitoring                              | Fail | N/A                  | This feature was not implemented as this would affect certain parts of the database, such as the ISBN value as it was supposed to be a unique value.  |
| F9 | Adding and removing members from the database | Pass | All of T-04 and T-05 | This feature was added successfully with new members being able to be added, with a username and valid email or password and old members are deleted. |

### Partially or unmet Usability Features and further development

The main feature that I failed was the stock monitoring feature (Feature F8). This is due to the fact as this would have caused for a redevelopment of the entire books table as duplicate ISBN's are not allowed to be entered into the system. To further develop this I would remake the table so the ISBN field does not have to be unique, so that the system would not crash and each new book can get their own Book ID. Then, I would have a new table which adds a new field for the quantity of books sold so that when a book is loaned, it minuses one from the available book quantity until there are no more and for convenience as it allowed the librarians to accurately see the books which are available.

To help with cataloguing books, a new function will be created to store all returned books from the previous day so the librarian can accurately see which books need to be placed back into the library. This will mean that the return date of a book will need to be logged. This may also be improved by being able to show the area of the library that the book should go.

For further Role-Based access control new roles will be created which can then be used to block users from doing different more specific things. For example, stopping younger members from loaning books or textbooks for older members, particularly during exam time. Another example of this is if the librarian wishes to get student helpers, which then as such need access to certain features without allowing them to have full librarian access. This would also mean a feature should be created to allow a librarian or administrator account to promote or demote different accounts. However, further security measures should be put in place to prevent unauthorised access to this feature.

To develop the graphical user interface in the future, I would like to improve on the overall feel of the program, allowing for greater personalisation, such as allowing the user to create their own themes. This would work using a new database table which allows users to take in hex code values for certain features, such as background or text colour which will then be saved along with their member id so whenever they sign into the system their own personalised theme will be shown.

## Maintenance and Limitations

To make my program easier to maintain I used several comments as well as clear variable names. to allow myself or other developers to be able to undrerstand the code easier making further development easier to continue. Furthermore, my code was very modular, using separate classess such as Book Manager or Member Manager. This helped to keep features isolated, making it easier to update or expand on without having to affect the entire system.

To improve on maintenance, a back up system to back up the contents of the database file would be useful to maintain the system without losing data. Furthermore, I could have used software like Git or Github for release management. Within the development of my system, I simply renamed and resaved the name of my main program file to denote a version change however using software like Git would have given me better tracking of changes, allwoing me to see what has been committed or deleted, as well as being able to include branches away from the main release so I can integrate and test certain new features. This also helps for debugging purposes as it encourages collaboration as it makes it easier for new developers to contribute as rollbacking to previous versions if a bug is found.

## Algorithm Critique

While my library management system uses a quicksort algorithm when searching books by ISBN, before doing a binary search, on a small database the extra computational cost of a quicksort may have made it slower to run than alternatives like insertion sort. Therefore, within further development, testing should include testing the time for certain functions to occur, at different sizes of databases, (such as 10 entries vs 10 thousand vs 1 million) in order to pick the best sort in that case. Similarly, using a binary sort may also have slowed down the system over a linear sort as it needs to be sorted. This may have been improved by sorting the book in the background once on startup, or when a new book is added to the system, meaning the quick sort does not need to called every time. Nonetheless, as there was a requirement for scalability within the program, using a quicksort and binary search can still be considered good however it is importeant to potentially consider an adpative algorithm choice depending on the number of entries within the database.

## Future Development

A hardware contrainst of my program is not being able to have multiple users on the system at once. This is because there is not a server for my program to run on at consistently. To improve, in the future hosting the server online would allow for multiple librarians and students to access the system from different devices, whether theyre at home or actually in the library, instead of just one device. This allows for more features to be created such as being able to reserve books at home and collect them when they are next in the library. However this would also mean developing features to prevent multiple bookings happening at the same time, ensuring data integrity in the database.

Furthermore, as I did not have access to a Barcode or OCR (Optical Character Recognition) scanner I was not able to add this to the system in the form of letting librarians scan the book barcode or book id which is printed physically on the book. This would be a good feature to develop as it prevents human error when loaning out books, and speeds up the time it takes to loan out a book.

Relating to overdue books, it would be good to have develop an automated mailbot which sends out automatic emails reminding users of when books are overdue, meaning the librarian does not physically have to chase these people up in person. This could be furthered by a feature which can callculate the cost of overdue fines by taking in the difference between the current date (date of attempted return) and the actual due date of the loan and multioplying that by a set amount (such as 50p per day).

In the future, further development of accessibility features, such as being able to scale the size of the program, or make a full screen version could further help visually impaired users. For the graphical user interface itself, feature development could include the ability to pull the image of the book's cover from the internet using an ISBN lookup may help the user identify new books in the library. This could be furthered by new sorting or filtering books, such as sorting by popularity or author to allow users to find new books.

Another feature that could be developed in the future could be the implementation of a basic recommendation system which suggests new books to users based on their previous borrowing history and favourite genres by tracking the genres they borrow from the most and suggesting similar books.s

## Full Code

```
import bcrypt
import tkinter as tk
from tkinter import messagebox
import sqlite3
import re
from datetime import datetime, timedelta

def validate_login():
    #take username and password entry
    entry_username = entered_username.get()
    entry_password = entered_password.get()
    # create object of the member manager class and connect to the members table
    member_manager = MemberManager("members2.db")
    get_member = member_manager.get_member_by_username(entry_username)

    if get_member != None:
        #hashes the user entered password using the salt from the hashed password
        if bcrypt.checkpw(entry_password.encode('utf-8'), get_member.hashed_password) == True:
            messagebox.showinfo("Successful Login", "{} has logged into the system, Role is {}".format(get_member.username, get_member.role.capitalize()))
            root.destroy()

            #open library app window and pass in the user's role
            open_library_app(get_member.role)
        else:
            messagebox.showerror("Login Failed", "You have entered an incorrect username or password")
    else:
        messagebox.showerror("Login Failed", "You have entered an incorrect username or password")
```

```
class Member:
    def __init__(self, username, email, hashed_password, role="member"):
        self.username = username
        self.email = email
        self.hashed_password = hashed_password
        self.role = role # 'member' or 'librarian'

#defines book class with public attributes for title, author, genre and ISBN value (all entered as parameters)
class Book:
    def __init__(self, title, author, genre, isbn):
        self.title = title
        self.author = author
        self.genre = genre
        self.isbn = isbn

class DatabaseManager:
    def __init__(self, db_name):
        #connects to database which is passed in a parameter
        self.conn = sqlite3.connect(db_name)
        self.create_tables()
    def create_tables(self):
        pass
    #closes connection with database
    def close(self):
        self.conn.close()
```

```
class MemberManager(DatabaseManager):
    #create the library member table if needed, particularly when run for the first time, prevents error when modifying a database which does not exist.
    def create_tables(self):
        cursor = self.conn.cursor()
        cursor.execute('''CREATE TABLE IF NOT EXISTS members
        (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            email TEXT UNIQUE NOT NULL,
            password TEXT NOT NULL,
            role TEXT NOT NULL
        )''')
        self.conn.commit()

    #function to search the database for a certain user and create a member object with their information
    def get_member_by_username(self, username):
        cursor = self.conn.cursor()
        cursor.execute('''SELECT * FROM members WHERE username = ?''', (username,))
        #creates a list of the user information
        row = cursor.fetchone()
        if row != None: # if the user is found create a member object with their information
            return Member(row[1], row[2], row[3], row[4])
        return None # if no user
```

```

def get_member_by_id(self, member_id):
    cursor = self.conn.cursor()
    cursor.execute('''SELECT * FROM members WHERE id = ?''', (member_id,))
    #creates a list of the user information
    row = cursor.fetchone()
    if row != None: # if the user is found create a member object with their information
        return Member(row[1], row[2], row[3], row[4])
    return None # if no user

def add_member(self, member):
    #connects to database with cursor c
    cursor = self.conn.cursor()
    try:
        # add new user into the members database, using hashed password
        cursor.execute('''INSERT INTO members (username, email, password, role) VALUES (?, ?, ?, ?)''',
            (member.username, member.email, member.hashed_password, member.role))
        self.conn.commit()
        #grabs the primary key from the last row, i.e. the row that was just added.
        user_id = cursor.lastrowid
        return "The member {} has been added to the system. Its Member ID is {}".format(member.username, user_id)
    except sqlite3.IntegrityError: return "The email address or username you have entered is not unique"

```

```

def delete_member(self, member_id): #delete member from database
    cursor = self.conn.cursor()
    cursor.execute('''DELETE FROM members WHERE id = ?''', (member_id,))
    #if no rows containing the member was found, the user does not exist in the database
    if cursor.rowcount == 0:
        return "You have entered an incorrect Member ID."
    else:
        self.conn.commit()
        return "User {} has been deleted".format(member_id)

```

```

class BookManager(DatabaseManager):
    #create the library books tables if needed, particularly when run for the first time, prevents error when modifying a database which does not exist.
    def create_tables(self):
        cursor = self.conn.cursor()
        cursor.execute('''CREATE TABLE IF NOT EXISTS books
            (id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT,
            author TEXT,
            genre TEXT,
            isbn TEXT UNIQUE,
            checked_in INTEGER DEFAULT 1)''')

```

```

        cursor.execute('''CREATE TABLE IF NOT EXISTS loans
            (id INTEGER PRIMARY KEY AUTOINCREMENT,
            book_id INTEGER,
            member_id INTEGER,
            loan_date TEXT,
            due_date TEXT,
            returned INTEGER DEFAULT 0,
            FOREIGN KEY(book_id) REFERENCES books(id),
            FOREIGN KEY(member_id) REFERENCES members(id))''')
        self.conn.commit()

```

```

def add_book(self, newbook): #add book to database

    cursor = self.conn.cursor()
    #attempt to add the book to the books table error from sqlite3 integrity error as the ISBN (the book) is already in the database
    try:
        cursor.execute('''INSERT INTO books (title, author, genre, isbn) VALUES (?, ?, ?, ?)''',
            (newbook.title, newbook.author, newbook.genre, newbook.isbn))
        self.conn.commit()
        book_id = cursor.lastrowid
        return "The book {} has been added to the system. Its Book ID is {}".format(newbook.title, book_id)
    except sqlite3.IntegrityError: #ISBN must be unique
        return "You have entered an ISBN which is already in the database."

```

```

def delete_book(self, delete_book_id): #delete book from database using a search of the book id
    cursor = self.conn.cursor()
    cursor.execute('''DELETE FROM books WHERE id = ?''', (delete_book_id,))
    #if no rows containing the book ID was found, the book does not exist in the database
    if cursor.rowcount == 0:
        return "A book with this Book ID does not exist."
    else:
        self.conn.commit()
        return "The book with ID {} has been deleted".format(delete_book_id)

def search_book_by_title(self, title):# search for books according to the title
    cursor = self.conn.cursor()
    #LIKE so doesnt have to be a direct match (allows for wildcards)
    cursor.execute('''SELECT * FROM books WHERE title LIKE ?''', ('%' + title + '%',))
    filteredbooks = cursor.fetchall()
    return filteredbooks

def fetch_all_books(self):
    cursor = self.conn.cursor()
    cursor.execute('''SELECT * FROM books''')# return everything (*) from the books database
    rows = cursor.fetchall()
    #initialises empty list of books and then appends new book objects to the end of that list
    all_books=[]
    for i in range(len(rows)):
        all_books.append(Book(rows[i][1], rows[i][2], rows[i][3], rows[i][4]))

    return all_books

```

```

def quick_sort(self, books, low, high):
    if (low < high):
        #like a pre order traversal
        pivot=self.partition(books,low, high) # get the index of the pivot
        self.quick_sort(books,low, pivot-1) # sort the left subset
        self.quick_sort(books, pivot+1, high)# sort the right subset
    return books

def partition(self,books,low, high):
    pivot=books[low] #first book is the pivot
    leftwall=low#boundary between elements smaller and greater than the pivot
    for i in range(low+1,high+1):
        if books[i].isbn < pivot.isbn:# if curent element is less than the pivot move it to the left of the leftwall
            leftwall +=1
            #swap the current element and the element at index leftwall
            temp=books[i]
            books[i]=books[leftwall]
            books[leftwall]=temp
    #swap the pivot element with the element at the elft wall
    temp=books[low]
    books[low] = books[leftwall]
    books[leftwall]=temp
    #return index of the pivot
    return leftwall

```

```

def binary_search(self, isbn):
    allbooks = self.fetch_all_books() # grabs all books, making sure they're all objects
    low=0
    high = len(allbooks) - 1
    allbooks = self.quick_sort(allbooks,low,high) # sorts all the books for isbn (binary search must be in order)
    while low <= high:
        mid = (low + high) // 2
        if allbooks[mid].isbn == isbn: #if the mid index isbn = target, it is found
            return allbooks[mid]
        elif allbooks[mid].isbn < isbn:#cut off lower half of the array
            low = mid + 1
        else:
            high = mid - 1 #cut of larger half of the array
    return None

def sort_books_by_genre(self, genre):
    books= self.fetch_all_books()#grabs all the books
    sortedbooks=[]
    for i in range (len(books)):#linear search, if the book's genre matches the target genre add it to the sortedbooks list
        if books[i].genre == genre:
            sortedbooks.append(books[i])
    return sortedbooks

```

```

def merge(self,sortedbooks, low, mid, high):
    left = sortedbooks[low:mid + 1] #create temporary list for left half
    right=sortedbooks[mid+1:high+1] #create temporary list for right half
    loc1 = 0 #pointer for left array
    loc2 = 0 # pointer for right array
    locM = low # pointer for merged array

    while loc1 < len(left) and loc2 < len(right):
        #if the current array in the left half is smaller than the one in the right
        if left[loc1].title.strip().lower() <= right[loc2].title.strip().lower():
            sortedbooks[locM] = left[loc1] #put in the merged array
            loc1 += 1 # increment the pointer in the left half
        else:
            sortedbooks[locM] = right[loc2]#put in the merged array
            loc2 += 1 #increment the pointer in the right half
            locM += 1 #increment pointer in merged array
    #if there are any remaining add to merged array
    while loc1 < len(left):
        sortedbooks[locM] = left[loc1]
        locM += 1
        loc1 += 1
    while loc2 < len(right):
        sortedbooks[locM] = right[loc2]
        loc2 += 1
        locM += 1

```

```

def merge_sort(self,sortedbooks, low, high):
    if high > low:
        mid = (low + high) // 2
        #post order
        self.merge_sort(sortedbooks, low, mid,)#sort left half recursively
        self.merge_sort(sortedbooks, mid + 1, high,) #sort right half recursively
        self.merge(sortedbooks, low, mid, high)#combine both halves
    return sortedbooks

```

```

def loan_book(self, book_id, member_id):
    cursor = self.conn.cursor()
    #SQL query to see whether the book is out on loan
    cursor.execute('''SELECT checked_in FROM books WHERE id = ?''', (book_id,))
    book = cursor.fetchone()
    #the user has entered an incorrect book id, the book is not in the database
    if book == None:
        return "You have entered an incorrect Book ID."
    #if the checked in field is 0/False, then the book is currently on loan
    if book[0] == 0:
        return "This book is currently out on loan."

    loan_date = datetime.today().strftime("%d-%m-%Y") # the loan date, today
    due_date = (datetime.today() + timedelta(days=14)).strftime("%d-%m-%Y") # 14 days from loan date

    cursor.execute('''INSERT INTO loans (book_id, member_id, loan_date, due_date) VALUES (?, ?, ?, ?)''',
        (book_id, member_id, loan_date, due_date))
    # adds the book, member, loan date and due date in the loans table
    cursor.execute('''UPDATE books SET checked_in = 0 WHERE id = ?''', (book_id,)) #change book to loaned out
    self.conn.commit()
    return "You have successfully loaned out a book, your loan is due on {}".format(due_date)

```

```

def return_book(self, book_id): #Function to return a loaned book
    cursor = self.conn.cursor()
    cursor.execute('''UPDATE loans SET returned = 1 WHERE book_id = ? AND returned = 0''', (book_id,))
    #if not found in the loans database or has been returned already
    if cursor.rowcount == 0:
        return "This book is not currently loaned out."
    else:
        #change loan returned field to 1/True
        cursor.execute('''UPDATE books SET checked_in = 1 WHERE id = ?''', (book_id,)) #change book to returned
        self.conn.commit()
        return "You have successfully returned the book with Book ID {}".format(book_id)

def get_member_loans(self, member_id):#get a log of all the user's loans, (current and previous)
    cursor = self.conn.cursor()
    #return the id of the book, its title, its author, the due date and whether it has been returned
    #join both the loans and books table, with the book id as the foreign key
    cursor.execute('''SELECT books.id, books.title, books.author, loans.due_date, loans.returned
        FROM loans
        JOIN books ON loans.book_id = books.id
        WHERE loans.member_id = ?''', (member_id,))
    loans = cursor.fetchall()
    return loans

```

```

class LibraryApp:
    def __init__(self, root, role):
        #starts both the book and member managers
        self.member_manager = MemberManager('members2.db')
        self.book_manager = BookManager('books2.db')

        self.root = root# contains the Tkinter root window

        self.role = role #store role of the logged in user

        self.root.title("Library Management System")
        self.is_high_contrast = False #Boolean value to check whether the user is using the high visibility mode or not (initially not)
        self.frame = tk.Frame(self.root) #the main frame for all the UI widgets
        self.create_widgets()

```

```

def create_widgets(self):
    self.frame.pack(padx=10, pady=10)
    tk.Label(self.frame, text="Library Management System", font=("Arial", 16)).grid(row=0, columnspan=2, pady=10)
    tk.Button(self.frame, text="Add Book", command=self.add_book).grid(row=1, column=0, padx=5, pady=5)
    tk.Button(self.frame, text="Delete Book", command=self.delete_book).grid(row=1, column=1, padx=5, pady=5)
    tk.Button(self.frame, text="Add Member", command=self.add_member).grid(row=4, column=0, padx=5, pady=5)
    tk.Button(self.frame, text="View Member Loans", command=self.view_member_loans).grid(row=5, column=0, padx=5, pady=5)
    tk.Button(self.frame, text="Delete Member", command=self.delete_member).grid(row=4, column=1, padx=5, pady=5)
    tk.Button(self.frame, text="Search Book by Title", command=self.search_book_by_title).grid(row=2, column=0, padx=5, pady=5)
    tk.Button(self.frame, text="Search Book by ISBN", command=self.search_book_by_isbn).grid(row=2, column=1, padx=5, pady=5)
    tk.Button(self.frame, text="Loan Book", command=self.loan_book).grid(row=3, column=0, padx=5, pady=5)
    tk.Button(self.frame, text="Return Book", command=self.return_book).grid(row=3, column=1, padx=5, pady=5)
    tk.Button(self.frame, text="Sort Books Alphabetically", command=self.sort_books_alphabetically_by_genre).grid(row=5, column=1, padx=5, pady=5)
    tk.Button(self.frame, text="Toggle High Visibility", command=self.toggle_high_contrast).grid(row=6, columnspan=2, padx=5, pady=5)
    tk.Button(self.frame, text="Exit", command=self.root.destroy).grid(row=7, columnspan=2, padx=5, pady=5)

    elif self.role == 'member': # will only show if on a member account
        self.frame.pack(padx=10, pady=10)
        tk.Label(self.frame, text="Library Management System", font=("Arial", 16)).grid(row=0, columnspan=2, pady=10)
        tk.Button(self.frame, text="Search Book by Title", command=self.search_book_by_title).grid(row=1, column=0, padx=5, pady=5)
        tk.Button(self.frame, text="Search Book by ISBN", command=self.search_book_by_isbn).grid(row=1, column=1, padx=5, pady=5)
        tk.Button(self.frame, text="Loan Book", command=self.loan_book).grid(row=2, column=0, padx=5, pady=5)
        tk.Button(self.frame, text="Return Book", command=self.return_book).grid(row=2, column=1, padx=5, pady=5)
        tk.Button(self.frame, text="Sort Books Alphabetically", command=self.sort_books_alphabetically_by_genre).grid(row=3, column=0, padx=5, pady=5)
        tk.Button(self.frame, text="Toggle High Visibility", command=self.toggle_high_contrast).grid(row=3, column=1, columnspan=2, padx=5, pady=5)
        tk.Button(self.frame, text="Exit", command=self.root.destroy).grid(row=4, padx=5, pady=5)

```

```

def add_member(self):
    def submit():
        username = name_entry.get().strip() #remove any whitespace
        email = email_entry.get().strip()
        password = password_entry.get()

        if username == "" or email == "":
            messagebox.showerror("Invalid Entry", "You need to enter an entry for all fields.")
        #regular expression, to validate whether an email address was entered
        email_regex = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'

        #ensuring valid username and password
        if re.match(email_regex, email)==None:
            messagebox.showerror("Invalid Email", "You have entered an invalid email address.")
            return

        if len(password) < 8: # make sure the password is at least 8 characters long
            messagebox.showerror("Weak Password", "Your password should be at least 8 characters long.")
            return

        #hash the entered password
        hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())

        # creates a new member class
        member = Member(username, email, hashed_password)

        # add the member to the database
        result = self.member_manager.add_member(member)
        messagebox.showinfo("Result", result)

        add_member_window.destroy()

    # create new window
    add_member_window = tk.Toplevel(self.root)
    add_member_window.title("Add Member")

```

```

tk.Label(add_member_window, text="Username").grid(row=0, column=0, padx=5, pady=5)
name_entry = tk.Entry(add_member_window)
name_entry.grid(row=0, column=1, padx=5, pady=5)

tk.Label(add_member_window, text="Email").grid(row=1, column=0, padx=5, pady=5)
email_entry = tk.Entry(add_member_window)
email_entry.grid(row=1, column=1, padx=5, pady=5)

tk.Label(add_member_window, text="Password").grid(row=2, column=0, padx=5, pady=5)
password_entry = tk.Entry(add_member_window, show="*")
password_entry.grid(row=2, column=1, padx=5, pady=5)

tk.Button(add_member_window, text="Submit", command=submit).grid(row=3, columnspan=2, pady=10)

def delete_member(self):
    # code for deleting memeber from member database using member ID
    def submit():
        try: member_id = int(member_id_entry.get())
        except ValueError:
            messagebox.showerror("Invalid Entry", "The member ID should be a number.")
            return
        result = self.member_manager.delete_member(member_id)
        messagebox.showinfo("Result", result)
        delete_member_window.destroy()

    delete_member_window = tk.Toplevel(self.root)
    delete_member_window.title("Delete Member")

    tk.Label(delete_member_window, text="Member ID").grid(row=0, column=0, padx=5, pady=5)
    member_id_entry = tk.Entry(delete_member_window)
    member_id_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Button(delete_member_window, text="Submit", command=submit).grid(row=1, columnspan=2, pady=10)

```

```

def add_book(self):
    def submit():
        #takes in user entry
        title = title_entry.get()
        author = author_entry.get()
        genre = genre_entry.get()
        isbn = isbn_entry.get()
        if title=="" or author=="" or genre=="" or isbn=="":
            messagebox.showerror("Invalid Input", "You must enter entries for every field.")
            return
        #creates new book object
        book = Book(title, author, genre, isbn)
        #attempts to insert it into the database
        result = self.book_manager.add_book(book)
        messagebox.showinfo("Result", result)
        add_book_window.destroy()

    add_book_window = tk.Toplevel(self.root)
    add_book_window.title("Add Book")

    tk.Label(add_book_window, text="Title").grid(row=0, column=0, padx=5, pady=5)
    title_entry = tk.Entry(add_book_window)
    title_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Label(add_book_window, text="Author").grid(row=1, column=0, padx=5, pady=5)
    author_entry = tk.Entry(add_book_window)
    author_entry.grid(row=1, column=1, padx=5, pady=5)

    tk.Label(add_book_window, text="Genre").grid(row=2, column=0, padx=5, pady=5)
    genre_entry = tk.Entry(add_book_window)
    genre_entry.grid(row=2, column=1, padx=5, pady=5)

    tk.Label(add_book_window, text="ISBN").grid(row=3, column=0, padx=5, pady=5)
    isbn_entry = tk.Entry(add_book_window)
    isbn_entry.grid(row=3, column=1, padx=5, pady=5)

```

```

tk.Button(add_book_window, text="Submit", command=submit).grid(row=4, columnspan=2, pady=10)

def delete_book(self):
    # function to delete book from database
    def submit():
        try: book_id = int(book_id_entry.get())
        except ValueError:
            #if the user did not enter an integer
            messagebox.showerror("Invalid Input", "Book ID must be a number.")
            return
        #calls delete book function to remove the book id from the database
        result = self.book_manager.delete_book(book_id)
        messagebox.showinfo("Result", result)
        delete_book_window.destroy()

    delete_book_window = tk.Toplevel(self.root)
    delete_book_window.title("Delete Book")

    tk.Label(delete_book_window, text="Book ID").grid(row=0, column=0, padx=5, pady=5)
    book_id_entry = tk.Entry(delete_book_window)
    book_id_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Button(delete_book_window, text="Submit", command=submit).grid(row=1, columnspan=2, pady=10)

```

```

def search_book_by_title(self):
    # code to search for a book according to its title
    def submit():
        booktitle = title_entry.get()
        result.delete(1.0, tk.END)# clear old results
        #2d list containing filtered books
        filteredbooks = self.book_manager.search_book_by_title(booktitle)#find the book() using a sql like query, function from book manager
        if filteredbooks == []:#if book not found and so list is empty
            result.insert(tk.END, "No books found.")
        else:
            for i in range (len(filteredbooks)):
                #checks the status of the current book
                if filteredbooks[i][5]== 1:
                    status="Checked In"
                else:
                    status = "Checked Out"
                result.insert(tk.END, "ID: {}, Title: {}, Author: {}, Genre: {}, ISBN: {}, Status: {}\n"
                    .format(filteredbooks[i][0],filteredbooks[i][1],filteredbooks[i][2],filteredbooks[i][3],filteredbooks[i][4],status))

    search_book_window = tk.Toplevel(self.root)
    search_book_window.title("Search Book by Title")

    tk.Label(search_book_window, text="Title").grid(row=0, column=0, padx=5, pady=5)
    title_entry = tk.Entry(search_book_window)
    title_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Button(search_book_window, text="Submit", command=submit).grid(row=1, columnspan=2, pady=10)

    result= tk.Text(search_book_window, width=60, height=15)
    result.grid(row=2, columnspan=2, padx=5, pady=5)

```

```

def search_book_by_isbn(self):
    def submit():
        searched_isbn = isbn_entry.get()
        book = self.book_manager.binary_search(searched_isbn)
        result.delete(1.0, tk.END)
        if book == None:# if book is not found
            result.insert(tk.END, "No book found with that ISBN.")
        else:
            result.insert(tk.END, "Title: {}, Author: {}, Genre: {}, ISBN: {}\n"
                .format(book.title,book.author,book.genre,book.isbn))

    search_book_window = tk.Toplevel(self.root)
    search_book_window.title("Search Book by ISBN")

    tk.Label(search_book_window, text="ISBN").grid(row=0, column=0, padx=5, pady=5)
    isbn_entry = tk.Entry(search_book_window)
    isbn_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Button(search_book_window, text="Submit", command=submit).grid(row=1, columnspan=2, pady=10)

    result = tk.Text(search_book_window, width=60, height=15)
    result.grid(row=2, columnspan=2, padx=5, pady=5)

```

```

def sort_books_alphabetically_by_genre(self):
    def submit():
        genre = genre_entry.get().strip()
        books = self.book_manager.fetch_all_books()# fetch the list of all books in the database
        if genre != "":
            books=self.book_manager.sort_books_by_genre(genre)#filter for all books of that genre

        sortedbooks = self.book_manager.merge_sort(books,0, len(books)-1)#merge sort to get it into alphabetical order so it can be placed

        result.delete(1.0, tk.END)

        # if there are no books within that genre
        if sortedbooks == []:
            result.insert(tk.END, "No books found of the genre {}.".format(genre))
        else:
            # print all of the books and its data in order
            for i in range (len(sortedbooks)):
                result.insert(tk.END, "Title: {}, Author: {}, Genre: {}, ISBN: {}\n"
                    .format(sortedbooks[i].title, sortedbooks[i].author,sortedbooks[i].genre,sortedbooks[i].isbn))

    sort_books_window = tk.Toplevel(self.root)
    sort_books_window.title("Sort Books Alphabetically")

    tk.Label(sort_books_window, text="Genre").grid(row=0, column=0, padx=5, pady=5)
    genre_entry = tk.Entry(sort_books_window)
    genre_entry.grid(row=0, column=1, padx=5, pady=5)
    tk.Button(sort_books_window, text="Sort Books", command=submit).grid(row=3, column=0, columnspan=2, pady=10)

    #text widget for the results
    result = tk.Text(sort_books_window, width=60, height=15)
    result.grid(row=1, columnspan=2, padx=5, pady=5)

```

```

def loan_book(self):
    # code/function for loaning a book
    def submit():
        try: book_id = book_id_entry.get()
        except ValueError:
            messagebox.showerror("Invalid Entry", "The book ID should be a number.")
            return
        try: member_id = int(member_id_entry.get())
        except ValueError:
            messagebox.showerror("Invalid Entry", "The member ID should be a number.")
            return

        is_valid_user= self.member_manager.get_member_by_id(member_id)
        if is_valid_user==None:
            messagebox.showerror("Invalid Input", "You have entered an incorrect Member ID.")
            return
        result = self.book_manager.loan_book(book_id, member_id)
        messagebox.showinfo("Result", result)
        loan_book_window.destroy()

    loan_book_window = tk.Toplevel(self.root)
    loan_book_window.title("Loan Book")

    tk.Label(loan_book_window, text="Book ID").grid(row=0, column=0, padx=5, pady=5)
    book_id_entry = tk.Entry(loan_book_window)
    book_id_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Label(loan_book_window, text="Member ID").grid(row=1, column=0, padx=5, pady=5)
    member_id_entry = tk.Entry(loan_book_window)
    member_id_entry.grid(row=1, column=1, padx=5, pady=5)

    tk.Button(loan_book_window, text="Submit", command=submit).grid(row=2, columnspan=2, pady=10)

```

```

def return_book(self):
    # function for returning a book
    def submit():
        try: book_id = int(book_id_entry.get())
        except ValueError:
            messagebox.showerror("Invalid Entry", "The book ID should be a number.")
            return
        result = self.book_manager.return_book(book_id)
        messagebox.showinfo("Result", result)
        return_book_window.destroy()

    return_book_window = tk.Toplevel(self.root)
    return_book_window.title("Return Book")

    tk.Label(return_book_window, text="Book ID").grid(row=0, column=0, padx=5, pady=5)
    book_id_entry = tk.Entry(return_book_window)
    book_id_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Button(return_book_window, text="Submit", command=submit).grid(row=1, columnspan=2, pady=10)

```

```

def view_member_loans(self):#function to view all loans from one member
    def submit():
        try: member_id = int(member_id_entry.get())
        except ValueError:
            messagebox.showerror("Invalid Entry", "The Member ID should be a number.")
            return
        loans = self.book_manager.get_member_loans(member_id)
        result.delete(1.0, tk.END)
        if loans == []:#if the loans list is empty
            result.insert(tk.END, "There are no loans for this member.")
        else:
            for i in range(len(loans)):
                #references the specific returned books field and gives a text indicator/ "status"
                if loans[i][4] == 1:
                    status= "Returned"
                else:
                    status = "Not Returned"
                result.insert(tk.END, "Book ID: {}, Title: {}, Author: {}, Due Date: {}, Status: {}\n".
                    format(loans[i][0], loans[i][1],loans[i][2],loans[i][3],status))

    view_loans_window = tk.Toplevel(self.root)
    view_loans_window.title("View Member Loans")

    tk.Label(view_loans_window, text="Member ID").grid(row=0, column=0, padx=5, pady=5)
    member_id_entry = tk.Entry(view_loans_window)
    member_id_entry.grid(row=0, column=1, padx=5, pady=5)

    tk.Button(view_loans_window, text="Submit", command=submit).grid(row=1, columnspan=2, pady=10)

    result = tk.Text(view_loans_window, width=60, height=15)
    result.grid(row=2, columnspan=2, padx=5, pady=5)

```

```

def toggle_high_contrast(self):
    #swaps between high visibility/contrast theme and normal theme
    if self.is_high_contrast==True:
        self.disable_high_contrast()
        self.is_high_contrast= False
    else:
        self.enable_high_contrast()
        self.is_high_contrast= True

def enable_high_contrast(self):
    apply_high_contrast(self.root)
    self.root.configure(background="black")
    self.frame.configure(bg="black")

def disable_high_contrast(self):
    apply_normal_contrast(self.root)
    self.root.configure(background="SystemButtonFace")
    self.frame.configure(bg="SystemButtonFace")

```

```

def open_library_app(role):
    #open the library app
    library_root = tk.Tk()
    app = LibraryApp(library_root, role)
    library_root.mainloop()

```

```

def apply_high_contrast(widget):
    #changes all items in the program to yellow text with a black background
    #Try to change the background to black and the text to yellow or skip as certain widgets do not support this
    try: widget.configure(bg="black", fg="yellow")
    except: pass
    try: widget.configure(insertbackground="yellow")
    except: pass
    #recursively apply this colour scheme to all widgets
    children=widget.winfo_children()
    for i in range (len(children)):
        apply_high_contrast(children[i])

def apply_normal_contrast(widget):
    #return all the items in the program to black text on white background
    try:widget.configure(bg="SystemButtonFace", fg="black")
    except: pass
    try: widget.configure(insertbackground="black")
    except: pass
    #recursively apply this colour scheme to all widgets
    children=widget.winfo_children()
    for i in range (len(children)):
        apply_normal_contrast(children[i])

```

```

root = tk.Tk()
root.title("Login Screen")
root.geometry("300x200")

# username entry
label_username = tk.Label(root, text="Username:")
label_username.pack(padx=5, pady=5)
entered_username = tk.Entry(root)
entered_username.pack(padx=5, pady=5)

# password entry
label_password = tk.Label(root, text="Password:")
label_password.pack(padx= 5,pady=5)
entered_password = tk.Entry(root, show="*") #show asterisks so you cant see the password
entered_password.pack(padx=5, pady=5)

#login button
login_button = tk.Button(root, text="Login", command=validate_login)
login_button.pack(pady=20)
#tkinter main loop
root.mainloop()

```